



POLITÉCNICA

"Ingeniamos el futuro"

CAMPUS
DE EXCELENCIA
INTERNACIONAL



23/09/2025

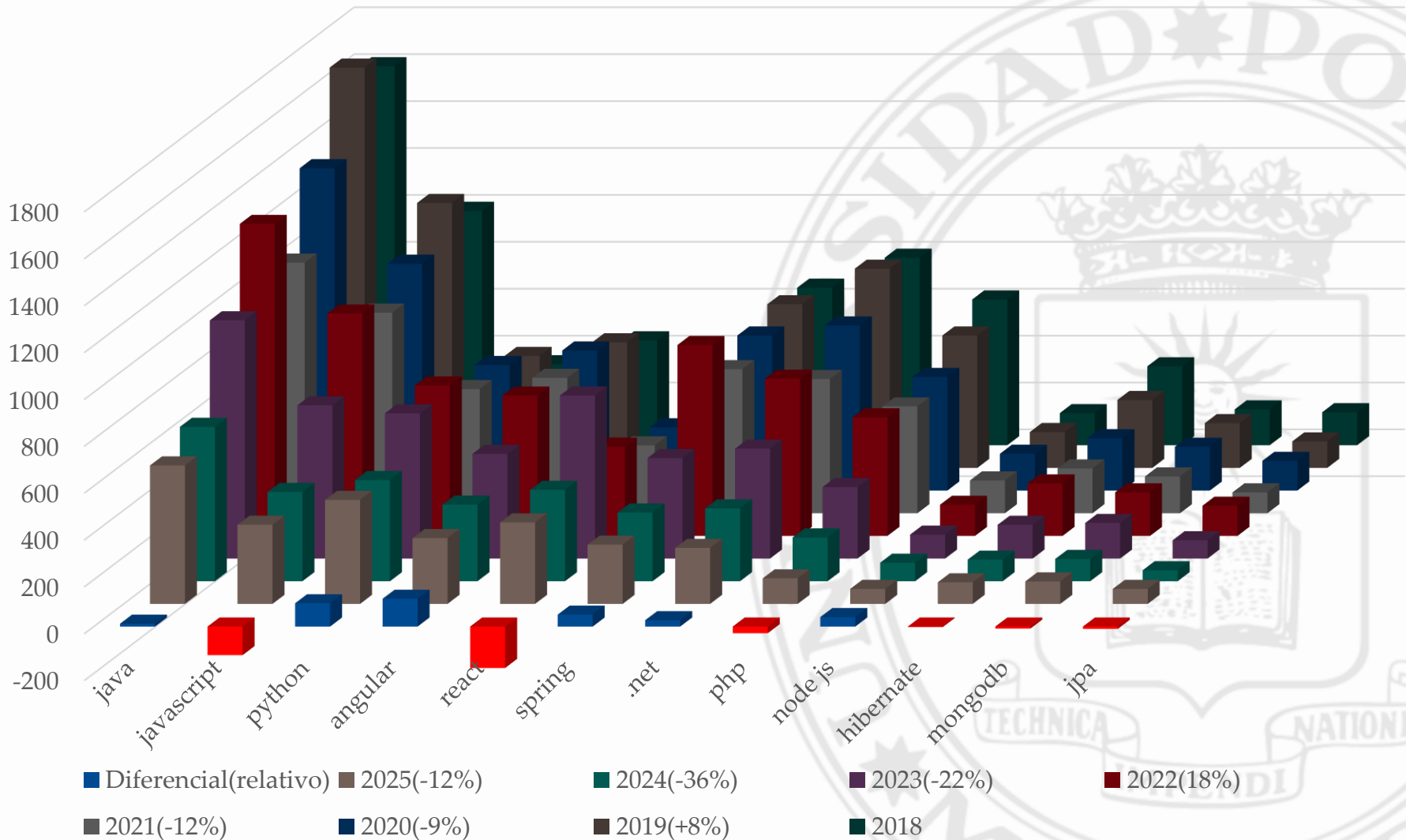
Back-end con Tecnologías de Código Abierto -BETCA-

Spring

Jesús Bernal Bermúdez

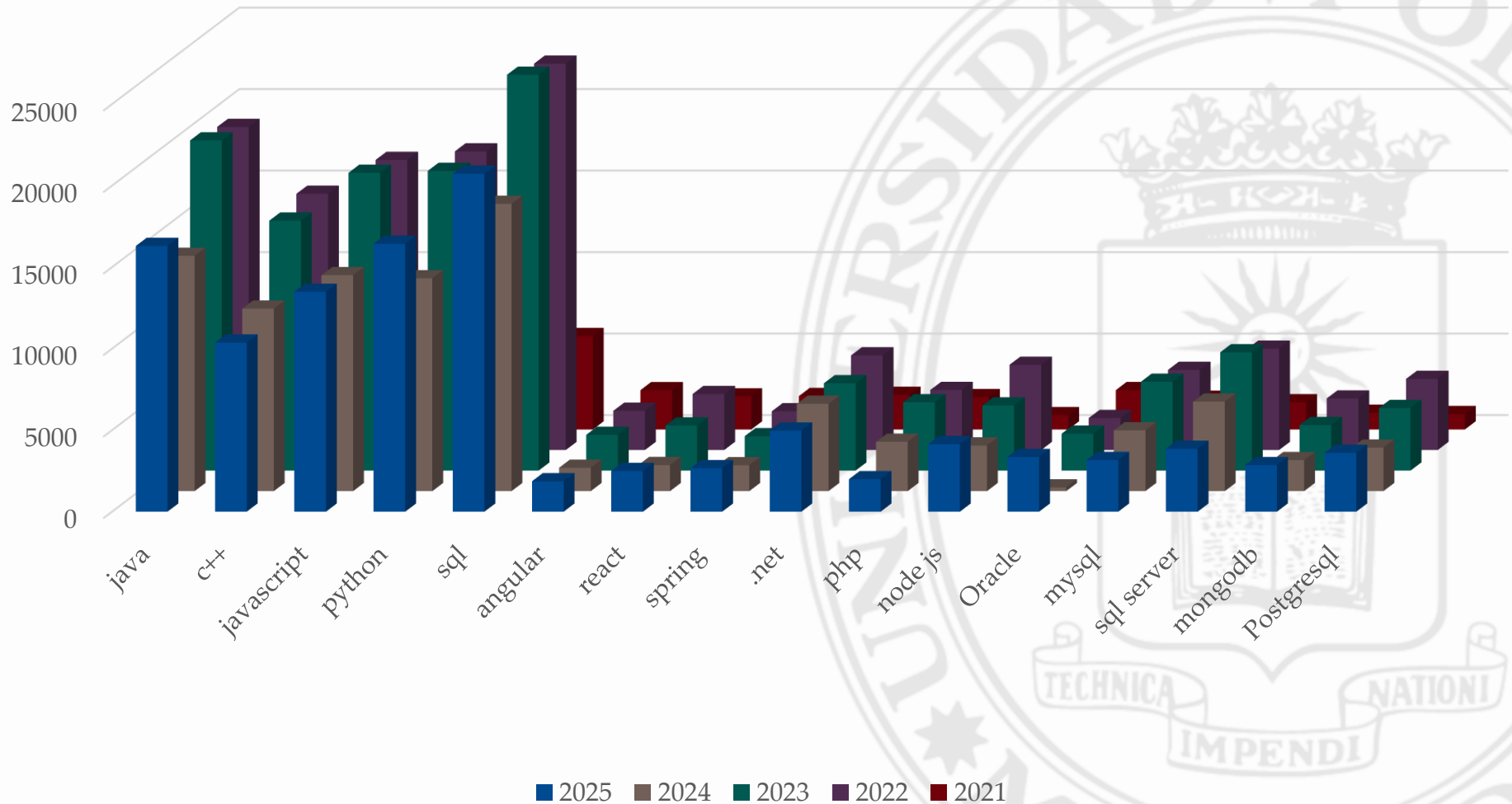
Tendencia del Mercado

InfoJobs (03-02-2025)



Tendencia del Mercado

Linkedin (03-02-2025)

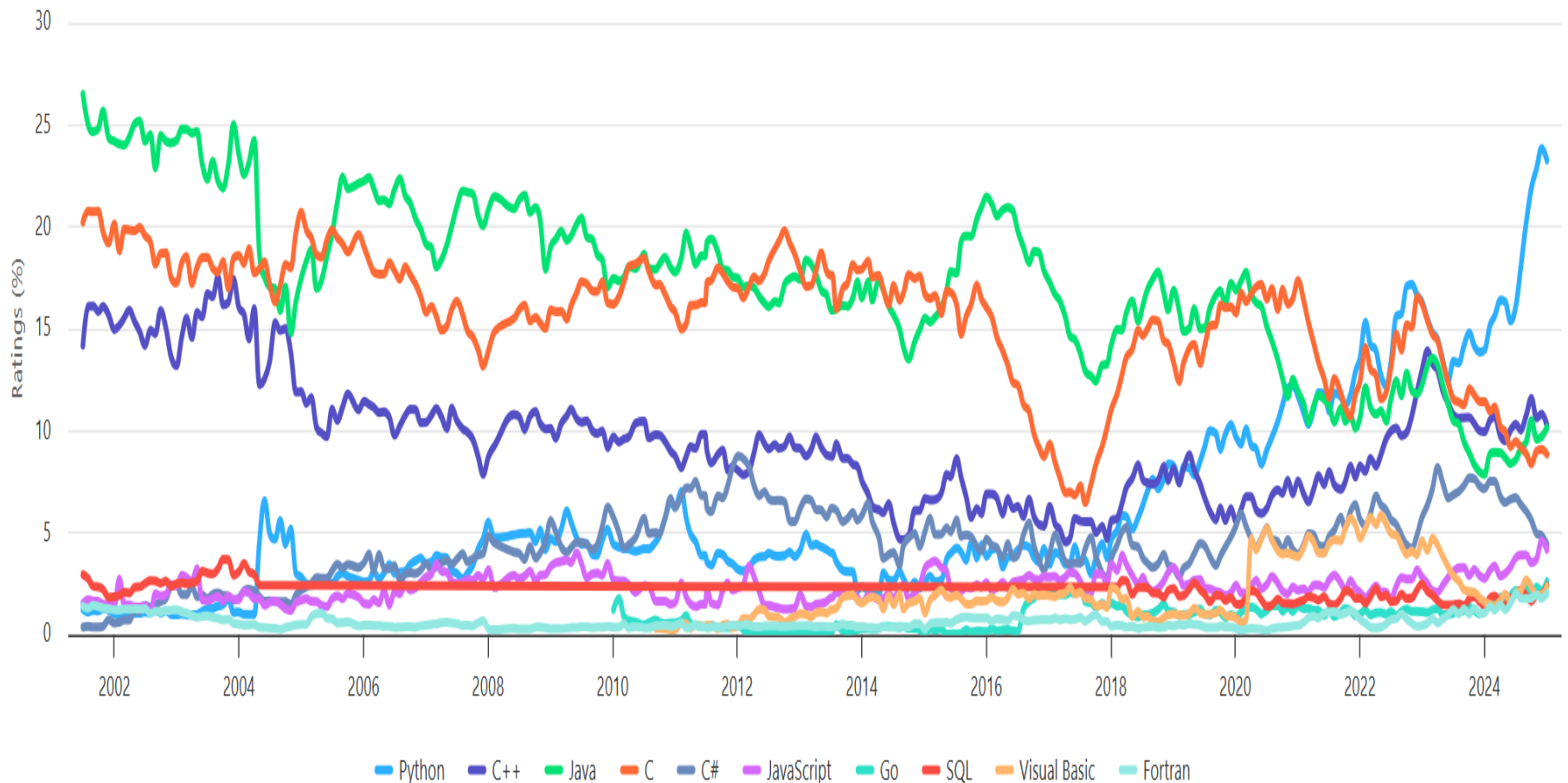


Tendencia del Mercado

TIOBE (03-02-2025)

TIOBE Programming Community Index

Source: www.tiobe.com





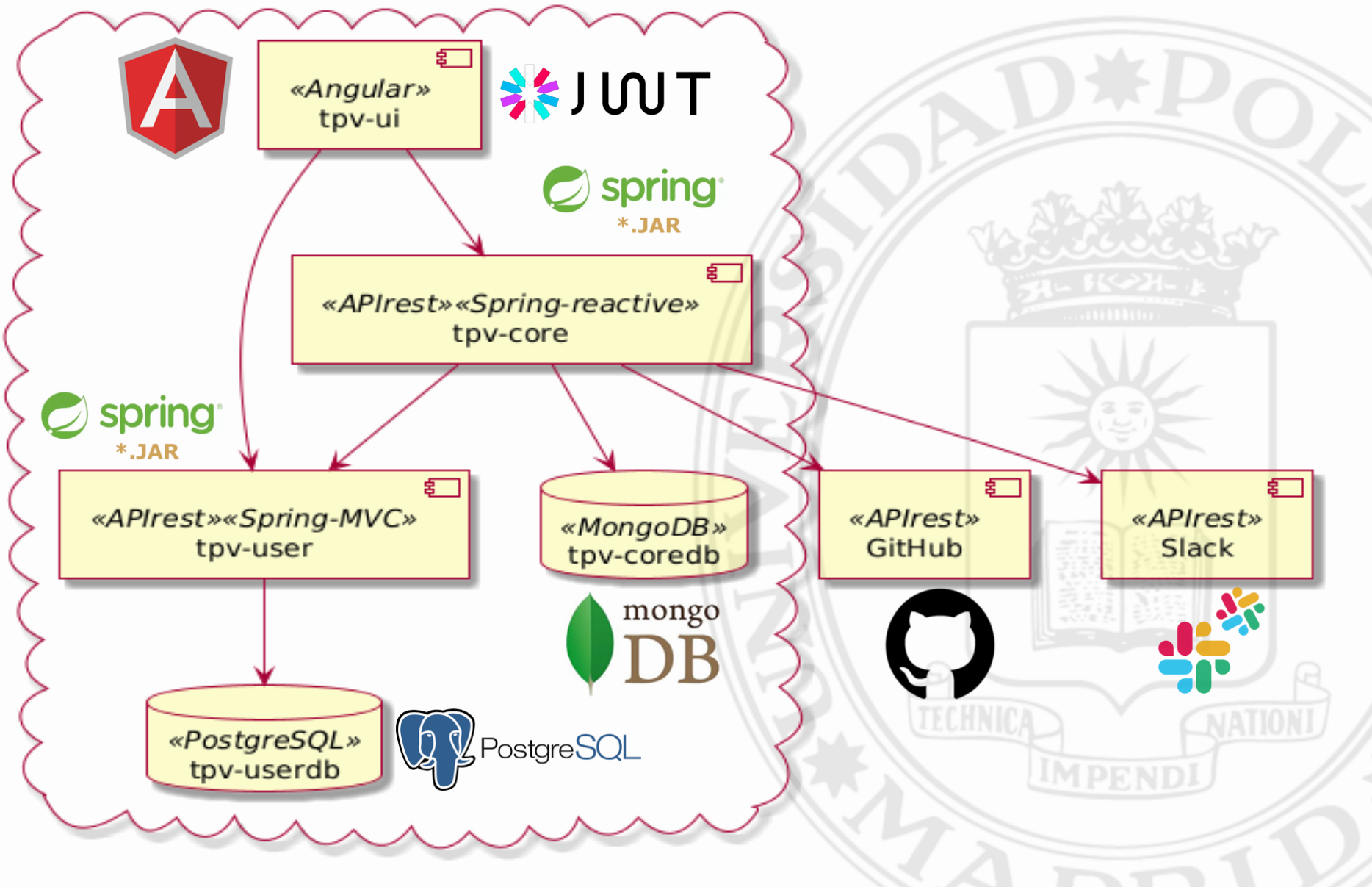
TPV

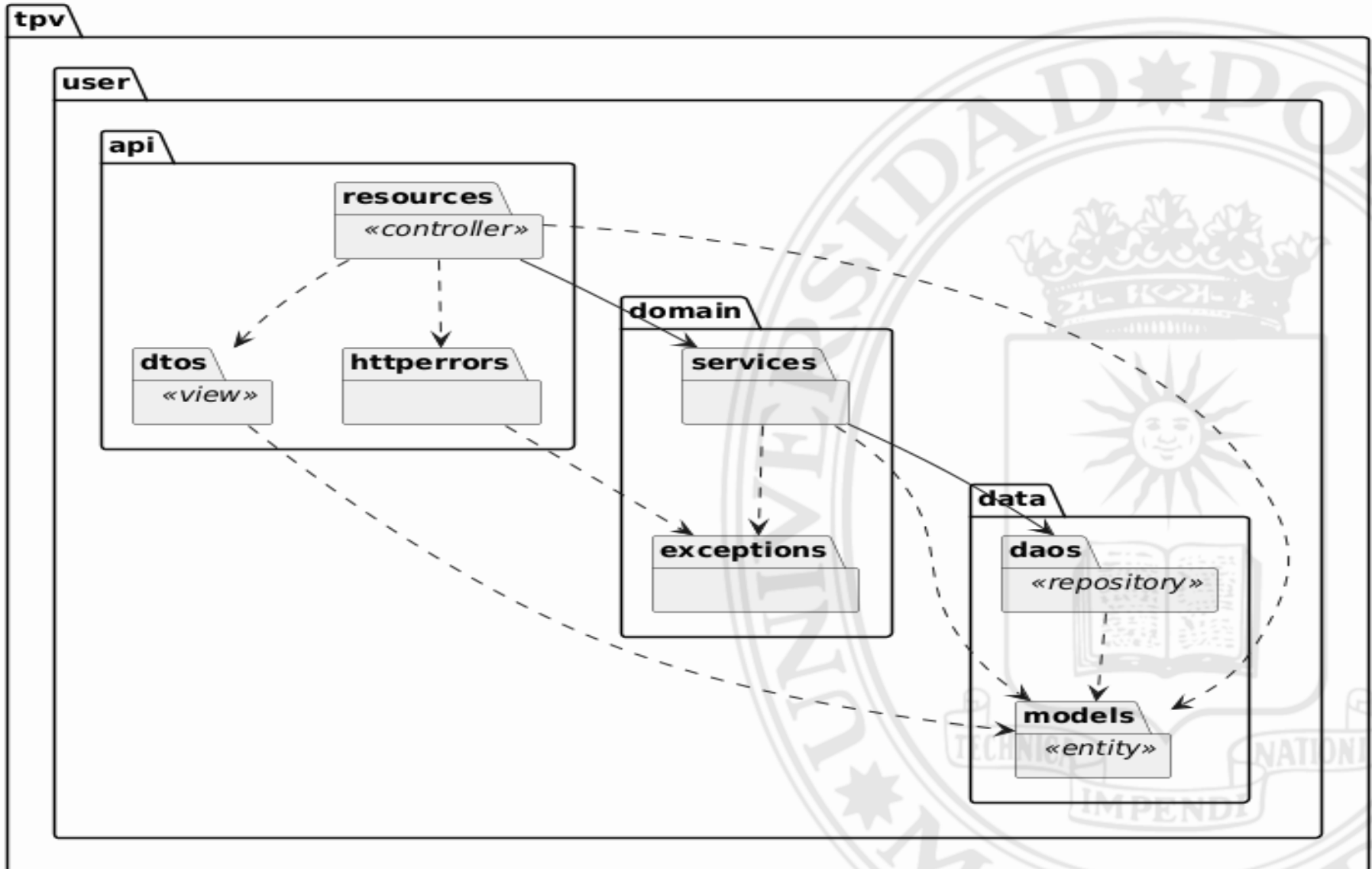
Terminal Punto Venta

Aplicación Web

Full-Stack

- Front-end: *Typescript-Angular (reactivo)*
- Back-end: *Java-Spring: MVC (funcional)*
- Back-end: *Java-Spring: Rective-Funcional*
- Database: *PostgreSQL & Mongodb*





tpv

core

infrastructure

mongodb

«adapter-OUT»

persistence

repositories

«asynchronous»

synchronous

entities

api

«adapter-IN»

resources

«controller»

httperrors

dtos

«view»

restgateway

«adapter-OUT»



RestGatewayClient

domain

services

exceptions

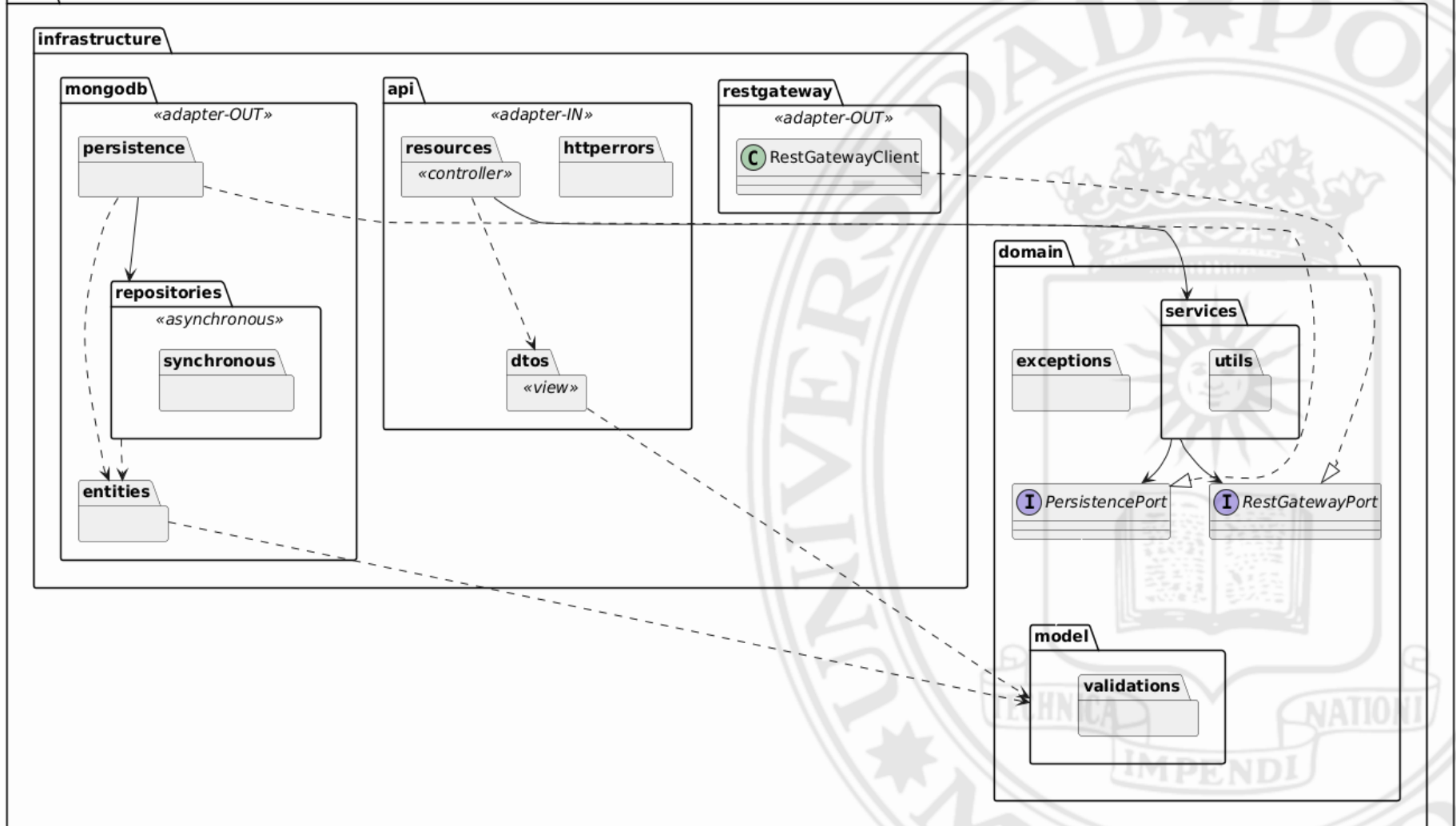
utils

PersistencePort

RestGatewayPort

model

validations



BETCA

Entrega Continua (*Continuous Delivery*)

Práctica del
desarrollo del
software para
liberar código
en ciclos
cortos

El objetivo es que las mejoras del desarrollo sean visibles para el cliente lo antes posible.

Despliegue Continuo

(*Continuous Deployment*) resulta cuando las entregas se despliegan automáticamente

BETCA

New PostgreSQL



Dashboard

tpv-user-postgresql

Info

tpv-user-postgresql

Connections

Hostname: dpv-...-a

Port: 5432

Database: tpv_user

Username: miw_upm

Password: tbul-...Toi2f

application-prod.properties

```
server.port=10000
# PostgreSQL -----
spring.jpa.hibernate.ddl-auto=update
spring.datasource.driverClassName=org.postgresql.Driver
spring.datasource.url=${DATABASE_URL}
spring.datasource.username=${DATABASE_USERNAME}
spring.datasource.password=${DATABASE_PASSWORD}
#JWT -----
miw.jwt.secret=secret-${JWT_SECRET}
```

WEB SERVICE

betca-tpv-user:latest

Image Free Upgrade your instance →

miw-upm / betca-tpv-user latest

<https://betca-tpv-user-latest.onrender.com>

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

Key	Value
DATABASE_PASSWORD	tbul-...Toi2f
DATABASE_URL	jdbc:postgresql://dpv-...-a:5432/tpv_user
DATABASE_USERNAME	miw_upm
JWT_SECRET	791c-...c3e0
PORT	10000

Render



MongoDB®
Atlas

<https://www.mongodb.com/es/atlas>

Dispone de un plan gratuito.

Soporta BD multi-cloud.

BETCA

MongoDB. Atlas

Create Organization
(UPM)

Create Project
(MIW)

Create Cluster:

- Plan (Free), Name: BETCA, Provider: AWS.
- Regin: Paris, User & Password

User Rol:
atlasAdmin@admin

Free

\$0/hour

Free forever

For learning and exploring MongoDB in a cloud environment.

STORAGE	RAM	vCPU
512 MB	Shared	Shared

Try Free

View shared tier pricing >

RECOMMENDED

Dedicated

\$0.08/hour

Pay as you go

For production applications with sophisticated workload requirements.

STORAGE	RAM	vCPU
10 GB	2 GB	2vCPUs

Get Started

View dedicated pricing >

Flex

\$0.011/hour

Up to \$30/month

For application development and testing; resources and costs scale to your needs.

STORAGE	RAM	vCPU
Up to 50B	Shared	Shared

Get Started

View Flex pricing >

UPM > MIW

Clusters

Find a database deployment...

BETCA Connect View

Visualize Your Data

Build dashboards and charts, and embed them in your apps with MongoDB Charts.

Dismiss Explore charts

VERSION REGION

Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver: Node.js Version: 6.7 or later

2. Install your driver

Run the following on the command line

```
npm install mongodb
```

[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☐ View full code sample

```
mongodb+srv://betca-tpv:<db_password>@betca.8aeqx.mongodb.net/?retryWrites=true&w=majority&appName=BETCA
```


Source Code

Name (*único*)Region
*Frankfurt (EU Central)*Instance Type
*Free \$0 / month*Enviroment
VariablesHealth Check Path
/actuator/health

Source Code

Git Provider Public Git Repository Existing Image

Image URL

Deploy an image from a Docker registry

ghcr.io/miw-upm/betca-tpv-user:latest

Credential (Optional)

RENDER_DEPLOY_TOKEN

Connect →

Add Credential

Learn more about storing external registry credentials.

Name

RENDER_DEPLOY_TOKEN

Registry

GitHub

You'll need to create a personal access token with the required `read:packages` permission in GitHub [Settings > Developer Settings](#).

Username

miw-upm

Personal Access Token

Cancel

Add Credential

Settings / Developer Settings

GitHub Apps

OAuth Apps

Personal access tokens

Fine-grained tokens

Preview

Tokens (classic)

Personal access tokens (classic)

Generate new token

Tokens you have generated that can be used to access the [GitHub API](#).RENDER_DEPLOY_TOKEN — `read:packages`

Last used within the last week

Delete

⚠ This token has no expiration date.

application-prod.properties

server.port=10000

Database MONGODB external

spring.data.mongodb.uri=\${MONGODB_URI}

#JWT

miw.jwt.secret=secret-\${JWT_SECRET}

Environment

WEB SERVICE

betca-tpv-core:latest

Image

Free

Upgrade your instance →

miw-upm / betca-tpv-core latest

<https://betca-tpv-core-latest.onrender.com>

Environment

Key	Value
JWT_SECRET	791d[REDACTED]c3e0
MONGODB_URI	mongodb+srv://betca-tpv:NM[REDACTED]FYabetca.8aeqx.mongodb.net/tpv?retryWrites=true&w=majority&appName=BETCA
PORT	10000



AWS (Amazon Web Services) es una plataforma de computación en la nube. Ofrece una amplia variedad de servicios, como almacenamiento, bases de datos, cómputo, redes, análisis, inteligencia artificial, seguridad...

EC2

- *Elastic Compute Cloud*. Permite ejecutar servidores virtuales, instancias personalizables y auto escalables.
- Son máquinas virtuales vacías...

S3

- *Simple Storage Service*. Almacenamiento escalable de objetos.
- Bueno para servir páginas estáticas.

RDS

- *Relational Database Service*. Bases de datos relacionales y gestionadas.

DynamoDB

- Base de datos NoSQL totalmente gestionada y de alto rendimiento.

Lightsail

- Simplifica la implementación y gestión de servidores virtuales, sitios web y aplicaciones pequeñas.
- Costes fijos y muy bajos para proyectos pequeños y medianos, pago por mes.
- Escalabilidad limitada, se puede manual, pero no automática.
- Permite instalar lo que necesites: docker, bases de datos...

App Runner

- Gestionar Docker muy fácilmente y automática.
- Escalabilidad automática.
- Integrada con Amazon *ECR* (*Elastic Container Registry*).
- Costes basado en consumo, mas altos que *Lightsail*.
- No es apto para Bases de datos o aplicaciones de alto rendimiento.

ECS

- *Elastic Container Service*. Plataforma para ejecutar contenedores Docker sin administrar servidores directamente. Tiene dos versiones:
 - *EC2*. Se gestiona la instancia EC2. Se puede configurar la instancia y se paga por ello.
 - *Fargate*. No se gestiona la instancia EC2. Se paga por la vCPU y GB asignado al contenedor. Son costes mas altos.
- Se apoya en *ECR* (*Elastic Container Registry*), registro de contenedores para almacenar.

Elastic Beanstalk

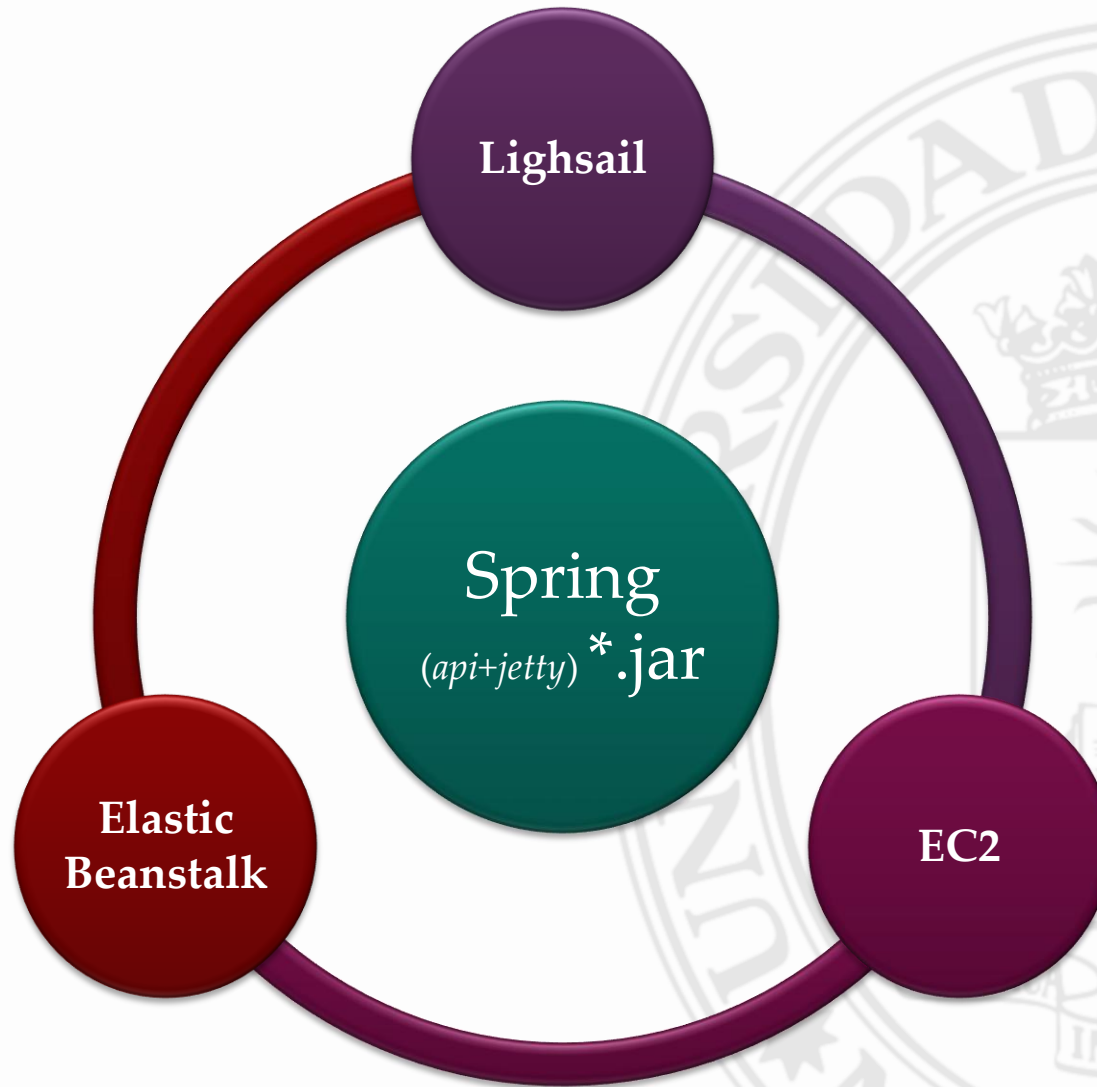
- Permite desplegar y administrar aplicaciones Web sin necesidad de gestionar la infraestructura (*Python, Java, Node.js, .NET, PHP, Ruby, Go, Docker...*).
- Escalabilidad automática.
- La configuración de **EC2, RDS, S3, Load Balancer...** es automática.
- Mas cara que *Lightsail*.

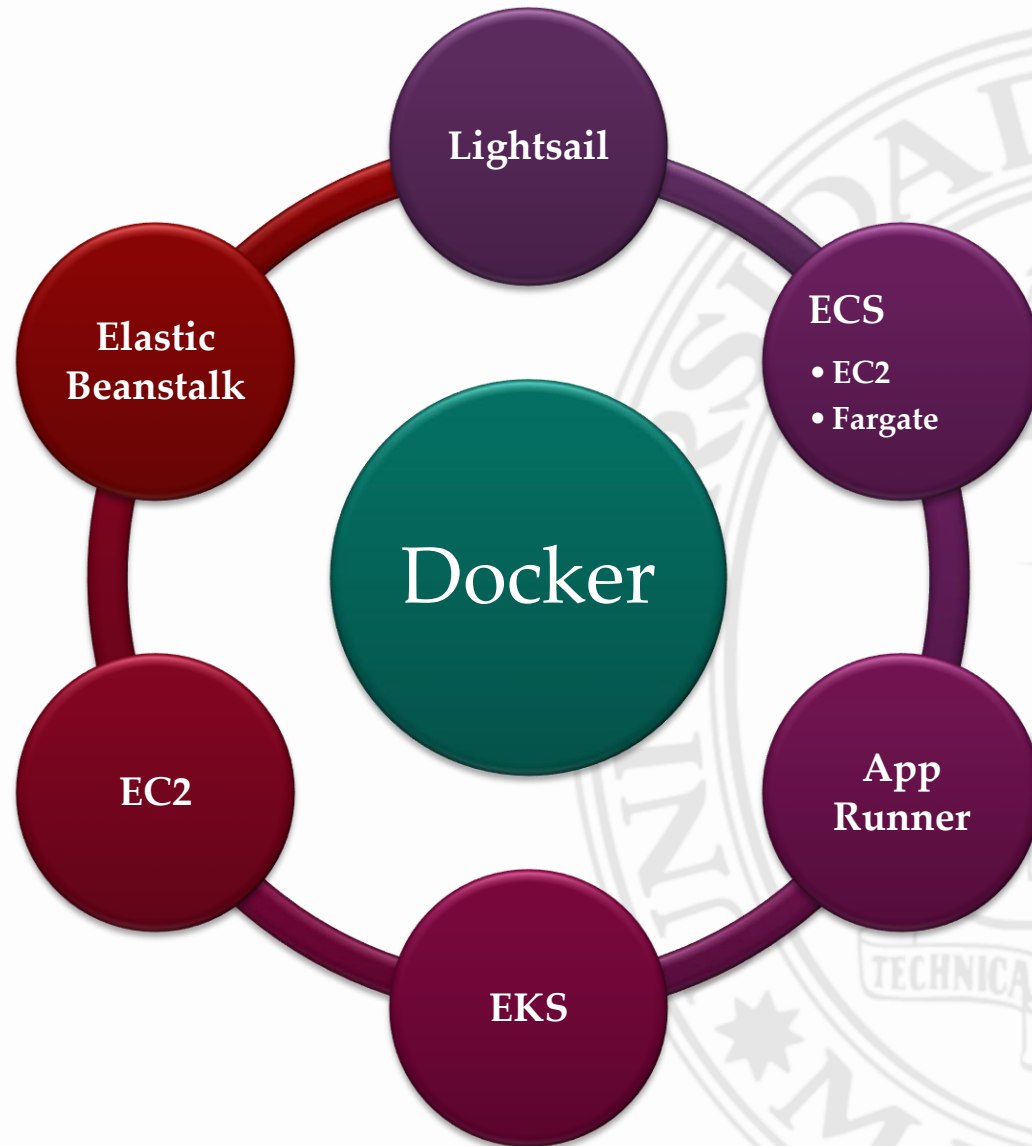
Lambda

- Ejecución de código, sin estado, orientado a eventos.
- Se debe evolucionar el *API Rest de Spring Boot*, utilizando *Spring Cloud Function*, esto incluye las librerías para conectar Spring Boot con AWS Lambda.
- Se debe exponer el API con API Gateway, para enrutar las peticiones HTTP a funciones lambda.
- Escala automáticamente según las peticiones.
- Solo se paga por uso, no tiene coste del servicio Tiempo máximo de 15 minutos y 1000 invocaciones concurrentes.
- Puede experimentar retrasos cuando lleve tiempo sin utilizar.

EKS. Virtual Private Cloud

- *Elastic Kubernetes Service*. Si necesitas una solución robusta y flexible de orquestación de contenedores.
- Se apoya en ECS.
- Se paga por cada cluster \$0.10/hora.





BETCA

AWS. GitHub: *deployed-aws*

API rest

- Descomprimir la plantilla.
- Cambiar el nombre de la carpeta y el nombre del artefacto dentro del fichero *pom.xml*.
- Abrir el proyecto con IntelliJ.
- Crear el repositorio.
- Crear la rama de desarrollo: *develop*.
- **Crear** el repositorio en **GitHub**.
- Conectar y subir el proyecto a GitHub: *develop*.
- Crear el Docker en entorno *dev* y probarlo.

Deploy

- Solo hasta *Packages* de **GitHub**.
- Crear la rama master-lightsail.
- Preparar la *release*.
- Probar el deploy sobre *Packages* de *GitHub*.



Identity and Access Management

- Es el servicio de gestión de identidad y acceso de AWS

Usuario

- Identidad individual, puede ser desarrollador, administrador o sistema.
- Puede ser mediante *user-password* o *Access Key*
- *Access Key*: Funciona con (Clave privada – clave pública)

Grupo

- Agrupación de usuarios y asignación de permisos conjuntamente.

Rol

- Es una identidad con permisos que pueden asumir un *Servicio AWS* o un *Usuario*.
- Son credenciales temporales.
- Ejemplo: **ECR-ECS-Access-Role** (*AmazonEC2ContainerRegistryFullAccess* & *AmazonECS_FullAccess*).

Políticas

- Se definen los *permisos* para que los usuarios, grupos o roles puedan hacer acciones sobre un recurso.
- Ejemplo: *AmazonEC2ContainerRegistryFullAccess*, acceso completo a ECR

Relaciones de confianza

- Se define quien o que servicio puede asumir un rol.



Relaciones de confianza

- ECS: *"ecs-tasks.amazonaws.com"*
- Lambda: *"lambda.amazonaws.com"*
- EC2: *"ec2.amazonaws.com"*
- EKS: *"eks.amazonaws.com"*
- App Runner Build: *"build.apprunner.amazonaws.com"*
- App Runner Service: *"tasks.apprunner.amazonaws.com"*
- API Gateway: *"apigateway.amazonaws.com"*
- S3: *"s3.amazonaws.com"*
- A todos los servicios de la cuenta: "AWS": *"arn:aws:iam::765470696147:root"*

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "build.apprunner.amazonaws.com"
      },
      "Action": "sts:AssumeRole",
      "Condition": {}
    }
  ]
}
```

AWS guarda toda la configuración en Json, siempre se puede utilizar directamente



Lightsail

- Crear una instancia
- Elegir región
- Elegir SO: *Ubuntu 22.04*
- Elegir Plan: *1GB recomendado*
- Bajarse el SSH key
- Incluir scrips para instalación de Docker & group



Crear IP estática

- Networking



GitHub

- Preparar el CD.yml
- Preparar variables de entorno
 - AWS_LIGHTSAIL_IP
 - AWS_LIGHTSAIL_USER
 - AWS_LIGHTSAIL_SSH_KEY
- <https://77zcxsjnqg.execute-api.eu-west-1.amazonaws.com>



Crear un API Gateway

- Tipo: HTTP
- Ruta: ANY – `http://ip:port`

BETCA

AWS. Lightsail



Lightsail

```
ssh -i "LightsailDefaultKey-REGION.pem" ubuntu@ip.ip.ip.ip
```

```
lsb_release -a  
echo $USER
```

```
sudo apt update  
sudo apt install -y docker.io  
sudo usermod -aG docker $USER  
newgrp docker
```

```
docker --version
```

```
- name: Deploy to AWS Lightsail  
uses: appleboy/ssh-action@v1.2.1  
with:
```

```
host: ${ secrets.AWS_LIGHTSAIL_IP }  
username: ${ secrets.AWS_LIGHTSAIL_USER }  
key: ${ secrets.AWS_LIGHTSAIL_SSH_KEY }  
script: |
```

```
echo ${ secrets.GHCR_PERSONAL_ACCESS_TOKEN } | docker login ghcr.io -u ${ github.actor } --password-stdin  
docker stop ${ env.ARTIFACT }-app || true  
docker rm ${ env.ARTIFACT }-app || true  
docker rmi ${ env.GHCR_URL }/${ env.REPOSITORY }:latest || true  
docker pull ${ env.GHCR_URL }/${ env.REPOSITORY }:latest  
docker run -d --name ${ env.ARTIFACT }-app --restart always -p 80:8080 ${ env.GHCR_URL }/${ env.REPOSITORY }:latest
```

AWS te proporciona una consola.
Conectarse a la SSH client e instalar
Docker y meter al usuario en el grupo
Docker.

```
-----BEGIN RSA PRIVATE KEY-----  
MIIEowIBAAKCAQEA7v60X8XzUoJ3Zf3...  
...  
7hH3H8oKzDQ0E+XKTbZgM1LfUQ==  
-----END RSA PRIVATE KEY-----
```

Deploy to Lightsail



IAM



Elastic Container Registry

IAM. Identity & Access Management

- Crear un usuario para deploy y asignarle permisos



ECR. Elastic Container Registry

- Crear un repositorio
- Crear regla de ciclo de vida



GitHub Actions

- Crear el action CD



IAM



Elastic Container Registry

IAM

- Crear cuenta en AWS
- Crear un usuario para el despliegue
 - Nombre: por ejemplo: *github-actions*
 - Sin acceso a consola
 - Adjuntar políticas directamente, buscar las siguientes y marcarlas: *AmazonEC2ContainerRegistryFullAccess* & *AmazonECS_FullAccess*
- Editar el usuario y Crear clave de acceso
 - Aplicación ejecutada fuera de AWS
 - Nombre recomendado: *github-actions-aws-deploy*
 - Anotar clave, luego no será posible verla: `AWS_SECRET_ACCESS_KEY`
 - `AWS_ACCESS_KEY_ID` & `AWS_ACCOUNT_ID`

Credenciales

- `AWS_ACCOUNT_ID`: 123456789012
- `AWS_ACCESS_KEY_ID`: `AKIA3EOM34LJYDL6E3YS`
- `AWS_SECRET_ACCESS_KEY`: `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`



IAM



Elastic Container Registry

ECR

- Elegir región: Irlanda, para que pueda trabajar con App Runner: *eu-west-1*
- Crear repositorio
- Nombre: el mismo que en GitHub: *miw-upm/deployed-aws*
- *Mutable*
- *AES-256*
- URI: *123456789012.dkr.ecr.eu-west-1.amazonaws.com/miw-upm/deployed-aws*
- Crear regla de ciclo de vida:
 - Descripción: *Elimina imágenes antiguas*
 - *Sin etiquetar*
 - *Desde que se envió la imagen*



IAM



Elastic Container Registry

ECR

- CD - Actions

evn:

AWS_ECR_URL: \${ secrets.AWS_ACCOUNT_ID }.dkr.ecr.\${ secrets.AWS_REGION_RUNNER

...

- **name:** Configure AWS Credentials

uses: aws-actions/configure-aws-credentials@v2

with:

aws-access-key-id: \${ secrets.AWS_ACCESS_KEY_ID }

aws-secret-access-key: \${ secrets.AWS_SECRET_ACCESS_KEY }

aws-region: \${ secrets.AWS_REGION_RUNNER }

- **name:** Login to AWS

uses: aws-actions/amazon-ecr-login@v2

- **name:** Tag and Push AWS ECR

run: |

docker tag \$REPOSITORY:\$VERSION \$AWS_ECR_URL/\$REPOSITORY:latest

docker push \$AWS_ECR_URL/\$REPOSITORY:latest



IAM



AWS App Runner

IAM. Crear Rol

- Servicio de AWS
- Caso de uso: EC2
- Nombre: *ECR-Runner-Access-Role*
- Permisos: *AmazonEC2ContainerRegistryReadOnly* & *AWSAppRunnerFullAccess*
- Editar el rol
 - Relaciones de confianza: *"build.apprunner.amazonaws.com"*
- Actualizar

App Runner. Crear servicio

- *Registro de contenedor*
- *Amazon ECR*
- Elegir la imagen...
- *Automático*
- Usar un rol de servicio existente: *ECR-Runner-Access-Role*
- Nombre: *deployed-aws-runner-service*
- 0.5 vCPU & 1GB
- Comprobación de estado: *HTTP, /actuator/health, intervalo 20*
- *<https://v5uy6hjz3n.eu-west-1.awsapprunner.com>*



IAM



Elastic Container Registry



Elastic Container Service

ECR. Elastic Container Registry

- Crear un repositorio
- Crear Política de Ciclo de Vida



GitHub Actions

- Push on ECR



IAM



Elastic Container Registry

AWS

- Crear cuenta y activarla. Esta es la cuenta **Raiz**
- Crear una cuenta de **admin**, permisos *IAMUserChangePassword* & *AdministratorAccess*

Crear User
deploy

- **Nombre:** por ejemplo: *github-actions*
- **Permisos:** *AmazonEC2ContainerRegistryFullAccess* & *AutoDeployECS*

Credenciales

- **AWS_REGION:** *eu-south-2*
- **AWS_ACCOUNT_ID:** *123456789012*
- **AWS_ACCESS_KEY_ID:** *AKI*****YXN*
- **AWS_SECRET_ACCESS_KEY:** *4AZ*****kYY*

Crear
repositorio

- **Nombre:** el mismo que en GitHub: *miw-upm/deployed-aws-ecs*
- **URI:** *123456789012.dkr.ecr.eu-south-2.amazonaws.com/miw-upm/deployed-aws-ecs*



Elastic Container Service



EC2

Cluster

- Crear cluster: devops-cluster
- AWS Fargate

IAM. Definición de rol

- Nombre: *ECR-ECS-Access-Role*
- Permisos: *AmazonEC2ContainerRegistryFullAccess, AmazonECS_FullAccess & AmazonECSTaskExecutionRolePolicy*
- Relación de confianza: "Service": "*ecs-tasks.amazonaws.com*"

Definición de tarea

- Nombre: *deployed-aws-ecs-task*
- AWS Fargate
- .5 vCPU & 1GB
- Rol de ejecución de tareas: *ECR-ECS-Access-Role*
- Contenedor
 - Nombre: *deployed-aws-ecs-container*
 - URI de imagen: *123456789012.dkr.ecr.eu-south-2.amazonaws.com/miw-upm/deployed-aws*
 - Puerto Contenedor: *8080*
 - Variables de entorno...



Elastic Container Service



EC2

ECS. Cluster. Crear Servicio

- *Tipo de lanzamiento, con FARGATE & LATEST*
- *Servicio*
- *Familia: deployed-aws-ecs-task*
- *Nombre: deployed-aws-ecs-service*
- *Tareas deseadas: 1*
- **Usar Equilibrio de carga (EC2)**
 - *Balanceador de carga de aplicaciones*
 - *Contenedor: deployed-aws-ecs-container 8080:8080*
 - *Crear Balanceador*
 - *Nombre: deployed-aws-ecs-alb*
 - *Crear un nuevo agente de escucha*
 - *Crear un nuevo agente de destino*
- *<http://deployed-aws-ecs-alb-1177297874.eu-south-2.elb.amazonaws.com/>*



Preparar
Deploy

• Preparar *CD.yml*

- **name:** Update ECS service

run: |
aws ecs update-service --cluster devops-cluster --service deployed-aws-ecs-service --force-new-deployment



Despliegue en Render

A través de Imagen Docker

Desde el repositorios de
GitHub

BETCA

Angular. CD

Etapa de construcción

```
FROM node:22.13.1-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build-prod
```

Parte de una imagen con Node con Alpine

Copia el *package.json* de la ruta raíz del proyecto a la carpeta de trabajo del contenedor

Se instala las dependencias

Se construye en entorno producción

Etapa de runtime con un servidor ligero de ficheros (Nginx)

```
FROM nginx:stable-alpine
WORKDIR /usr/share/nginx/html
COPY --from=build /app/dist/betca-tpv-angular/browser/ ./
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Puerto HTTP abierto

Etapa de runtime, express un servidor mas pesado

```
FROM node:22.13.1-alpine
WORKDIR /app
COPY --from=build /app/dist/betca-tpv-angular /app
RUN npm install -g serve
EXPOSE 10000
CMD ["serve", "-s", "-l", "10000", "/app"]
```

Establece un comando para cuando se inicialice el contenedor: *RUN serve -s -l 10000*

Angular. *Despliegue Continuo*

```
name: CD
on:
  push:
    branches:
      - master
jobs:
  cd:
    name: Build & Push GitHub Package & Deploy on Render Docker Image
    runs-on: ubuntu-22.04
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: actions/setup-node@v4
        with:
          node-version: 22.13.1
      - name: Get version from package.json
        id: get-version
        run: |
          echo "REPOSITORY=${{ github.repository }}" >> $GITHUB_ENV
          echo "VERSION=$(node -pe "require('./package.json').version")" >> $GITHUB_ENV
          echo "PACKAGE_PATH=ghcr.io/${{ github.repository }}" >> $GITHUB_ENV
```



Angular. *Despliegue Continuo*

- **name:** Show NAMES

run: |
echo \$REPOSITORY
echo \$VERSION
echo \$PACKAGE_PATH

- **name:** Login to GitHub Container Registry

uses: docker/login-action@v3

with:

registry: ghcr.io

username: \${ github.actor }

password: \${ secrets.DOCKER_REGISTRY_TOKEN }

- **name:** Build and Tag and Push Docker image

run: |
docker build -t \$REPOSITORY:\$VERSION .
docker tag \$REPOSITORY:\$VERSION \$PACKAGE_PATH:\$VERSION
docker tag \$REPOSITORY:\$VERSION \$PACKAGE_PATH:latest

- **name:** Push Docker image to GitHub Container Registry

run: |
docker push \$PACKAGE_PATH:\$VERSION
docker push \$PACKAGE_PATH:latest

- **name:** Deploy on Render

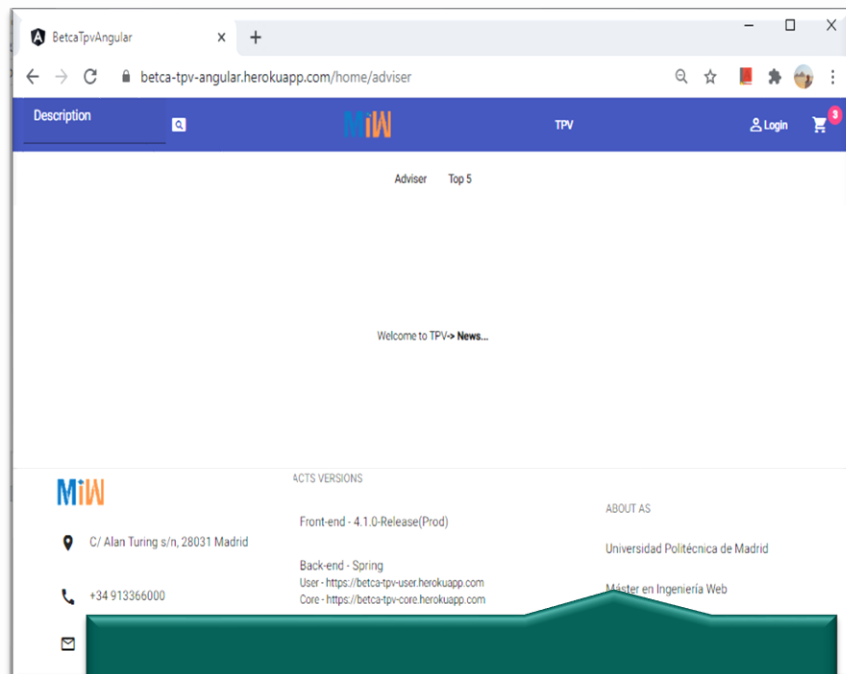
run: |
curl --fail -X POST "https://api.render.com/deploy/\${ secrets.DEPLOY_HOOK_TOKEN }" || { echo "Deployment failed"; exit 1; }
echo "Deployment succeeded"



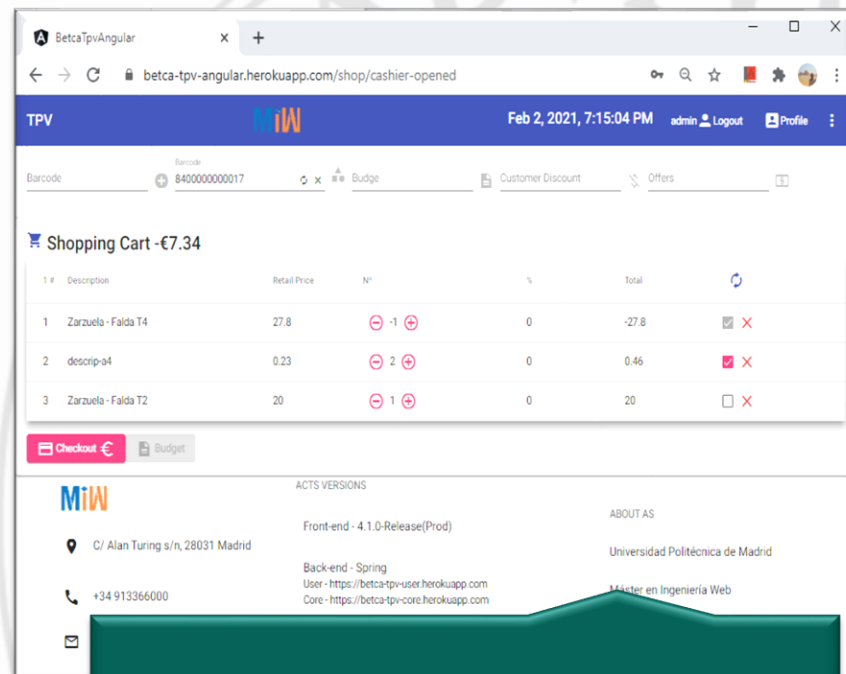
Lo mismo
que para
*.user,
*.core...

<https://github.com/miw-upm/betca-tpv>

- Asignación de prácticas: *Wiki - Labels*
- Gestión del proyecto: *TPV*
 - *Commits: “miw-upm/betca-tpv#xxx”*
- Enunciado: *README*
 - *Story Requirements: UI*
- Entrega



tpv-core



tpv-core



Angular Material (*Google*)

- <https://material.angular.io/>
- Github: ★24.5K



NG Bootstrap

- <https://ng-bootstrap.github.io>
- Github: ★8.2K



NG Zorro

- <https://ng.ant.design>
- Github: ★8.9K



PrimeNG

- <https://www.primefaces.org/primeng/>
- Github: ★11.1K



Onsen UI

- <https://onsen.io/>
- Github: ★8.8K

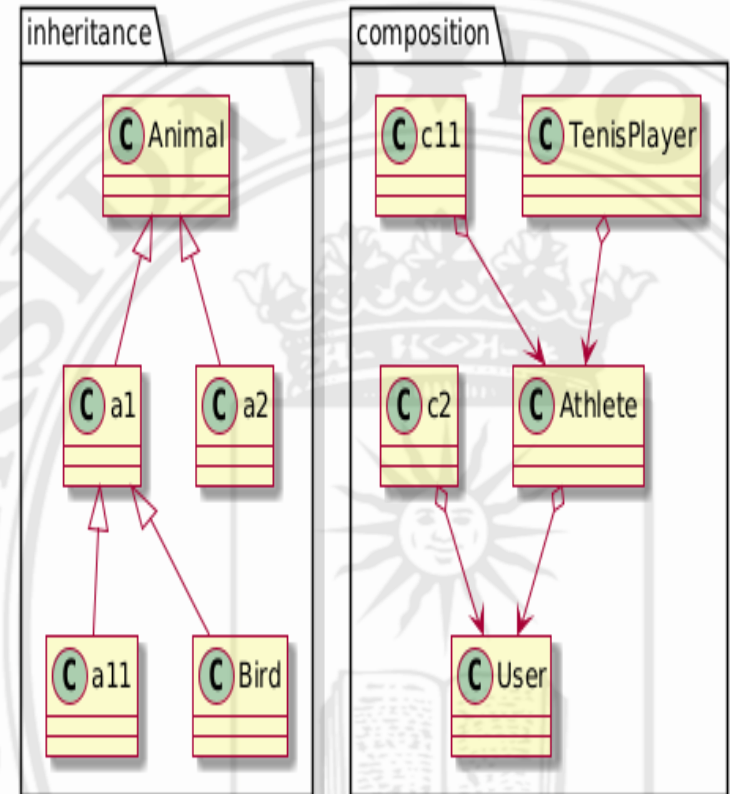
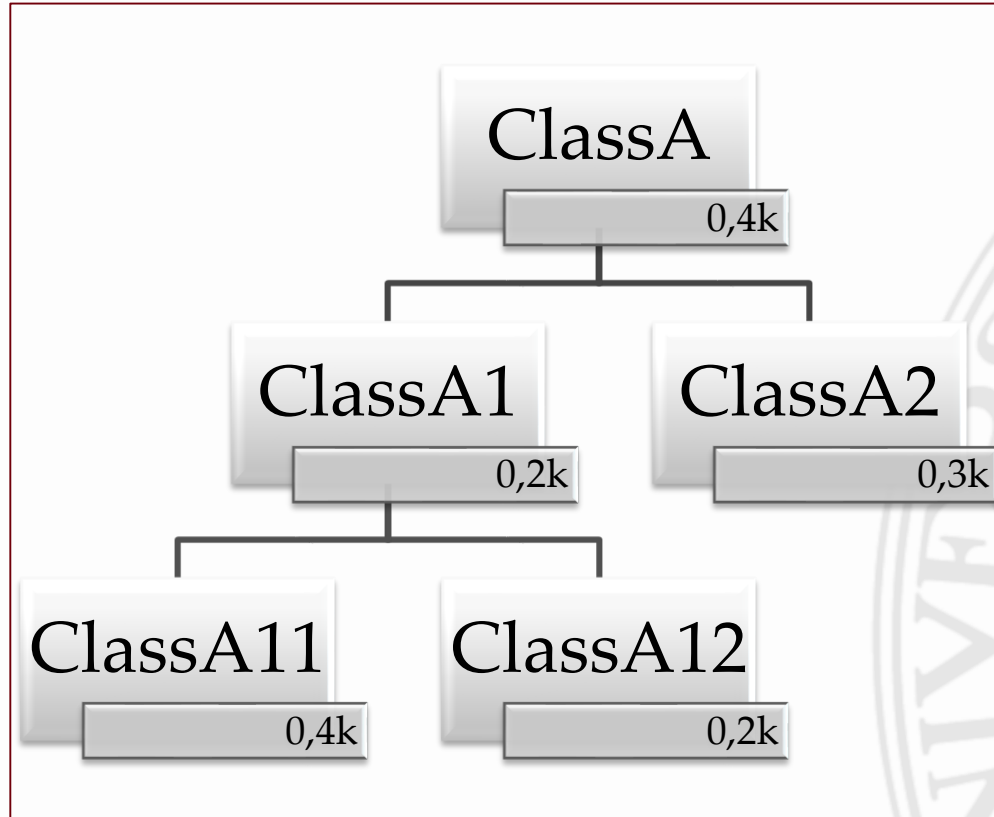


Clarity

- <https://clarity.design/>
- Github: ★6.4K

Angular

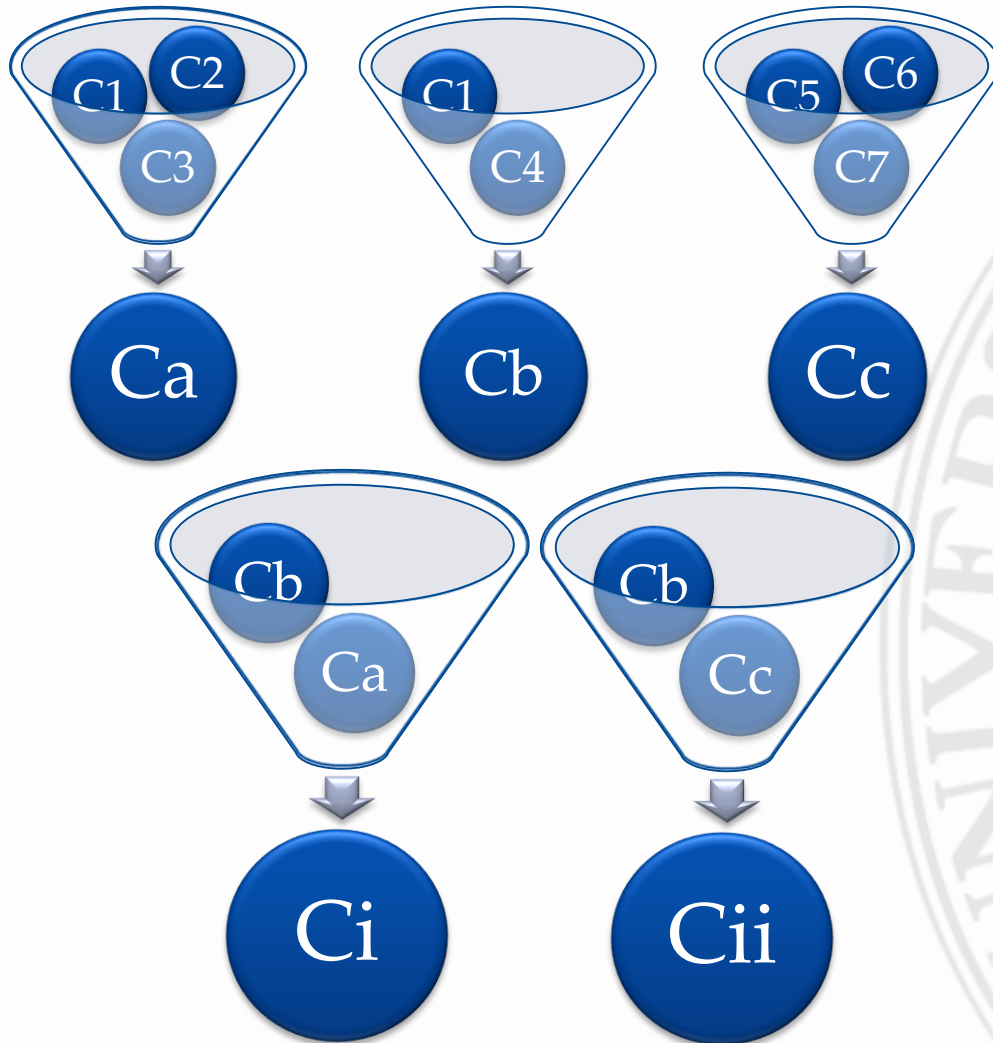
Composición vs Herencia



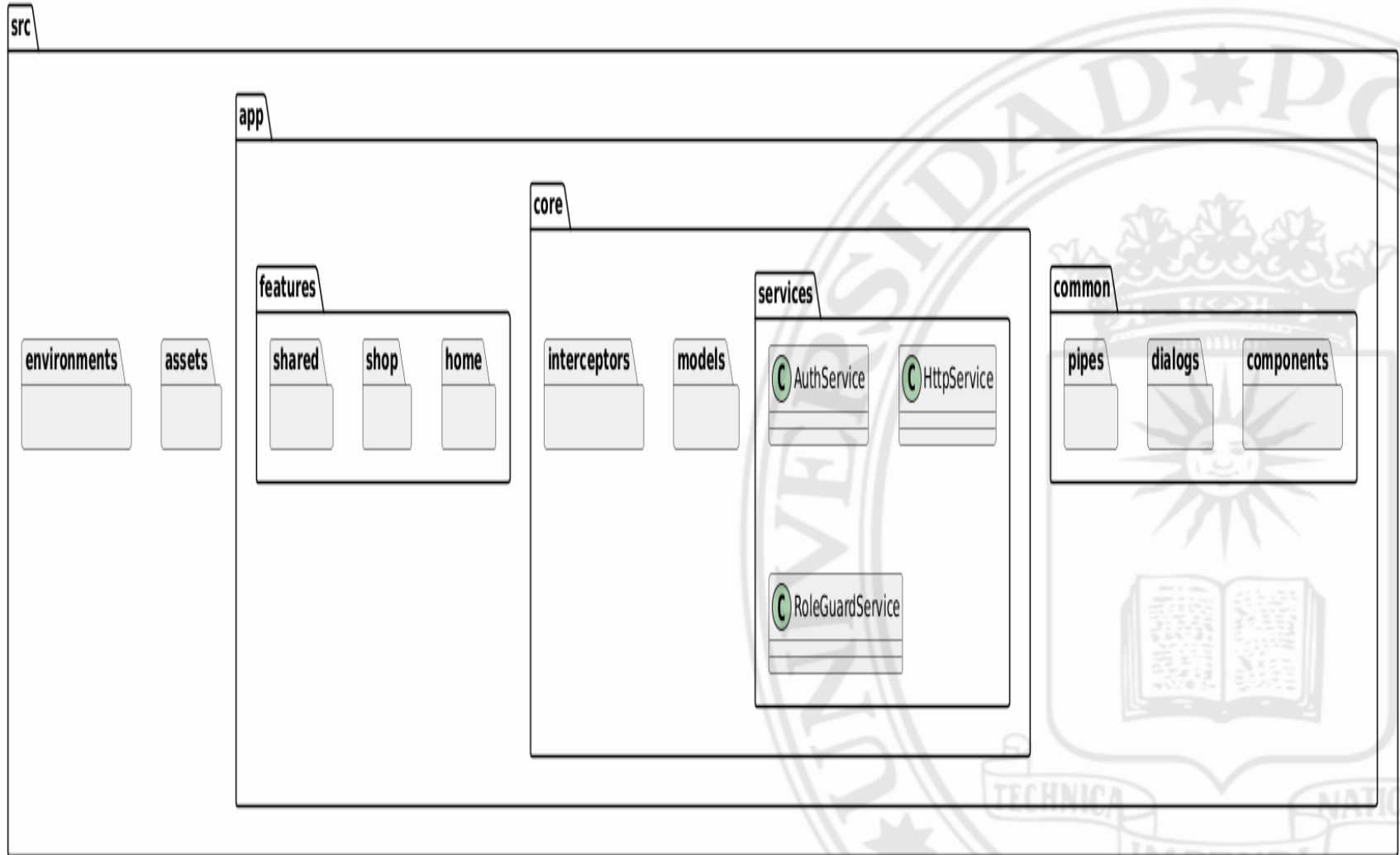
$$1,5K = 2,5K$$

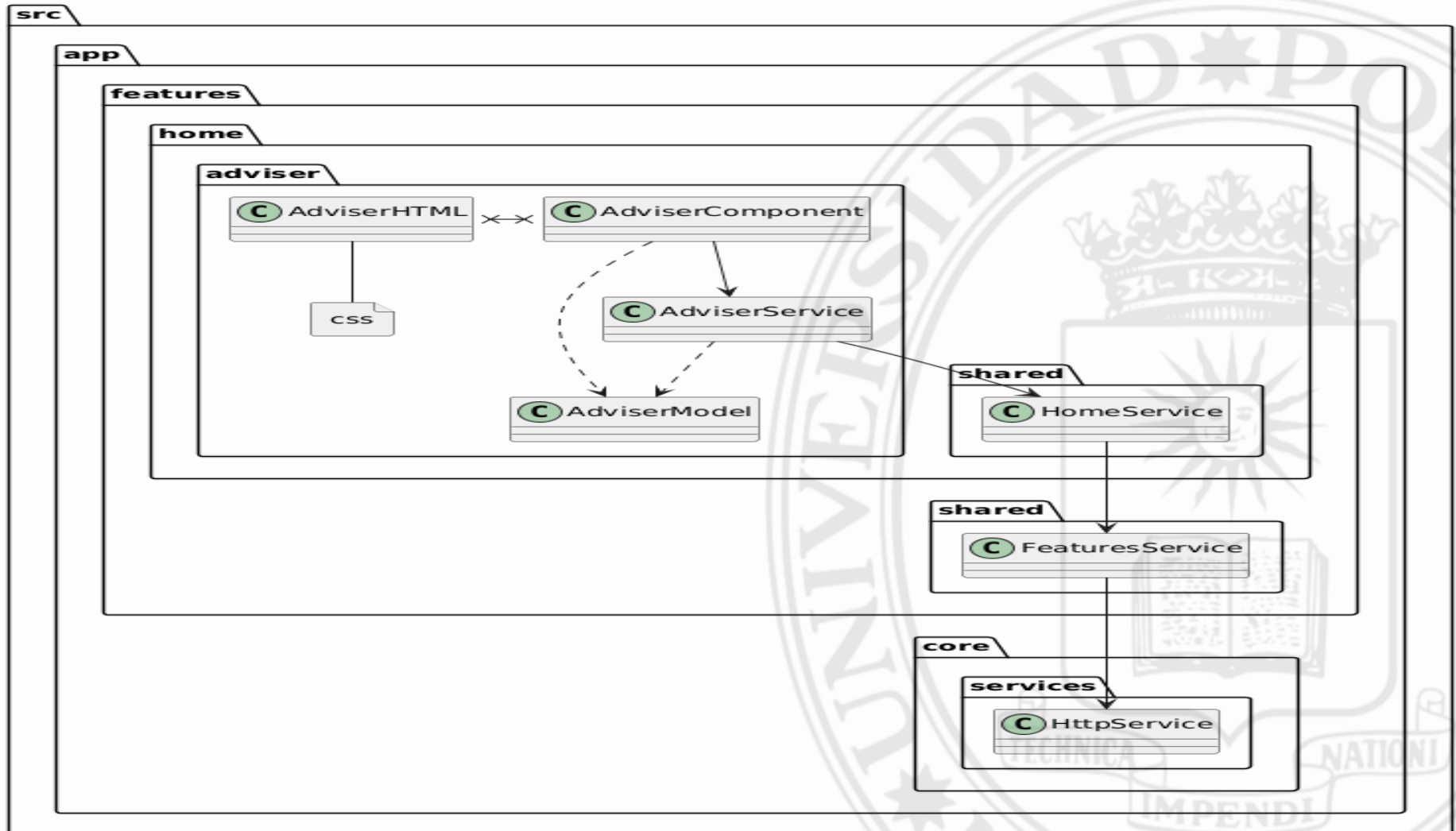
Angular

Componentes (Composición) & Servicios (Asociación)



Angular Arquitectura





Template: HTML*

```
@if (warning()) {  
  <mat-icon color="warn">report_problem</mat-icon> Uncommit articles & phone not registered  
}
```

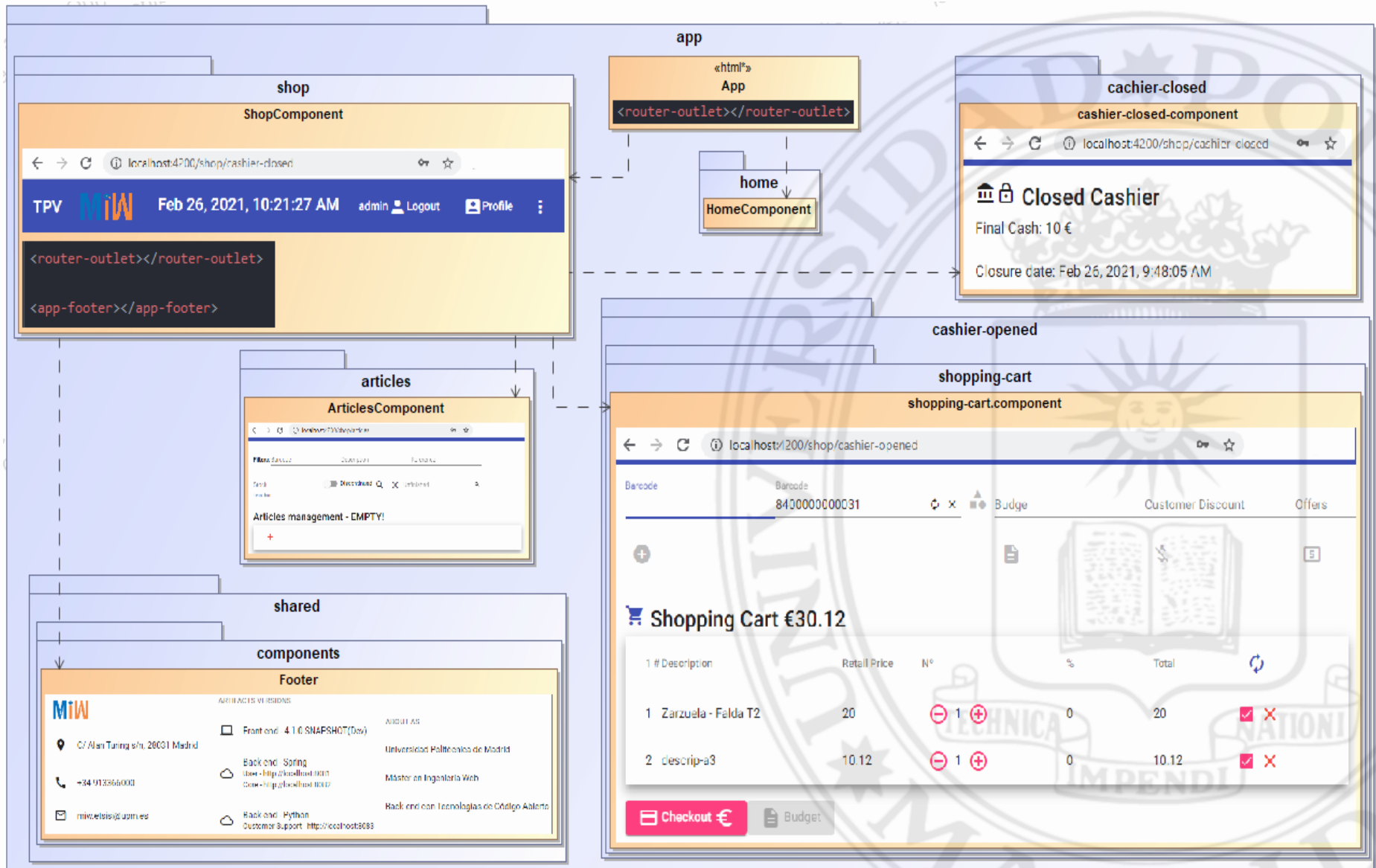
```
<mat-checkbox [(ngModel)]="requestedInvoice" [disabled]="invalidInvoice()">  
  <span> Create Invoice</span>  
</mat-checkbox>
```

Component

```
warning(): boolean {  
  return !this.managedMobile() && this.unCommitted();  
}
```

```
invalidInvoice(): boolean {  
  // TODO pendiente de calcular. Hace falta tener al usuario totalmente completado  
  return true;  
}
```

```
articles = of([]);  
search(): void { this.articles = this.articleService.search(this.articleSearch); }
```



```
export const routes: Routes = [
  {path: '', pathMatch: 'full', redirectTo: 'home/adviser'},
  {
    path: 'home', component: HomeComponent,
    children: [
      {path: 'adviser', component: AdviserComponent}, // PUBLIC
      {
        path: 'complaints',
        component: ComplaintsComponent,
        canActivate: [RoleGuardService],
        data: {roles: [Role.CUSTOMER]}
      }
    ]
  },
  {
    path: 'shop', component: ShopComponent,
    canActivate: [RoleGuardService],
    data: {roles: [Role.ADMIN, Role.MANAGER, Role.OPERATOR]},
    children: [ // or path: 'shop/articles'
      {path: 'articles', component: ArticlesComponent},
      {path: 'cashier-closed', component: CashierClosedComponent},
      {path: 'cashier-opened', component: CashierOpenedComponent},
      {path: 'providers', component: ProvidersComponent},
      {path: 'tickets', component: TicketsComponent},
    ]
  }
];
```

app.routes.ts



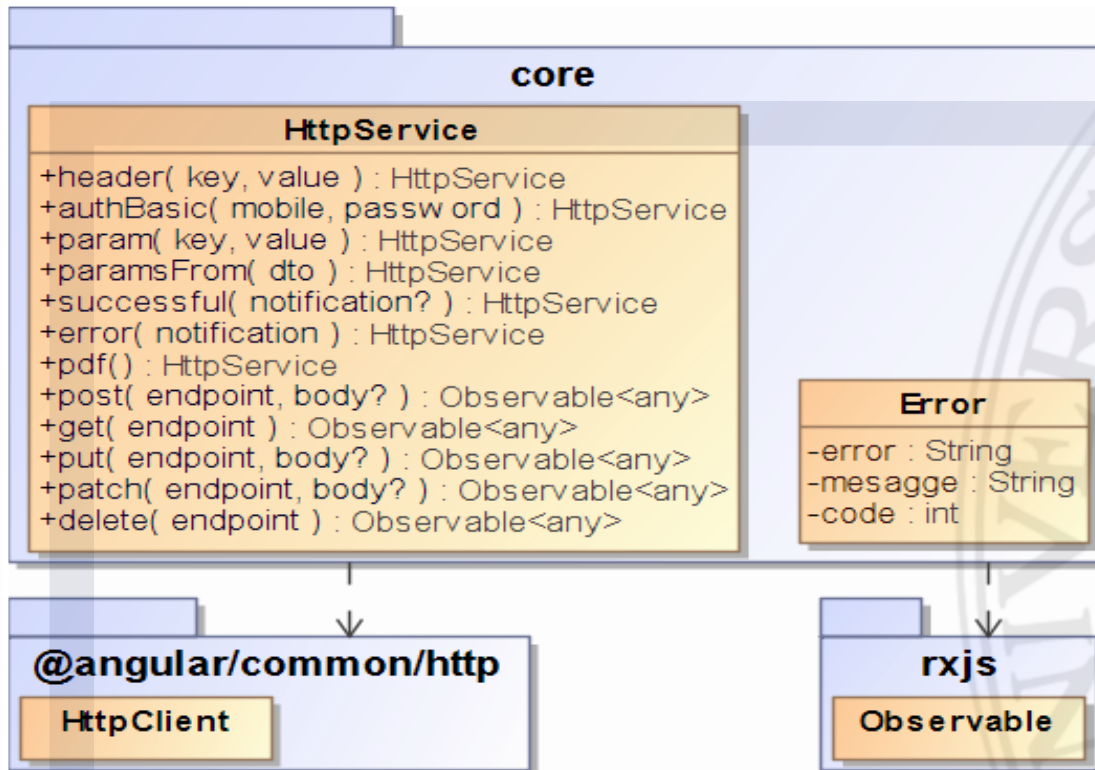
• Riesgo alto de conflicto

<https://github.com/miw-upm/betca-tpv-angular>

- *<https://github.com/miw-upm/betca-tpv>*
- **Clonar el proyecto betca-tpv-user**
 - Crear imagen y contenedor con ejecución. Manejarlo con *Docker Desktop*
- **Clonar el proyecto betca-tpv-core**
 - Crear imagen y contenedor. Manejarlo con *Docker Desktop*
- **Clonar el proyecto betca-tpv-angular, instalar y abrir con WebStorm**
 - `> git clone https://github.com/miw-upm/betca-tpv-angular`
 - `> cd betca-tpv-angular`
 - `> betca-tpv-angular> npm install`
- **Para ejecutar**
 - `> ng serve`

Carpetas

- Comprender la arquitectura



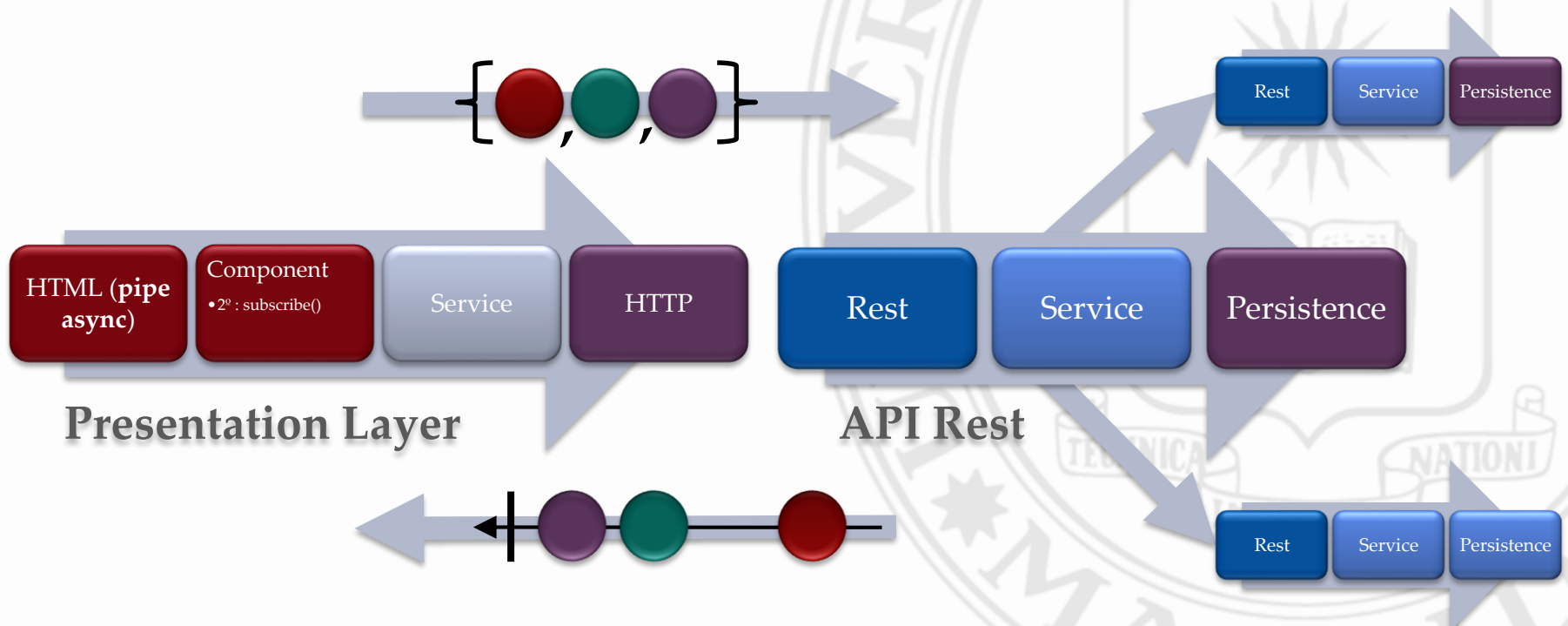
```
httpService.paramsFrom(articleSearch).get(endpoint)
```

```
httpService.pdf().get(endpoint)
```

HttpService

- Patrón **Facade & Builder**. Simplificar llamadas al API Rest y ayudar a crear la llamada
- Tratamiento de Seguridad: Basic Auth
- Tratamiento automático de errores con extracción del mensaje de error y presentación mediante **Snackbar**
- Tratamiento de **pdf** con presentación automática en ventana nueva

Métodos que devuelven un tipo *Publisher* no deberían subscribirse porque ello *podría romper la cadena del publicador*




```
<mat-dialog-content> <ng-container *ngIf="(object | async) as data">
  @for (def of labels(data); track def) {
    <mat-label><strong>{{ def | uppercaseWords }}: </strong>{{ data[def] }}</mat-label>
  }
</ng-container> </mat-dialog-content>
```

ReadDetailDialogHTML

```
export class ReadDetailDialogComponent {
  title: string; object: Observable<any>;
  constructor(@Inject(MAT_DIALOG_DATA) data: any) {
    this.title = data.title; this.object = data.object;
  }
  labels(object): string[] { return Object.getOwnPropertyNames(object); }
}
```

ReadDetailDialogComponent

```
read(article: Article): void {
  this.dialog.open(ReadDetailDialogComponent, {
    data: { title: 'Article Details', object: this.articleService.read(article.barcode)
  } }); }
```

ArticleComponent

```
read(barcode: string): Observable<Article> {
  return this.httpService.get(Endpoints.ARTICLES + '/' + barcode);
}
```

ArticleService

```
get(endpoint: string): Observable<any> {
  return this.http
    .get(endpoint, this.createOptions())
    .pipe(
      map(response => this.extractData(response)),
      catchError(error => this.handleError(error))
    );
}
```

HttpService

Article Details

Barcode: 8400000000024

Description: Zarzuela - Falda T4

Retail Price: 27.8

Reference: zz-falda-T4

Stock: 5

Discontinued: false

Registration Date: 2025-02-13 06:33:09

Provider Company: pro1

Close

HTTPS

Header

Authorization

Authorization

Basic user:password_{code64}

Bearer jwt



Basic user:api-key_{code64}



URL:/*/api-key

Angular

Seguridad: JSON Web Token

JWT

- <https://jwt.io>
- <https://auth0.com>

Header_{code64}

- Se define el tipo.
- HS256

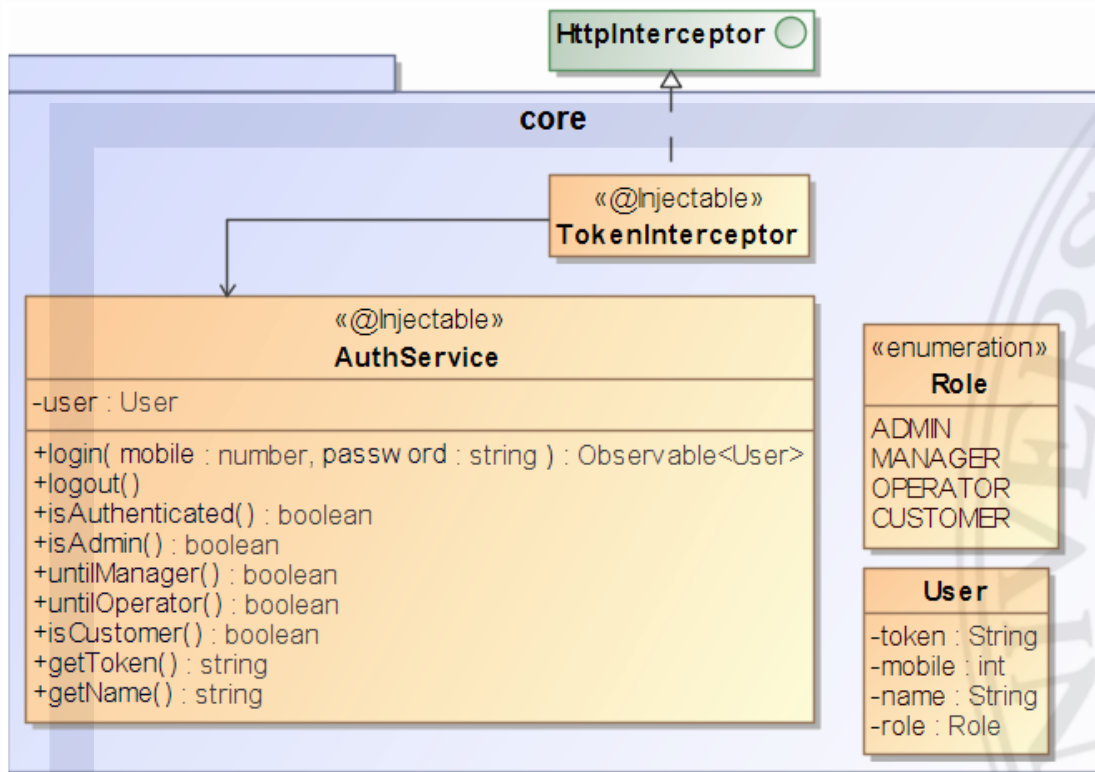
Payload_{code64}

- Datos o carga del token.
 - Proveedor
 - Usuario
 - Expiración
 - Validez
 - Privilegios: rol, nombre...

Signature_{code64}

- Se firma con *clave secreta* la carga del *token* para saber si ha sido alterado.

eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.
eyJ1Ym9keiOiE2MDc0NDk2NzYsInJvbGUiOiJBRE1JTilslmZcyI6Im1pdY5qd3QuaXNzdW
VyIiwibmFtZSI6ImFkbWluIiwiaXhwIjozNDUzMjc2LCJpYXQiOiE2MDc0NDk2NzYsIn
VzZXIiOiI2In0.
V9QW9gMeYH2cKFxErfhh51FROkUZzLKqlvYhBm2YNLc



Auth Basic: Para obtener el token

Bearer JWT: para todos los accesos mediante token

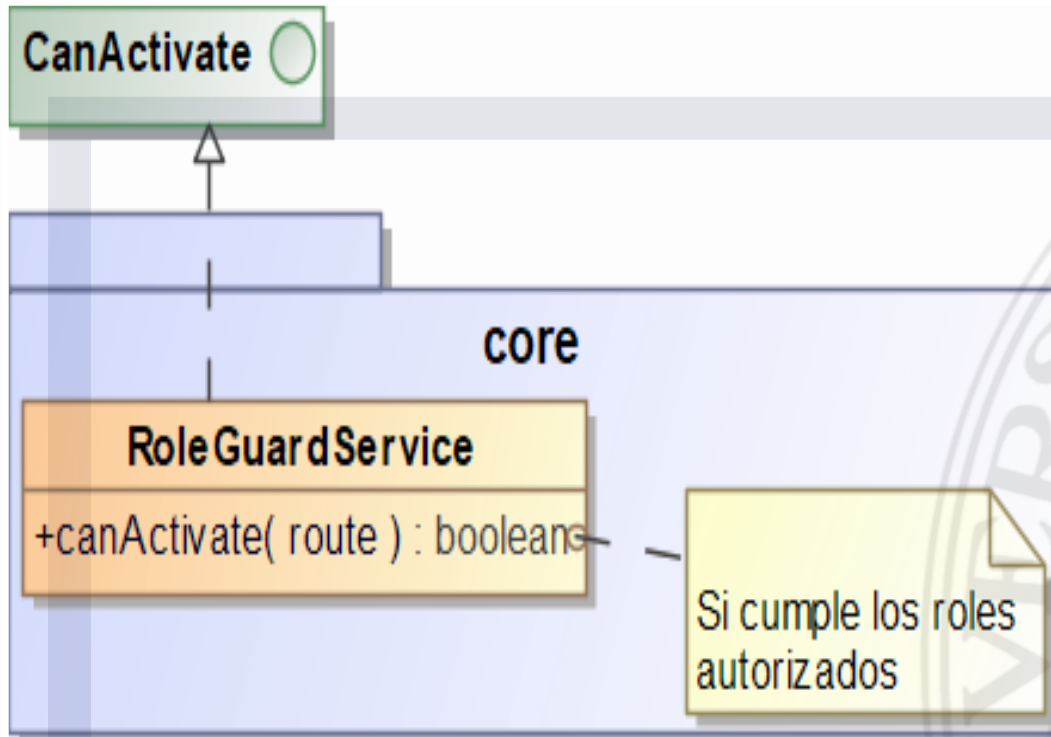
TokenInterceptor. Si existe un **token**, lo incluye automáticamente en la cabecera de todas las peticiones

- **login:** crea *user*: **POST betca-tpv-user//users/token**

- **logout:** elimina el *user*

Angular

Seguridad de enrutamiento



Ocultar rutas no permitidas y en cada ruta, indicar **roles permitidos**

RoleGuardService

Comprueba que el rol actual, el rol del token almacenado, cumple con los roles necesarios para acceder a la ruta

```
{path: 'adviser', component: AdviserComponent}, // public
{path: 'complaints', component: ComplaintsComponent, canActivate: [RoleGuardService], data: {roles: [Role.CUSTOMER]}}
```

Angular

Servicios Mock

Servicios mock

```
export class ArticleService {  
  create(article: Article): Observable<Article> {  
    return of(article);  
  }  
  read(barcode: string): Observable<Article> {  
    return of({barcode: barcode, description: '...', retailPrice: 10, providerCompany: 'pro1'})  
  }  
  search(articleSearch: ArticleSearch): Observable<Article[]> {  
    return of([  
      {barcode: 123, description: '...', retailPrice: 10, providerCompany: 'pro1'},  
      {barcode: 345, description: '...', retailPrice: 10, providerCompany: 'pro2'},  
      {barcode: 678, description: '...', retailPrice: 10, providerCompany: 'pro1'}  
    ])  
  }  
}
```

Angular

Pipe *async*

Uso del Pipe **async**

```
<p><button mat-raised-button (click)=requestSync() >Sync </button> {{sync}}</p>  
<p><button mat-raised-button (click)=requestAsync()>Async</button> {{asyn | async}}</p>
```

```
export class InputOverviewExample {  
  sync: string;  
  asyn: Observable<string>;  
  requestSync(): void {  
    this.serviceMock()  
      .subscribe(item => this.sync = item);  
  }  
  requestAsync(): void {  
    this.asyn = this.serviceMock();  
  }  
  serviceMock(): Observable<string> {  
    return of('Result');  
  }  
}
```

Angular

Componentes

my-comp.component.html

```
<p>Hello {{user}}! <input type="button" value="Ok" (click)="onClick()"></p>
```

```
import { Component, EventEmitter, Input, Output } from '@angular/core';
@Component({ selector: 'app-my-comp', templateUrl: './my-comp.component.html' })
export class MyCompComponent {
  @Input() user = 'By default';
  @Output() pressed = new EventEmitter<string>();
  onClick(){
    this.pressed.emit(this.user)
  }
}
```

Utilización:

```
<app-my-comp></app-my-comp>
<app-my-comp user="Value"></app-my-comp>
<app-my-comp [user]=user></app-my-comp>
<app-my-comp user="With event" (pressed)="onPressed($event)"></app-my-comp>
```

```
export class AppComponent {
  user = "Variable";
  onPressed(value:string){
    alert(value);
  }
}
```

Angular

Componentes

```
<app-crud (create)="create()" (read)="read($event)" (update)="update($event)" [data]="providers" [title]="title"></app-crud>
```

```
<app-crud (create)="create()" (read)="read($event)" (update)="update($event)"
  [data]="providers" [deleteAction]="true" [printAction]="true" [runAction]="true" [title]="title"></app-crud>
```

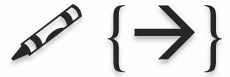
```
create(): void {
  this.dialog
    .open(ProviderCreationUpdatingDialogComponent)
    .afterClosed()
    .subscribe(() => this.search());
}
```

```
read(provider: Provider): void {
  this.dialog.open(ReadDetailDialogComponent, {
    data: {
      title: 'Provider Details',
      object: this.providerService.read(provider.company)
    }
  });
}
```

Async

```
update(provider: Provider): void {
  this.providerService
    .read(provider.company)
    .subscribe(fullProvider => this.dialog.open(ProviderCreationUpdatingDialogComponent, {data: fullProvider})
      .afterClosed()
      .subscribe(() => this.search()));
}
```

Sync



<https://github.com/miw-upm/betca-tpv>

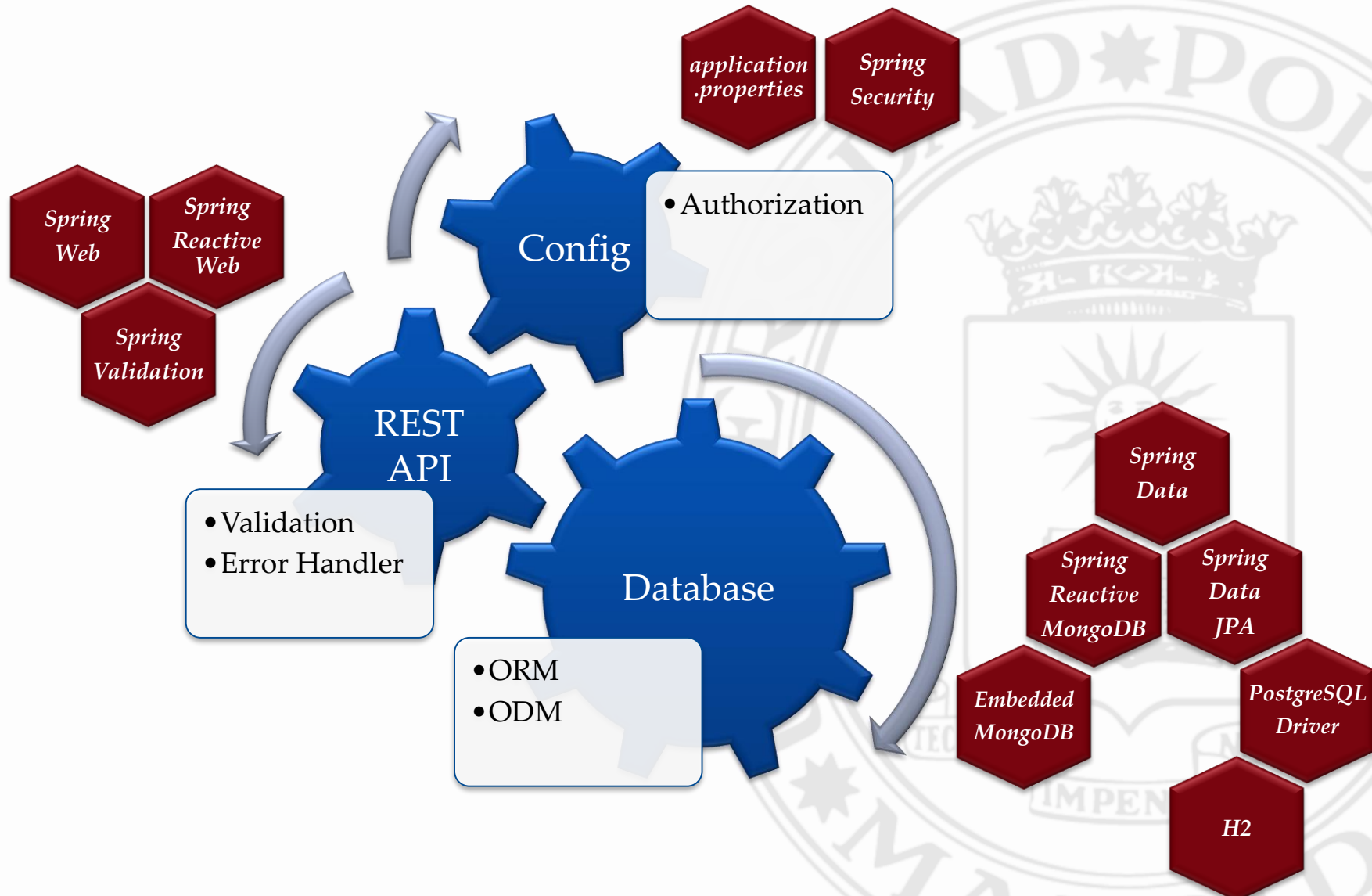
- Perfiles: environments
- Componentes
 - core: *CRUD, Footer, Search, Date*
 - shop/shared: *SearchByBarcode (Search), SearchByCompany*
- *package.json*
- Rutas: *tsconfig.json*

 Angular

- Comprender el código

Spring

Spring es un framework ligero de código abierto, creado por Rod Jhonson, para facilitar el desarrollo de Aplicaciones Empresariales modernas (*Java* o *kotlin*)



Spring Boot

- Construye y arranca rápidamente la aplicación con una configuración inicial mínima de Spring.

Spring Framework

- Proporciona un modelo completo de programación y configuración para aplicaciones empresariales modernas.

Spring Data

- Proporciona acceso a diferentes tipos de bases de datos.

Spring Security

- Protege la aplicación con diferentes sistemas de autenticación y autorización.

Spring Cloud

- Proporciona herramientas que implementas los patrones comunes de sistemas distribuidos.

Spring Cloud
Data Flow

Spring for
GraphQL

Spring AI

Spring REST
Docs

Spring
HATEOAS

Spring CLI

...

Spring Framework

<https://spring.io/projects/spring-framework>

Spring MVC

- Web síncrona, bloqueante.
- Un hilo por petición.
- Contenedor Servlets: Tomcat, Jetty...
- Repositorios: JDBC (Postgres, MySQL, SQL Server), JPA, MongoDB.
- Mas asentada en el mercado laboral.

Spring Reactive

- Web asíncrona, no bloqueante.
- Pocos hilos (procesadores multi-core) pero masivo de conexiones.
- Contenedor Servlets 3.1+: Netty.
- Repositorios: Reactive MongoDB, Cassandra, Redis, Couchbase, R2DBC (Postgres, MySQL, SQL Server)
- Apropiaada para conexiones masivas.
- Curva de aprendizaje mas compleja por la programación reactiva.

Spring Framework

Inyección de dependencias

betca_injection

```
«@SpringBootApplication»  
Application  
+main(args: String[])( )
```

```
«@Component»  
InjectionMain  
-serviceSingleton : SingletonMessageService  
-servicePrototype : PrototypeInjectionOnlyMessageService  
«@Autowired»+InjectionMain( serviceSingleton, servicePrototype )
```

```
«@Service»  
SingletonMessageService  
-name : String  
+SingletonMessageService( name : String )  
«@PreDestroy»+destroy()
```

```
«@Configuration»  
InjectionConfiguration  
«@Bean»+ service() : PrototypeInjectionOnlyMessageService
```

```
PrototypeInjectionOnlyMessageService
```

betca_injection

```
SingletonMessageServiceWithoutInjectionTest  
«@BeforeEach»+before()  
«@Test»+testGetMessage()  
«@Test»+testGetValue()
```

```
«@SpringBootTest»  
SingletonMessageServiceTest  
«@Autowired»-singletonMessageService : SingletonMessageService  
«@Test»+testGetMessage()  
«@Test»+testGetValue()
```

```
«@SpringBootTest»  
InjectionMainMessageServiceMockTest  
«@MockBean»-singletonMessageService : SingletonMessageService  
«@Test»+testGetMessage()  
«@Test»+testGetValue()
```

```
«@SpringBootTest»  
InjectionMainIT  
«@Autowired»-injectionMain  
«@Test»+testGetMessage()  
«@Test»+testGetValue()
```


Inyección de dependencias



<https://github.com/miw-upm/betca-spring>

- **betca-injection**
- <https://start.spring.io/>

Inyección de Dependencias en práctica

- Crear un servicio con un método que devuelve un *String* con el día de la semana, crear un test para probarlo (*LocalDate.now().getDayOfWeek().toString()*)
- Crear un componente que utiliza el servicio anterior y tiene un método que indica si es fin de semana (*isWeekEnd*), crear *test unitario* y de *integración*.
- Añadir la propiedad "*miw.author=****" con el nombre del autor en el fichero *application.properties* y añadir al servicio un segundo método que devuelve un *String* con el nombre del autor, ampliar test de prueba.

Persistencia

Spring Framework

JTA

- Java Transaction API
- Soporta el manejo de transacciones tanto de forma declarativa como programado.

JDBC

- Java Database Connectivity.
- Nos proporciona una capa de abstracción para acceder mediante SQL.
- *String name = jdbcTemplate.queryForObject("select name from actor where id = ?", 1212L);*

R2DBC

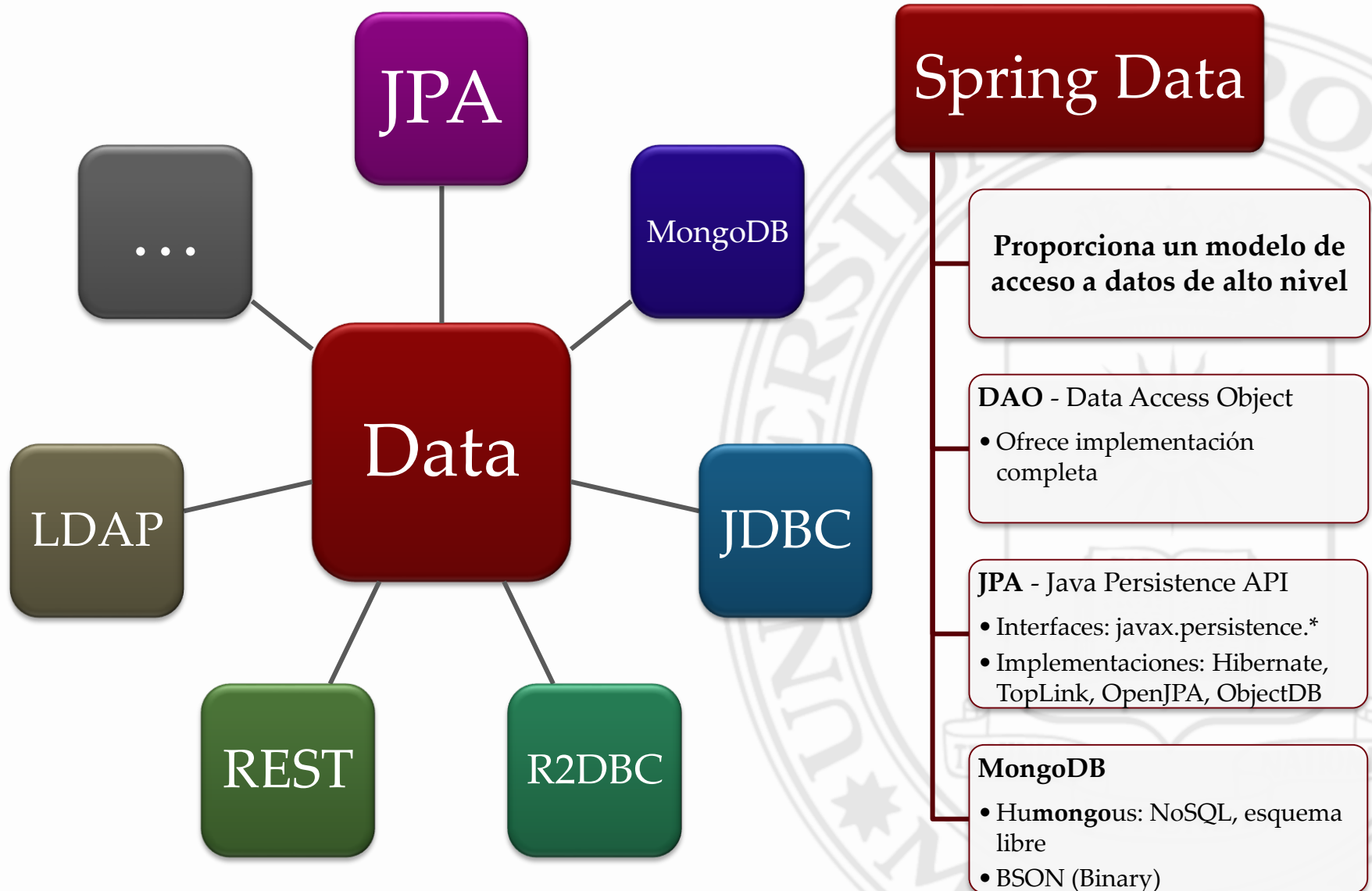
- Reactive Relational Database Connectivity.
- Nos da acceso a BD SQL utilizando patrones reactivos.
- *Flux<String> names = client.sql("SELECT name FROM person") .all();*

ORM

- Object Relational Mapping
- Soporta Java Persistence API (JPA) y Hibernate nativo

OXML

- Object-XML Mapping
- Nos proporciona la serialización XML (Marshaller & Unmarshaller)



Spring Data JPA. Dependencias



Dependencies

ADD DEPENDENCIES... CTRL + B

Lombok **DEVELOPER TOOLS**

Java annotation library which helps to reduce boilerplate code.

Spring Data JPA **SQL**

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

PostgreSQL Driver **SQL**

A JDBC and R2DBC driver that allows Java programs to connect to a PostgreSQL database using standard, database independent Java code.

H2 Database **SQL**

Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

MariaDB Driver **SQL**

MariaDB JDBC and R2DBC driver.

MySQL Driver **SQL**

MySQL JDBC and R2DBC driver.

Oracle Driver **SQL**

A JDBC driver to Oracle.

src/*/resources

- application-dev.properties
- test.properties
 - @TestPropertySource

Library: JAR

- @SpringBootApplication)
- @TestConfig

Package

- Jakarta (a part of Spring-boot 3)

Spring Data Database. Configuración

H2 (test.properties)

- **Database H2 Embedded**

```
spring.datasource.driver-class-name=org.h2.Driver  
spring.datasource.url=jdbc:h2:mem:testdb  
#spring.datasource.username=sa  
#spring.datasource.password=
```

Postgresql (application-dev.properties)

- **Database Postgresql external**

```
spring.jpa.hibernate.ddl-auto=update  
spring.datasource.url=jdbc:postgresql://localhost:5432/betca  
spring.datasource.username=postgres  
spring.datasource.password=
```

Postgresql (application-prod.properties)

- **Database Postgresql in server**

```
spring.jpa.hibernate.ddl-auto=update  
spring.datasource.driverClassName=org.postgresql.Driver  
spring.datasource.url=${DATABASE_URL}
```

«@Builder»
«@AllArgsConstructor»
«@Data»
«@NoArgsConstructor»

Dto

-id : String
-name : String
-surname : String
-email : String
«@Singular»-tags : List<String>

```
Dto.builder().id("1").tag("tag1").tag("tag2").build()
```

Proyecto Lombok

<https://projectlombok.org/>

- @Data
 - @Getter
 - @Setter
 - @ToString
 - @EqualsAndHashCode
 - @RequiredArgsConstructor
- @NoArgsConstructor
- @AllArgsConstructor
- @Builder
 - @Singular

• @Value

Spring Data

Proyecto Lombok

@Builder

@Data // @ToString, @EqualsAndHashCode, @Getter, and @Setter on all non-final fields, @RequiredArgsConstructor

@EqualsAndHashCode(onlyExplicitlyIncluded = **true**)

@NoArgsConstructor

@AllArgsConstructor

public class DtoWithLombok {

@EqualsAndHashCode.Include // only id

private String id; // @Setter(AccessLevel.NONE) for exceptions

private String name;

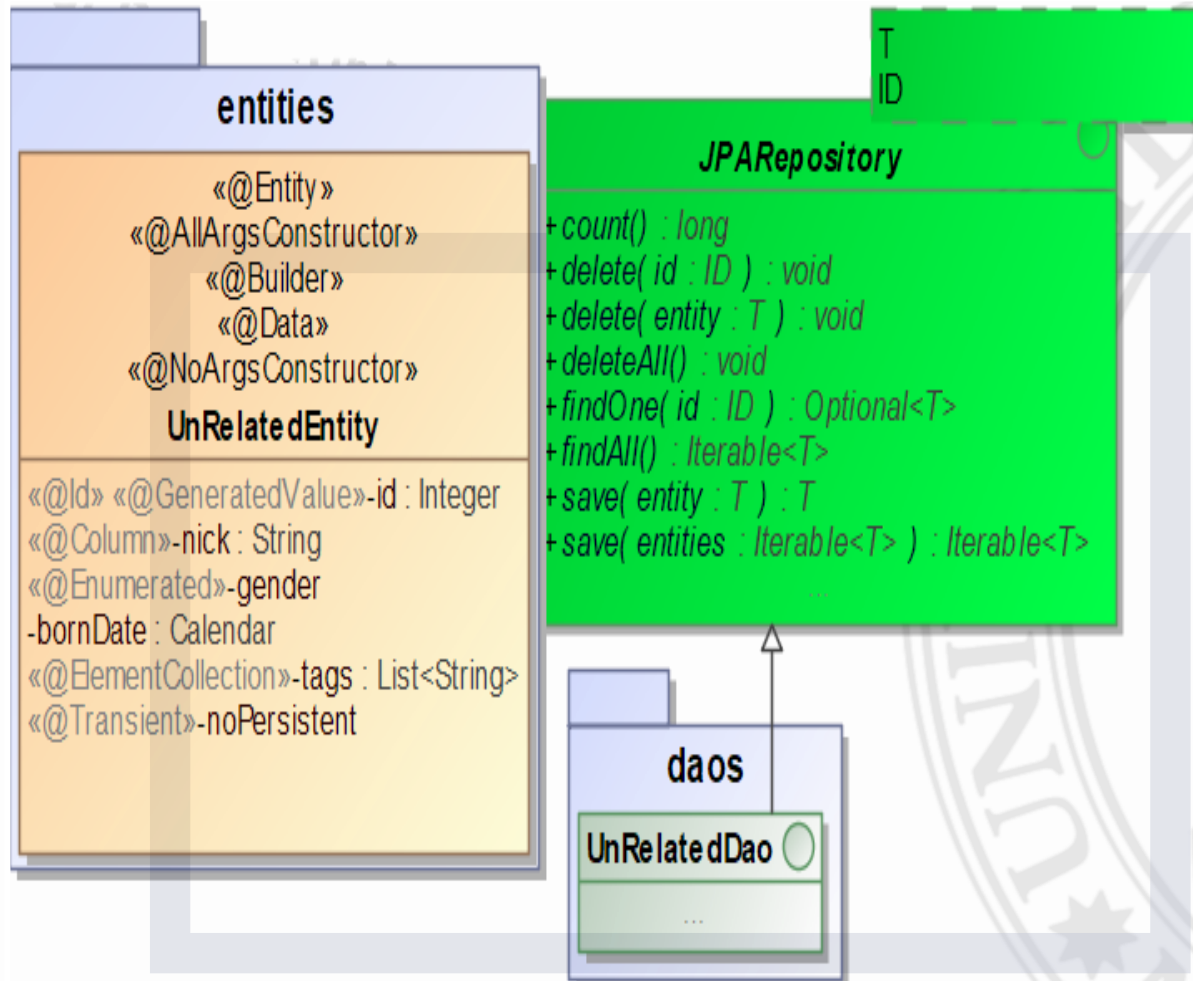
private String surName;

private String email;

@Singular // .tag().tag().tag()

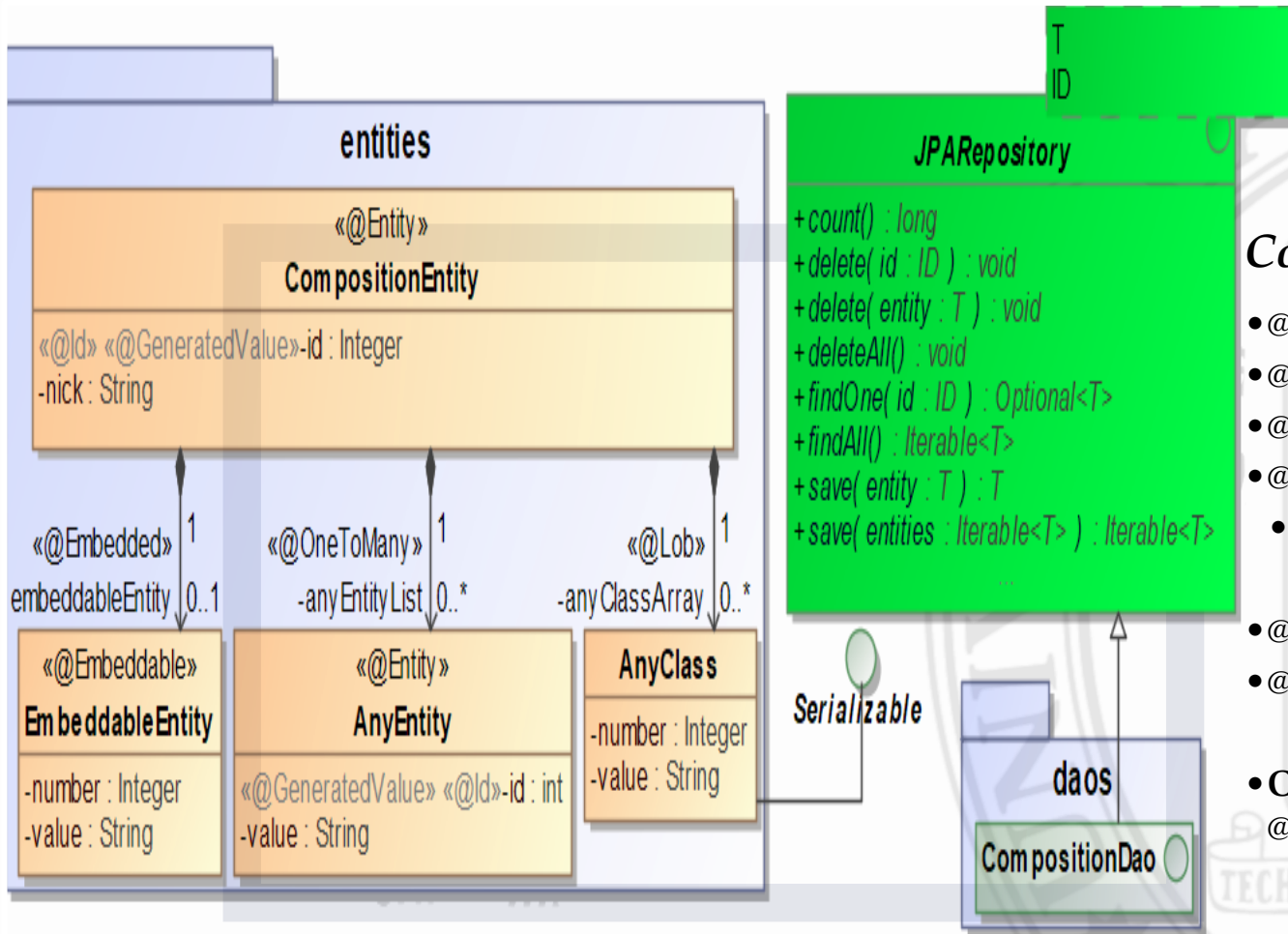
private List<String> tags;

}



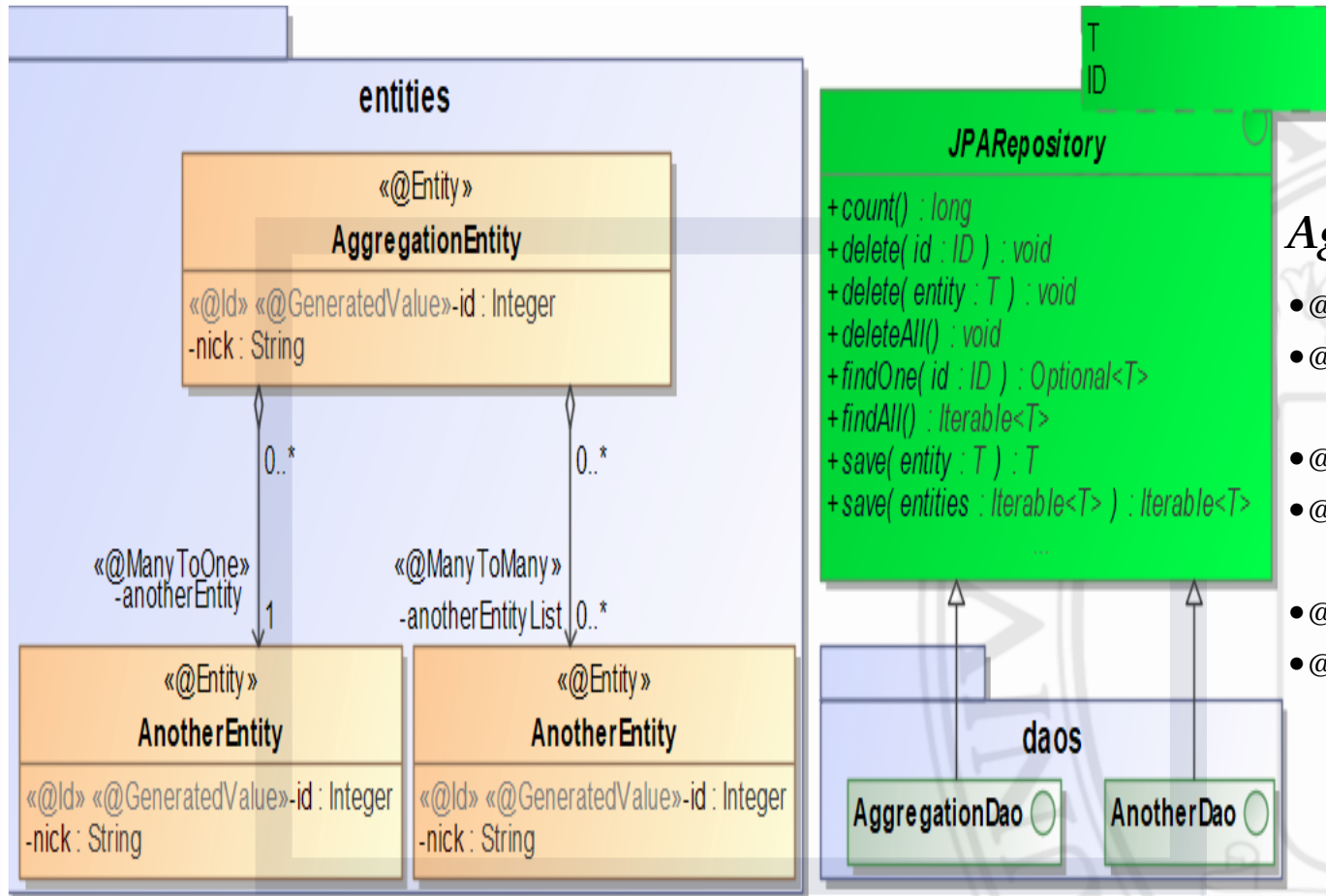
Sin Relación

- @Entity
- @Id
- @NoArgsConstructor
- @Data



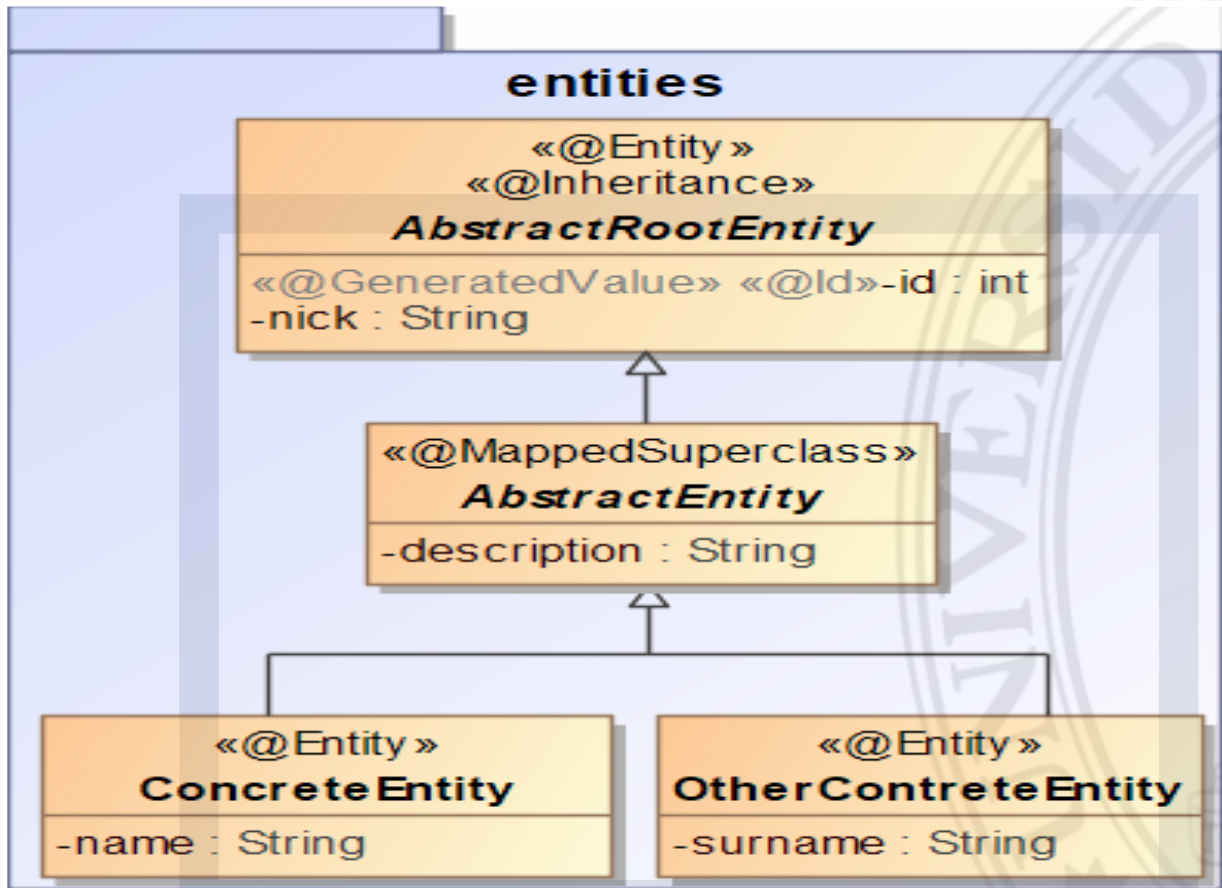
Composición

- `@Entity` o `@Embeddable`
- `@Id`
- `@Embedded` o `@Lob`
- `@OneToMany`
 - `cascade=cascadeTypeALL`
- `@NoArgsConstructor`
- `@Data`
- **Opcional:** `@Builder`, `@AllArgsConstructor`



Agregación

- `@Entity`
- `@Id`
- `@NoArgsConstructor`
- `@Data`
- `@ManyToOne`
- `@ManyToMany`

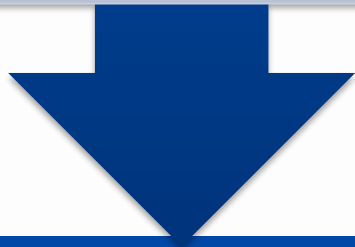


Herencia

- @Entity
- @Id
- @Inheritance
 - SINGLE_TABLE
 - JOINED
- @MappedSuperclass

Docker

*Iniciar el contenedor docker
con postgres*



```
> docker exec -it postgres-db psql -U postgres
```

Consola

- \? *Help*
- \l *Database list*
- \connect <db> <usr>
- \dt *Table list*
- \d <table>; *describe*
- \q *exit*
- create database <db>;
- drop database <db>;
- select * from <table>;
- delete from <table>;
- \timing *activa*

Spring Data

Postgresql. <https://www.postgresql.org/>

Bajarse el fichero *.zip

<https://www.enterprisedb.com/download-postgresql-binaries>

Descomprimir en el equipo, FUERA del proyecto

Inicializar las Bases de Datos

```
>bin> .\initdb -D ./data -U postgres -E UTF8
```

Arrancar – Parar

```
>bin> .\pg_ctl -D ./data -l logs start    >bin> .\pg_ctl -D ./data -l logs stop
```

Consola

```
>bin> chcp 1252    >bin> .\psql -U postgres
```

Crear Database “betca” & “tpv”

Consola

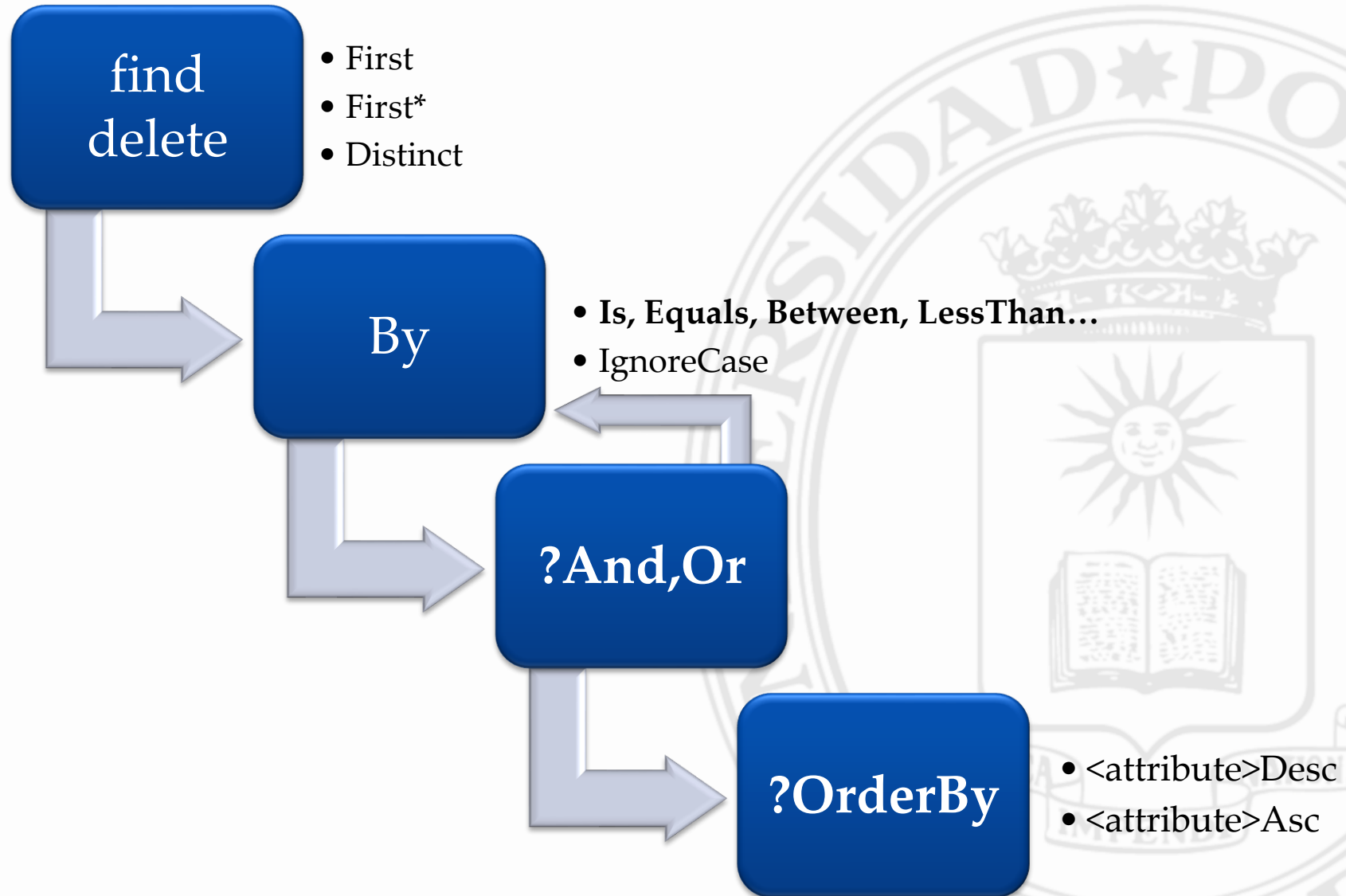
- `\? Help`
- `\l Database list`
- `\connect <db> <usr>`
- `\dt Table list`
- `\d <table>; describe`
- `\q exit`
- `create database <db>;`
- `drop database <db>;`
- `select * from <table>;`
- `delete from <table>;`
- `\timing activa`

<https://github.com/miw-upm/betca-spring>

- {→} Demo de Postgres
- Proyecto: betca-jpa
- Plugin lombok en IntelliJ (*Settings/plugins*)

JPA en práctica

- Crear nuevas entidades con tests. **Lanzar los Test con mucha frecuencia!!!**
 - Sin relación
 - Con Composición
 - Con Agregación
- Postgres
 - Instalar Postgres y crear las BD “betca” & “tpv”
 - Comentar la activación de H2 en *test.properties*
 - Arrancar el motor de Postgres y lanzar los test y mirar las tablas creadas



Búsquedas por nombre de método

- **findByLastnameAndFirstname**, **findByLastnameOrFirstname**
 - **findByFirstname**, **findByFirstnameIs**, **findByFirstnameEquals**
 - **findByStartDateBetween**
 - **findByAgeLessThan**, **findByAgeLessThanEqual**, **findByAgeGreaterThan**, **findByAgeGreaterThanEqual**
 - **findByStartDateAfter**, **findByStartDateBefore**
 - **findByAgeIsNull**, **findByAgeNotNull**, **findByAgeIsNotNull**
 - **findByFirstnameLike**, **findByFirstnameNotLike**
 - **findByFirstnameStartingWith**, **findByFirstnameEndingWith**
 - **findByFirstnameContaining**
 - **findByAgeOrderByLastnameDesc**
 - **findByLastnameNot**
 - **findByAgeIn**(Collection<Age> ages), **findByAgeNotIn**(Collection<Age> ages)
 - **findByActiveTrue()**, **findByActiveFalse()**
 - **findByFirstnameIgnoreCase**
-
- List<Entity> **findByAtr1**(String atr1);
 - int **deleteByAtr1GreaterThan**(int value);

Paginación

- *List<UnRelatedEntity> findByIdGreaterThan(int id, Pageable pageable);*
- *dao.findByIdGreaterThan(90, PageRequest.of(1, 5));*
- *dao.count();*

JPQL (Java Persistence Query Language)

- *@Query("select u.id from other_name_for_unrelatedentity u where u.id > ?1 and u.id < ?2")*
- *List<Integer> findIdByIdBetween(int initial, int end);*
- *@Modifying @Query(value="delete from other_name_for_unrelatedentity u where u.nick = ?1")*
- *int deleteByNickQuery(String nick);*

SQL

- *@Query(value = "SELECT * FROM un_related_entity WHERE NICK = ?1", nativeQuery = true)*

JPQL

- `findByLastNameAndFirstname ...where x.lastname = ?1 and x.firstname = ?2`
- `findByLastNameOrFirstname ...where x.lastname = ?1 or x.firstname = ?2`
- `findByFirstname, findByFirstnamesIs, findByFirstnameEquals ...where x.firstname = ?1`
- `findByStartDateBetween ...where x.startDate between ?1 and ?2`
- `findByAgeLessThan ...where x.age < ?1`
- `findByAgeLessThanEqual ...where x.age <= ?1`
- `findByAgeGreaterThan ...where x.age > ?1`
- `findByAgeGreaterThanEqual ...where x.age >= ?1`
- `findByStartDateAfter ...where x.startDate > ?1`
- `findByStartDateBefore ...where x.startDate < ?1`
- `findByAgeIsNull ...where x.age is null`
- `findByAgeNotNull, findByAgeIsNotNull ...where x.age not null`
- `findByFirstnameLike ...where x.firstname like ?1`
- `findByFirstnameNotLike ...where x.firstname not like ?1`
- `findByFirstnameStartingWith ...where x.firstname like ?1(parameter bound with appended %)`
- `findByFirstnameEndingWith ...where x.firstname like ?1(parameter bound with prepended %)`
- `findByFirstnameContaining ...where x.firstname like ?1(parameter bound wrapped in %)`
- `findByAgeOrderByLastNameDesc ...where x.age = ?1 order by x.lastname desc`
- `findByLastNameNot ...where x.lastname <> ?1`
- `findByAgeIn(Collection<Age> ages) ...where x.age in ?1`
- `findByAgeNotIn(Collection<Age> ages) ...where x.age not in ?1`
- `findByActiveTrue() ...where x.active = true`
- `findByActiveFalse() ...where x.active = false`
- `findByFirstnameIgnoreCase ...where UPPER(x.firstname) = UPPER(?1)`

JPQL - JOIN

- `SELECT agg.nick FROM AggregationEntity agg WHERE agg.anotherEntity.value = ?1`
- `SELECT anotherList.value FROM AggregationEntity agg JOIN agg.anotherEntityList anotherList`
- `SELECT another.value FROM AggregationEntity agg JOIN agg.anotherEntity another JOIN agg.anotherEntityList anotherList WHERE anotherList.value = ?1`

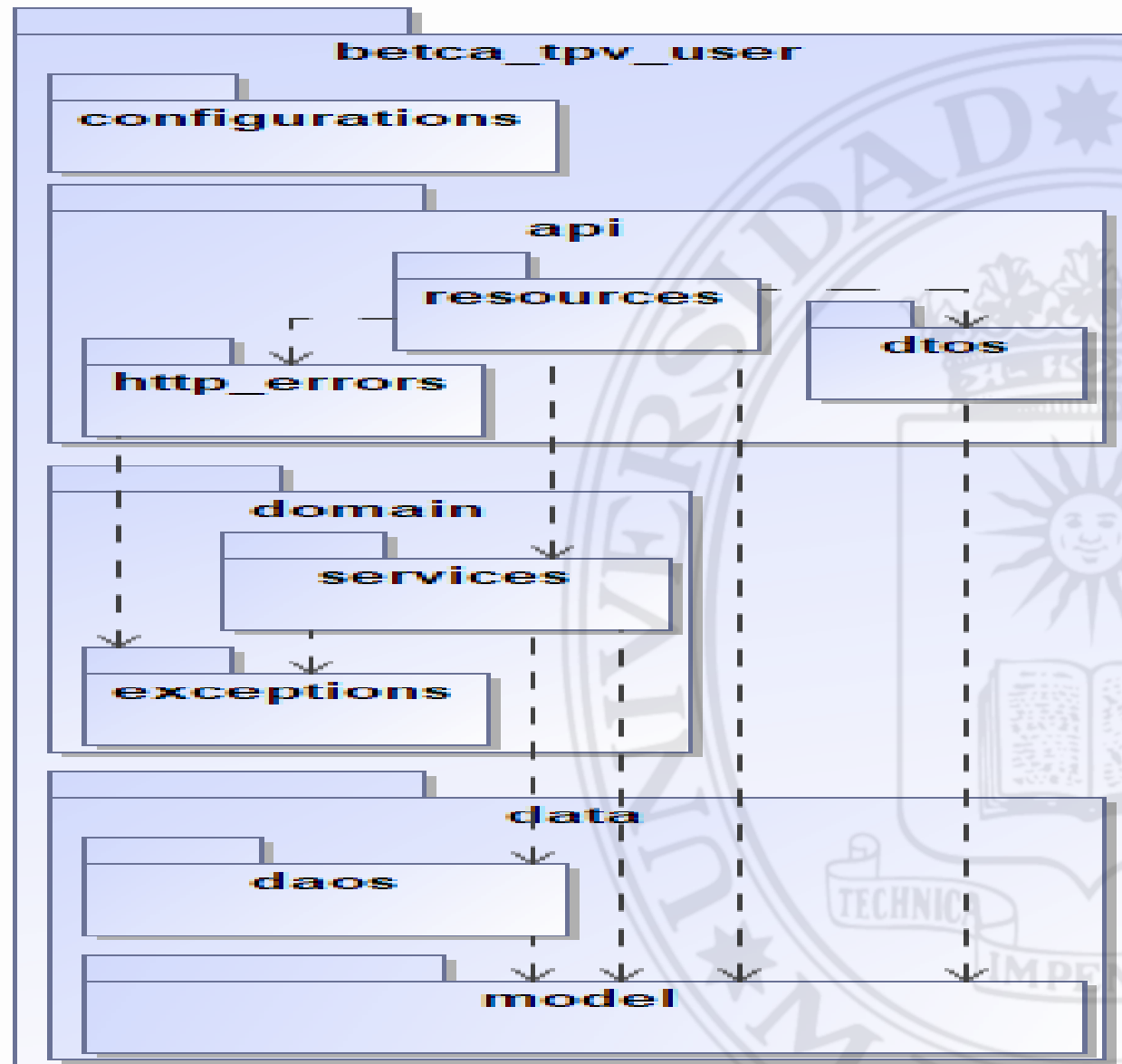


<https://github.com/miw-upm/betca-spring>

- **betca-jpa**

JPA en práctica

- Definir varias consultas para la entidad: *UnRelatedEntity*
- Definir una consulta con paginación para la entidad: *UnRelatedEntity*
- Definir varias consultas mediante JPQL para la entidad *UnRelatedEntity*
- Definir una consulta en SQL para la entidad *UnRelatedEntity*
- **Realizar los test correspondientes**





<https://github.com/miw-upm/betca-tpv-user>

- **betca-tpv-user::data**

 JPA en práctica

- Capa **Data** de la práctica **TPV-user**

Spring Data MongoDB. Dependencias



Dependencies

ADD DEPENDENCIES... CTRL + B

Lombok **DEVELOPER TOOLS**

Java annotation library which helps to reduce boilerplate code.

Spring Data MongoDB **NOSQL**

Store data in flexible, JSON-like documents, meaning fields can vary from document to document and data structure can be changed over time.

Spring Data Reactive MongoDB **NOSQL**

Provides asynchronous stream processing with non-blocking back pressure for MongoDB.

Embedded MongoDB Database **TESTING**

Provides a platform neutral way for running MongoDB in unit tests.

src/*/resources

- application-dev.properties
- test.properties
- @TestPropertySource

Library: JAR

- @SpringBootApplication

Embedded (Spring-boot 3)

- de.flapdoodle.embed.mongo.spring30x

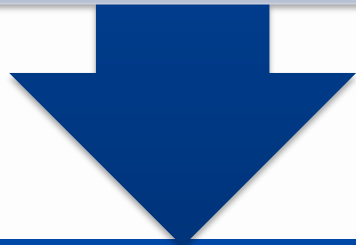
MongoDB (*.properties)

- MondoDB

```
# spring.data.mongodb.database=test  
# spring.data.mongodb.host=localhost  
# spring.data.mongodb.port=27017  
# spring.data.mongodb.username=  
# spring.data.mongodb.password=
```

Docker

*Iniciar el contenedor con
mongodb*



```
> docker exec -it mongo-db mongosh  
"mongodb://mongo:mongo@localhost:27017/tpv  
?authSource=admin"
```

Consola

- help
- db.version()
- db.stats()
- show databases
- use <myBD>
- show collections
- db.dropDatabase()
- db.coll1.drop()
- db.coll1.find()
- db.coll1.find().pretty()
- db.coll1.count()

Spring Data

Mongodb. <https://www.mongodb.com/>

Bajarse el fichero *.zip

<https://www.mongodb.com/try/download/community>

Descomprimir en equipo, FUERA del proyecto

Crear la carpeta de BD

```
>mongodb>mkdir data\db
```

Arrancar – Parar

```
>bin> mongod --dbpath **/data/db
```

```
Ctrl^C
```

Consola

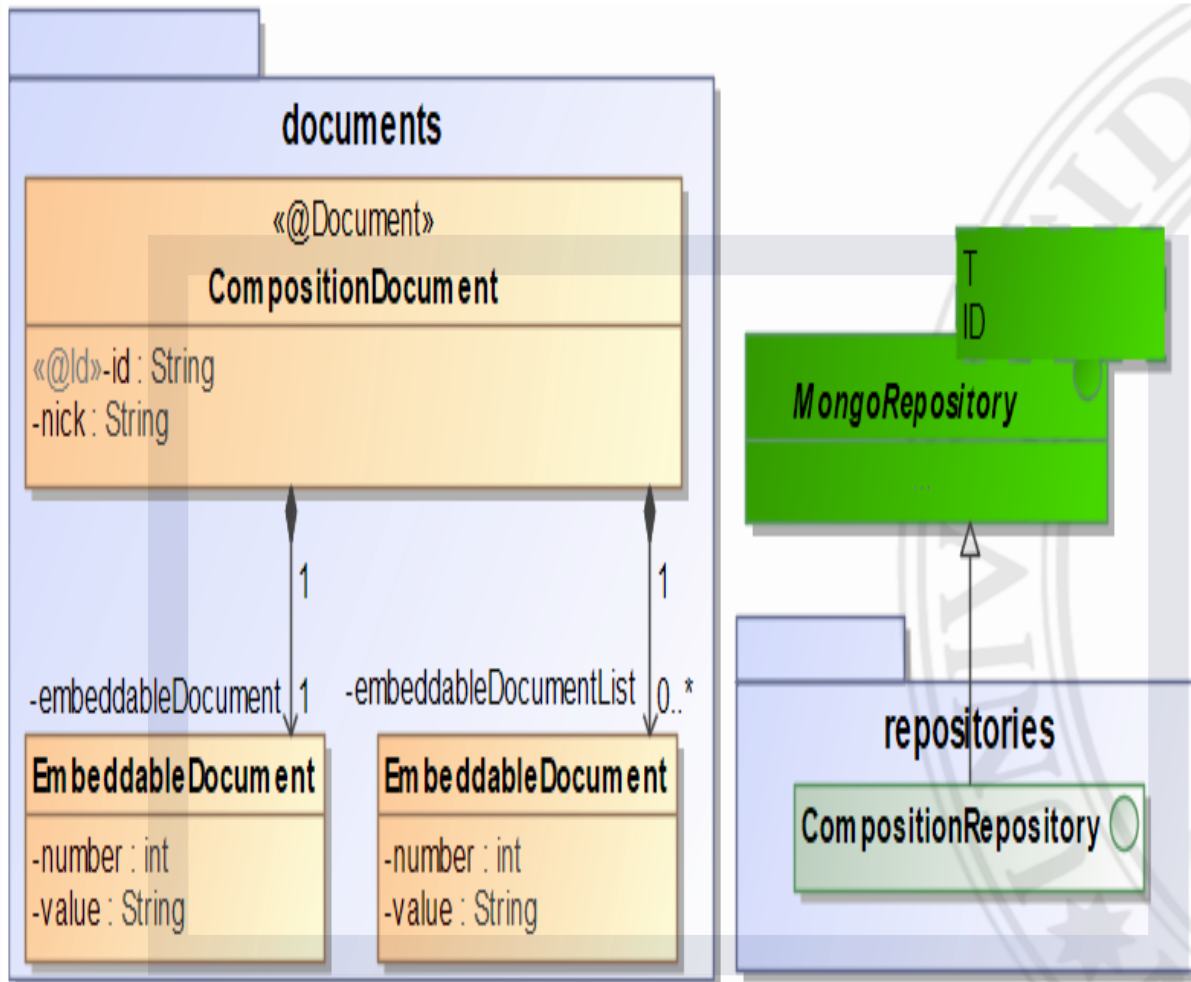
```
>bin> mongo
```

Consola

- help
- db.version()
- db.stats()
- show databases
- use <myBD>
- show collections
- db.dropDatabase()
- db.coll1.drop()
- db.coll1.find()
- db.coll1.find().pretty()
- db.coll1.count()

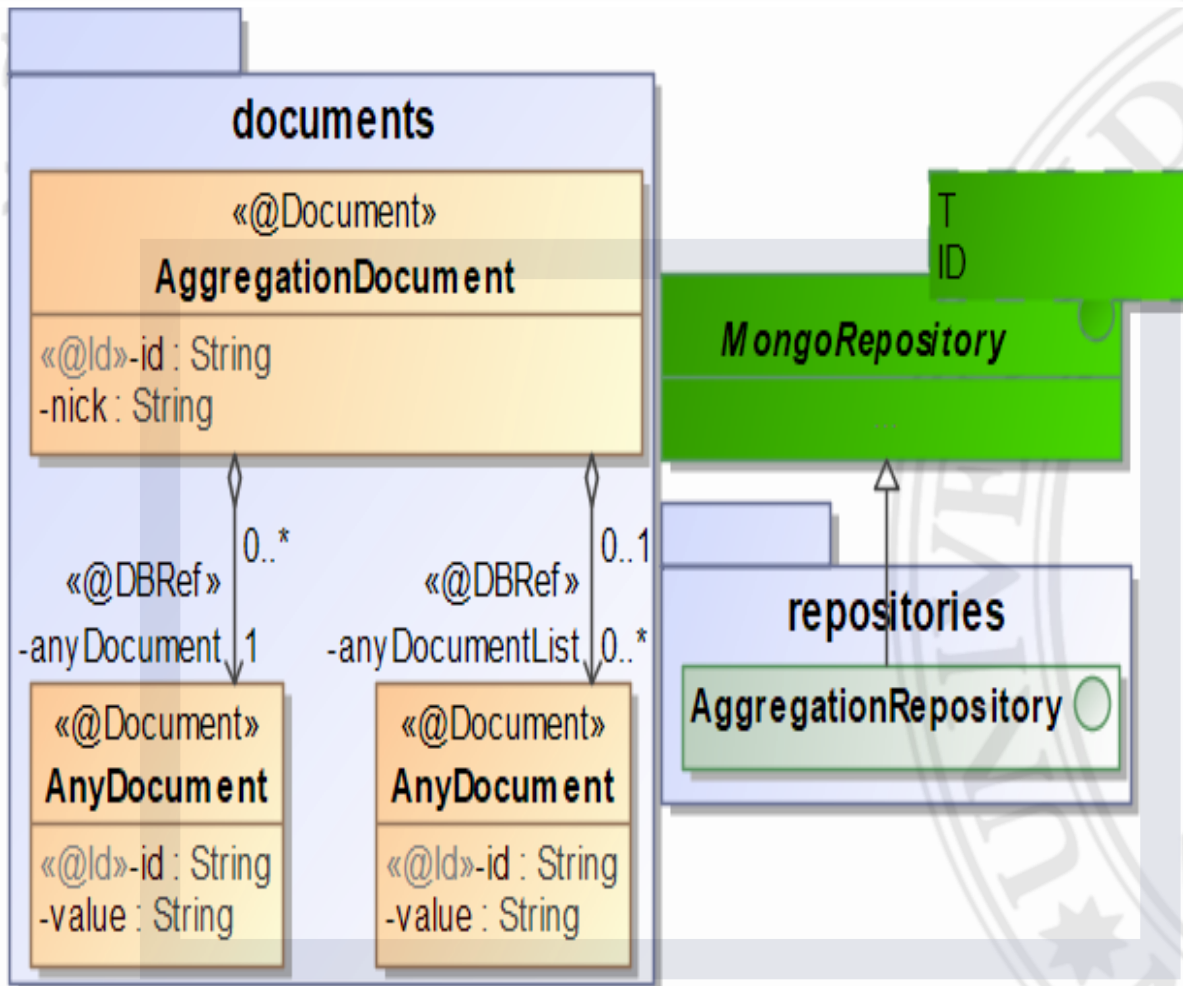
Consola

- `db.coll1.insert( { name:"jesus" } )`
 - `{ "_id" : ObjectId("5a86ad2e9d610a0fe33aa1ea"), "name" : "jesus" }`
- `db.coll1.insert( { _id:1 , name:"jesus" } )`
 - `{ "_id" : 1, "name" : "jesus" }`
- `db.coll1.insert ( { _id: "2", name:"ny", central: { _id:1, location:"Madrid" } } )`
 - `{ "_id" : "2", "name" : "ny", "central" : { "_id" : 1, "location" : "Madrid" } }`
- `db.coll1.insert( { _id: 3, name:"ny", central: [{ _id:2, location:"Madrid"},{_id:3, location:"Sevilla"}] } )`
 - `{ "_id" : 3, "name" : "ny", "central" : [ { "_id" : 2, "location" : "Madrid" }, { "_id" : 3, "location" : "Sevilla" } ] }`
- `db.other.insert( { _id: 1, name:"nameInOther" } )`
- `db.coll1.insert( { _id: 4, name:"nameInColl1", other: DBRef("other",1) } )`
 - `{ "_id" : 4, "other" : DBRef("other", 1) }`



Composición

- `@Document`
- `@Id`
- `@NoArgsConstructor`
- `@Data`



Agregación

- `@Document`
- `@Id`
- `@DBRef`
- `@NoArgsConstructor`
- `@Data`

Búsquedas por nombre de método VS Json Query

- **findByBirthdateAfter(Date date)** {"birthdate" : {"\$gt" : date}}
- **findByAgeGreaterThan(int age)** {"age" : {"\$gt" : age}}
- **findByAgeGreaterThanEqual(int age)** {"age" : {"\$gte" : age}}
- **findByBirthdateBefore(Date date)** {"birthdate" : {"\$lt" : date}}
- **findByAgeLessThan(int age)** {"age" : {"\$lt" : age}}
- **findByAgeLessThanEqual(int age)** {"age" : {"\$lte" : age}}
- **findByAgeBetween(int from, int to)** {"age" : {"\$gt" : from, "\$lt" : to}}
- **findByAgeIn(Collection ages)** {"age" : {"\$in" : [ages...]}}
- **findByAgeNotIn(Collection ages)** {"age" : {"\$nin" : [ages...]}}
- **findByFirstnameNotNull()** {"firstname" : {"\$ne" : null}}
- **findByFirstnameNull()** {"firstname" : null}
- **findByFirstnameLike(String name)** {"firstname" : name} (name as regex)
- **findByFirstnameNotLike(String name)** {"firstname" : { "\$not" : name }} (name as regex)
- **findByFirstnameContaining(String name)** {"firstname" : name} (name as regex)
- **findByFirstnameNotContaining(String name)** {"firstname" : { "\$not" : name }} (name as regex)
- **findByAddressesContaining(Address address)** {"addresses" : { "\$in" : address}}
- **findByAddressesNotContaining(Address address)** {"addresses" : { "\$not" : { "\$in" : address}}}
- **findByFirstnameRegex(String firstname)** {"firstname" : {"\$regex" : firstname }}
- **findByFirstname(String name)** {"firstname" : name}
- **findByFirstnameNot(String name)** {"firstname" : {"\$ne" : name}}
- **findByActiveIsTrue()** {"active" : true}
- **findByActiveIsFalse()** {"active" : false}
- **findByLocationExists(boolean exists)** {"location" : {"\$exists" : exists }}

Búsquedas por nombre de método VS Json Query

- `@Query("{nick:{$in:?0}}")` // Query NOT necessary
ListUnRelatedDocument findByNickInCollection nicks
- `@Query"?#{ [0] == null ? { $where : 'true' } : { nick : {$regex:[0], $options: 'i'} } }"`
ListUnRelatedDocument findByNickLikeIgnoreCaseNullSafe(String nick)
- `@Query "{$and:[\" // allow NULL: all elements
\"?#{ [0] == null ? {_id : {$ne:null}} : { nick : {$regex:[0], $options: 'i'} } },\"
\"?#{ [1] == null ? {_id : {$ne:null}} : { large : {$regex:[1], $options: 'i'} } }\"
\"] }"`
ListUnRelatedDocument findByNickLikeAndLargeLikeNullSafe(String nick,
String large)

Search

Mobile

666000666

Username

jes

Dni

Address

☒ Only Customer



<https://github.com/miw-upm/betca-spring>

- **betca-mongodb**

MongoDB en práctica

- Instalar Mongodb y observar como quedan las colecciones
- Definir varias consultas para la entidad: *UnRelatedEntity*
- Definir una consulta con paginación para la entidad: *UnRelatedEntity*
- Definir varias consultas mediante JSON Query para la entidad *UnRelatedEntity*
- **Realizar los test correspondientes**



<https://github.com/miw-upm/betca-tpv-core>

- **betca-tpv-core::persistencia**

 MongoDB en práctica

- Capa **Data** de la práctica **TPV-core**

Configuración

- Establecer la configuración de Spring y creación de los Beans.

Ends Points

- Establecer las **rutras de los End-points**.
- Establecer los parámetros y cuerpos requeridos.

Errores

- Manejar los errores que pudieran ocurrir.
- **JAMAS** se puede permitir un error no tratado.

Validación

- Validación de los datos de entrada.

Seguridad

- Control de acceso.



Dependencies

[ADD DEPENDENCIES... CTRL + B](#)

Lombok **DEVELOPER TOOLS**

Java annotation library which helps to reduce boilerplate code.

Spring Reactive Web **WEB**

Build reactive web applications with Spring WebFlux and Netty.

Spring Security **SECURITY**

Highly customizable authentication and access-control framework for Spring applications.

Validation **I/O**

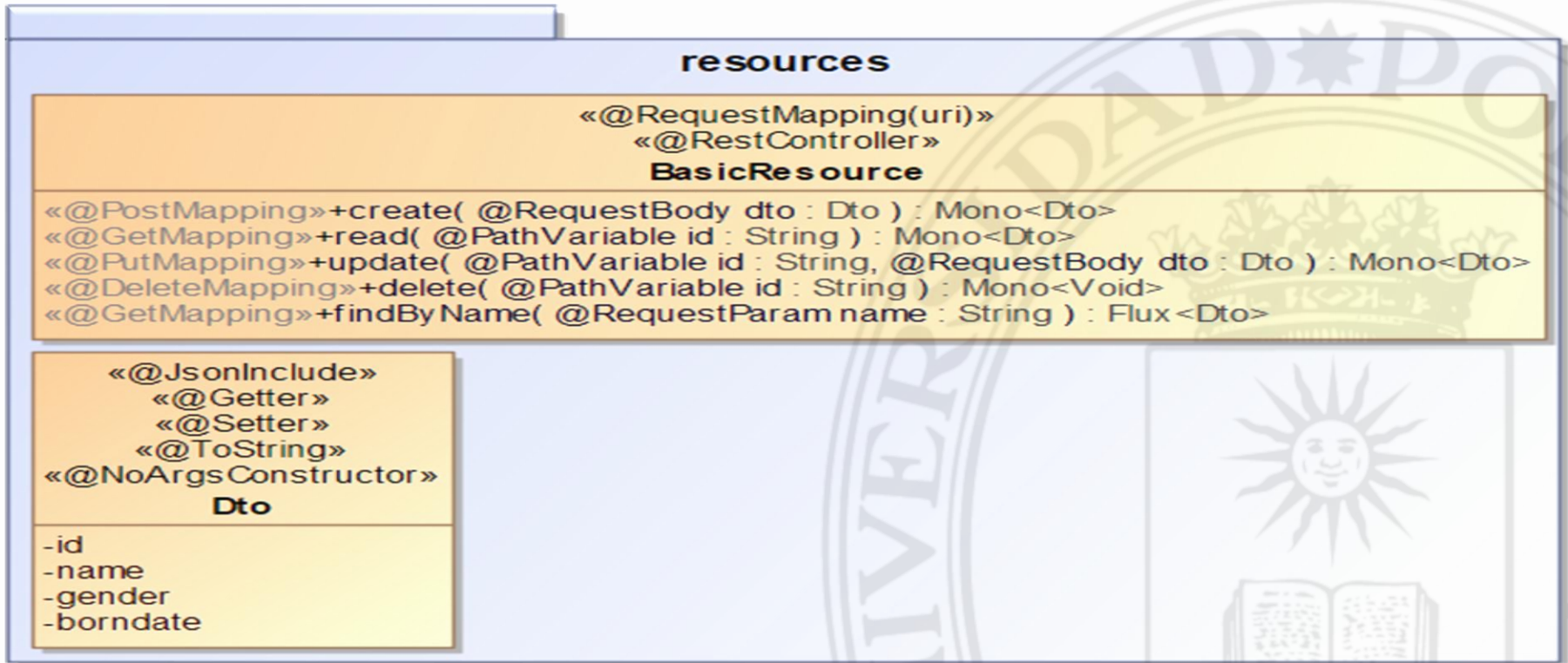
JSR-303 validation with Hibernate validator.

src/*/resources

- application.properties
- test.properties

@RestTestConfig

- @SpringBootTest
- @AutoConfigureWebTestClient
- @TestPropertySource



PUT /basic/666 {dto}

GET /basic/search?name=Miw&q=other

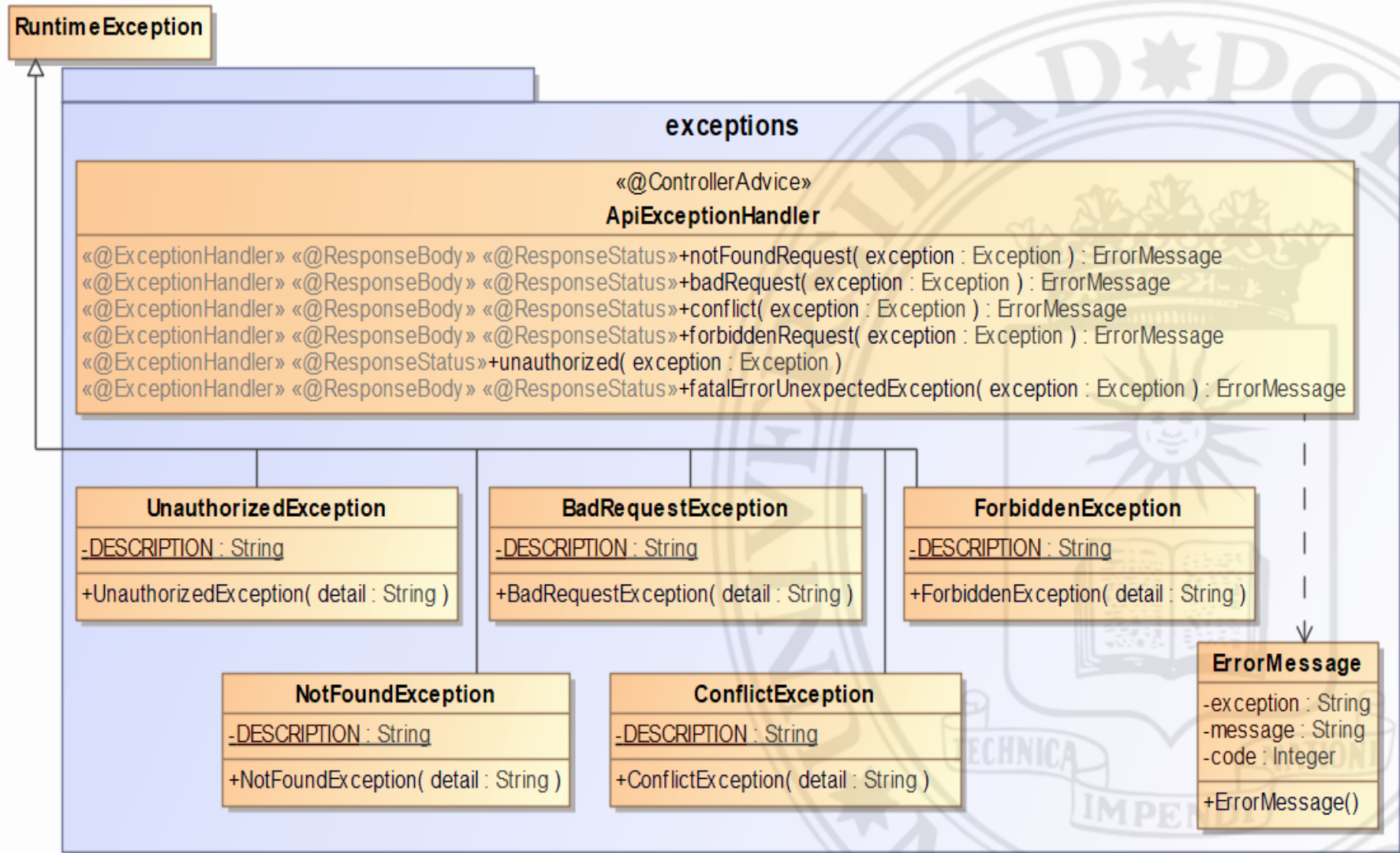


<https://github.com/miw-upm/betca-spring>

- **betca-rest**

Rest en práctica

- Basic Resource
- Añadir end-point PATCH /basic, recibe en el cuerpo `[[id,name]]` para actualizar



resources

«@RequestMapping(uri)»
«@RestController»

ExceptionHandler

«@PostMapping»+read(@Valid @RequestBody dto : ValidateDto)

validations

ValidatedDto

«@NotNull» «@Min»-id : Integer
«@NotBlank»-name : String
«@JsonInclude»-gender
«@JsonFormat» «@Past»-bornDate : LocalDateTime

Validador propio: *PositiveBigDecimal*

Validadores

- @DecimalMax, @DecimalMin
- @Digits
- @Email
- @Future, @FutureOrPresent
- @Max, @Min
- @Negative, @NegativeOrZero
- @Null, @NotNull
- @NotEmpty, @NotBlank
- @Past, @PastOrPresent
- @Positive, @PositiveOrZero
- @Pattern
- @Size



<https://github.com/miw-upm/betca-spring>

- **betca-rest**

 Rest en práctica

- *ExceptionResource, ValidatedDto*
- Añadir...

Role / Scope

- Es un nombre abstracto para configurar el permiso de acceso de un conjunto de recursos de una aplicación.

Usuario

- Es una identidad única reconocida por el servidor. Un usuario puede tener varios roles.

Recursos

- Se controla las acciones de diferentes roles.
- **Distribuido:** anotaciones en el recurso.
- Centralizado: Uri's.

Autenticación

- Memoria
- **Propio**
- API Key o Token
- LDAP
- **OAuth2**
- **OpenId Connect**
- ...

Seguridad

- OAuth2 se ha convertido en el estándar para la autorización en aplicaciones modernas, ofrece una forma segura, flexible y escalable el acceso a recursos.

OpenID Connect

- Identifica de forma segura a los usuarios. Es una capa construida sobre OAuth2. Se genera un *Token JWT*

OAuth2

- Es un protocolo de autorización que permite a las aplicaciones obtener acceso limitado a recursos protegidos en nombre de un usuario, sin necesidad de exponer sus credenciales. Se centra en el acceso a los recursos mediante *Token JWT*

Token JWT

- Son token que representan permisos y tienen un tiempo de vida limitado.

Scope & Roles

- Los tokens incluyen *Scopes* que definen el alcance o los permisos otorgados. Pueden mapear también *Roles*.

Spring

Seguridad: JSON Web Token

JWT	Header _{code64}	Payload _{code64}	Signature _{code64}
• https://jwt.io • https://auth0.com	• Se define el tipo. • RS256	• Datos o carga del token. • <i>Subject, Audience...</i>	• Se firma con <i>clave secreta</i> la carga del <i>token</i> para saber si ha sido alterado.

```
eyJraWQiOiI3ZmMxMGMwMi1hM2YyLTQ0YzMtOGQzNi05ODA4MTcwMWZjNzciLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiI2liwiYXV
kIjoic3BhQ2xpZW50SWQlLCJuYmYiOiJlE3NTg1MzU1MTYsInNjb3BliIpIm9wZW5pZCI6InByb2ZpbGUlLCJvZmZsaWZlIjoiIiwiaWF0Ijoi
yJdLCJyb2xlcyciOiI6ImFkbWluLiwiiaXNzIjoiaHR0cDovL2xvY2FsaG9zdDo4MDgwL2FwaS9nb2EtdXNlciIsIm5hbWUiOiJhZG1pbiIsImV4
cCI6MTc1ODUzOTExNiwiWF0iOjoxNzU4NTM1NTE2LCJqdGkiOiJmZDcyODkxZi03Y2EwLTQwZDktYWJmYS0yMDU2ODVhMGMw
MyMDYifQ.SCY1hL3Sv0KfQk_NHxSXPzWBuszEd5EmYQvz2eLHiyibNnvOfwHMOcJA7nRK4m5kQP2FNoaqkT4DGk_zlM6CTgY2
H2XnQdT7klCMsrUmvLR1anEq2aj7Nuj_qrdb1QNye1tyNW_iJwG5TuyFo49rQInGagMFU7ZAm4JfgYl4Y0wlH8UBQIPZG__f8Wa
AE41Wv_SFfaSfmABpPkAuSt4pSHHd4LrM4rgKQm9n04w0iJcPWl_8qjRF0GeY9BWhRoVVszxQf5cnwhPbT_8nRLSX8Qfx0HgBm6n
_puAZVOiZvgg9Cc59XoH_la60qmy2THKP37jlvGXnQK2xuRtSQ
```

```
{
  "kid": "7fc10c02-a3f2-44c3-8d36-98081701fc77",
  "alg": "RS256"
}
```

```
{
  "sub": "6",
  "aud": "spaClientId",
  "nbf": 1758535516,
  "scope": ["openid", "profile", "offline_Access"],
  "roles": "admin",
  "iss": "http://localhost:8080/api/goa-user",
  "name": "admin",
  "exp": 1758539116, "iat": 1758535516,
  "jti": "fd72891f-7ca0-40d9-abfa-205685d1c206"
}
```

HTTPS

Header

Authorization

Authorization

Basic user:password_{code64}

Bearer jwt



Basic user:api-key_{code64}



URL:/*/api-key

Resource Owner

- Propietario final, es el usuario que se inicia sesión en al App, para su uso.

Client

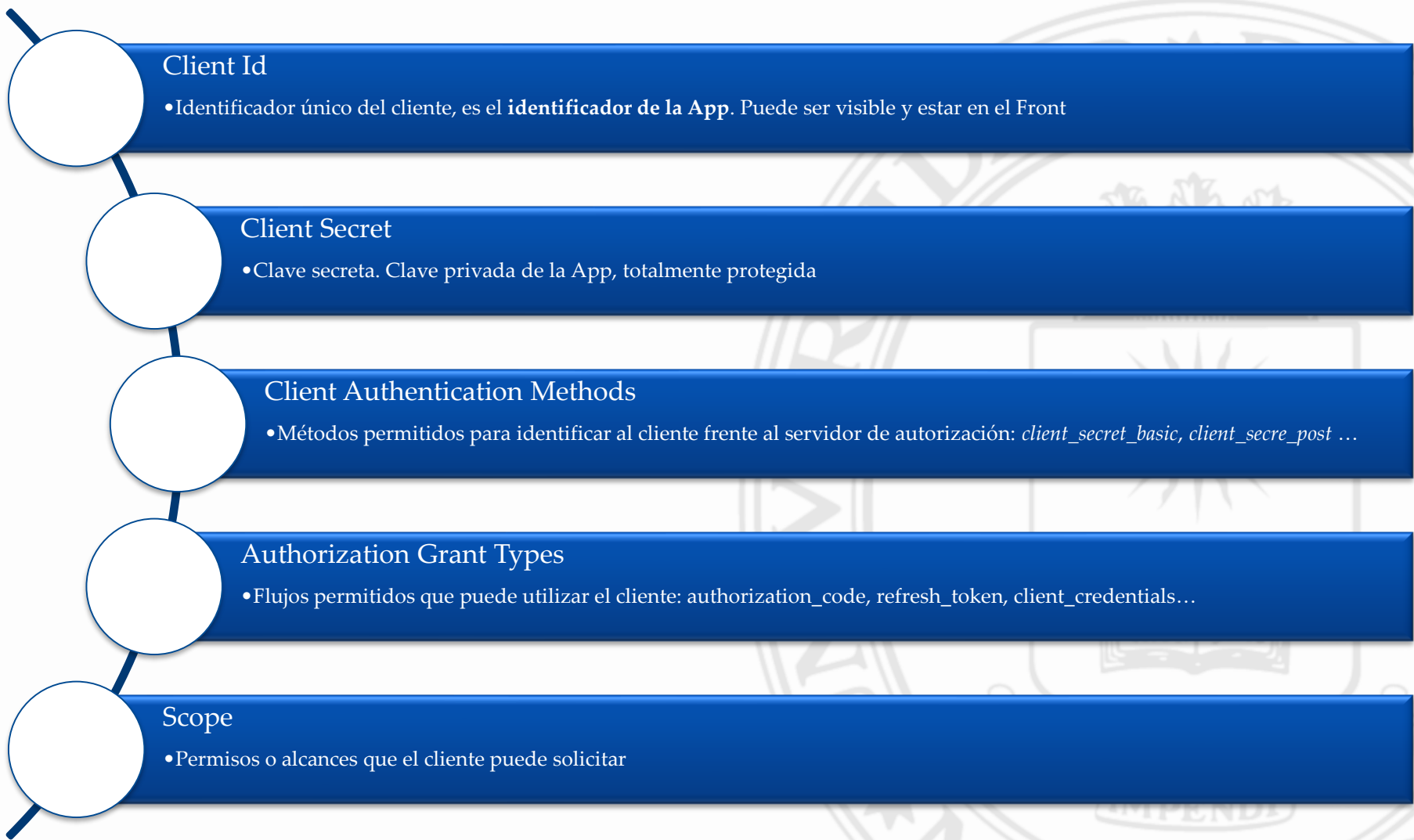
- Es la App: *Front-end & Back-end*.

Authorization Server

- Es el autentica al usuario y emite el *token (JWT)*: *goa-user*.

Resource Server

- Son los API's protegidos que se accede por *token (JWT)*: *goa-user, goa-engagement, goa-billing*.



Client Id

- Identificador único del cliente, es el **identificador de la App**. Puede ser visible y estar en el Front

Client Secret

- Clave secreta. Clave privada de la App, totalmente protegida

Client Authentication Methods

- Métodos permitidos para identificar al cliente frente al servidor de autorización: *client_secret_basic*, *client_secret_post* ...

Authorization Grant Types

- Flujos permitidos que puede utilizar el cliente: *authorization_code*, *refresh_token*, *client_credentials*...

Scope

- Permisos o alcances que el cliente puede solicitar

Spring

OAuth2. Métodos de autenticación

CLIENT_SECRET_BASIC

- El cliente envía su *client_id* y *client_secret* en la cabecera HTTP “Authorization” usando el esquema Basic. Es el método más utilizado y **recomendado**.

CLIENT_SECRET_POST

- El cliente envía su *client_id* y *client_secret* en el cuerpo de la solicitud.

CLIENT_SECRET_JWT

- El cliente envía su *client_id* dentro de un token JWT firmado por su *client_secret*, en el cuerpo de la petición.

PRIVATE_KEY_JWT:

- El cliente utiliza un par de claves asimétricas.

TLS_CLIENT_AUTH

- El cliente se autentica mediante un certificado TLS

NONE

- Clientes públicos

Spring

OAuth2. Flujos de obtención del token

Authorization Code Grant *Webs & móviles*

1. El cliente (*App: goa-front*) se redirige al servidor de autorización (*goa-user*) que muestra un formulario de *login*, se asegura con *PKCE* (*crea un código secreto para un proceso de intercambio seguro*). Se apoya en *angular-auth-oidc-client*, que trabaja con *PKCE*.
2. El usuario se identifica y se le da un *código de autorización* de un solo uso.
3. El cliente (*App: goa-front*) envía el código, y el servidor (*goa-user*) devuelve un *access token* y un *refresh token*.

Client Credentials Grant *API to API*

- El cliente se autentica directamente usando su propio *client_id* y *client_secret*, sin involucrar a un usuario.

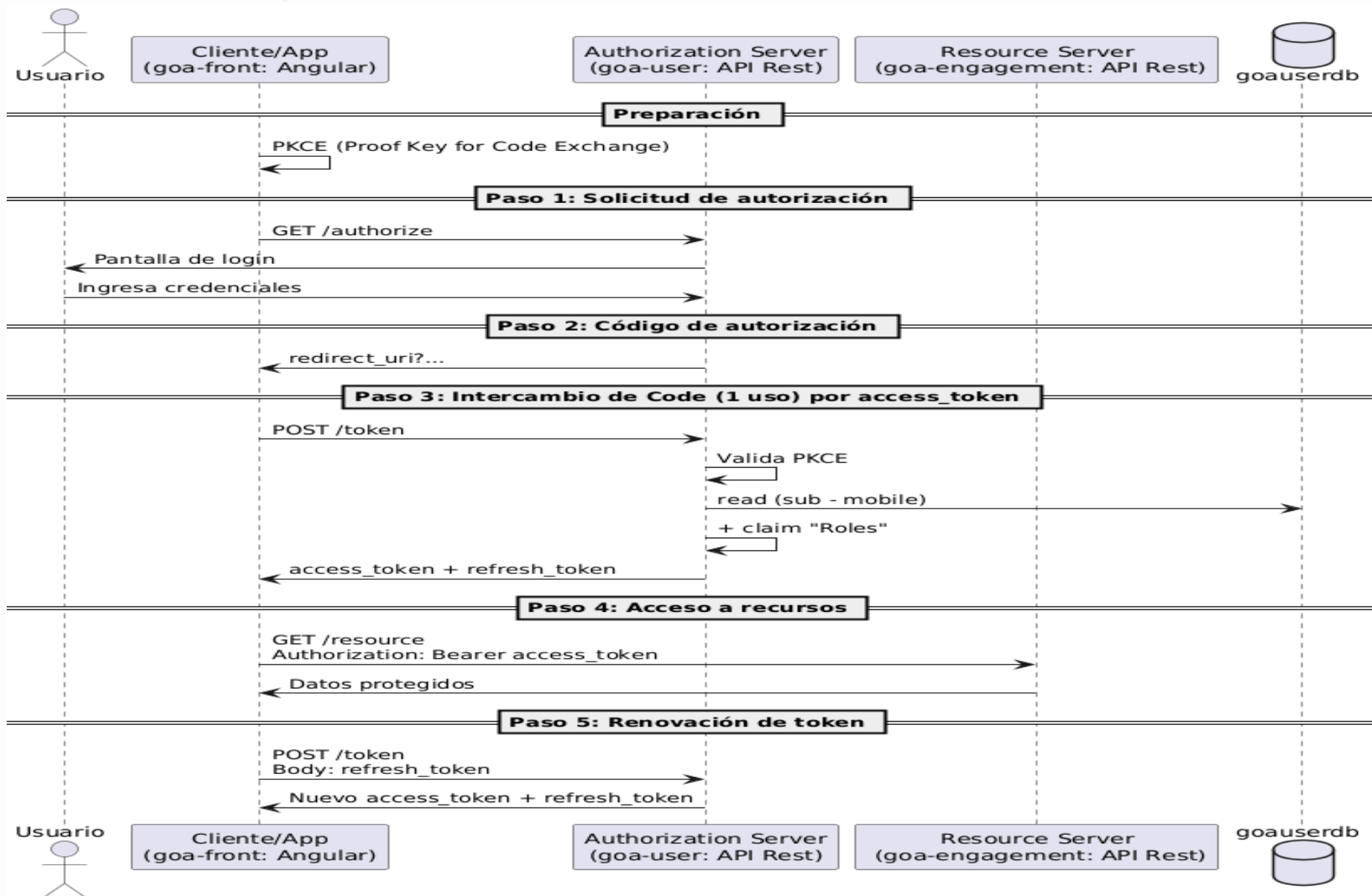
Refresh Token

- Con el token, el cliente puede recibir un token de refresco, para poder solicitar otro token de acceso cuando caduque.

Device Authorization Grant

- Para dispositivos con capacidad limitada (tv, consolas...). Se solicita un código de dispositivo y el usuario lo acepta con otro medio (móvil).

Spring OAuth2. Flujos: Usuario



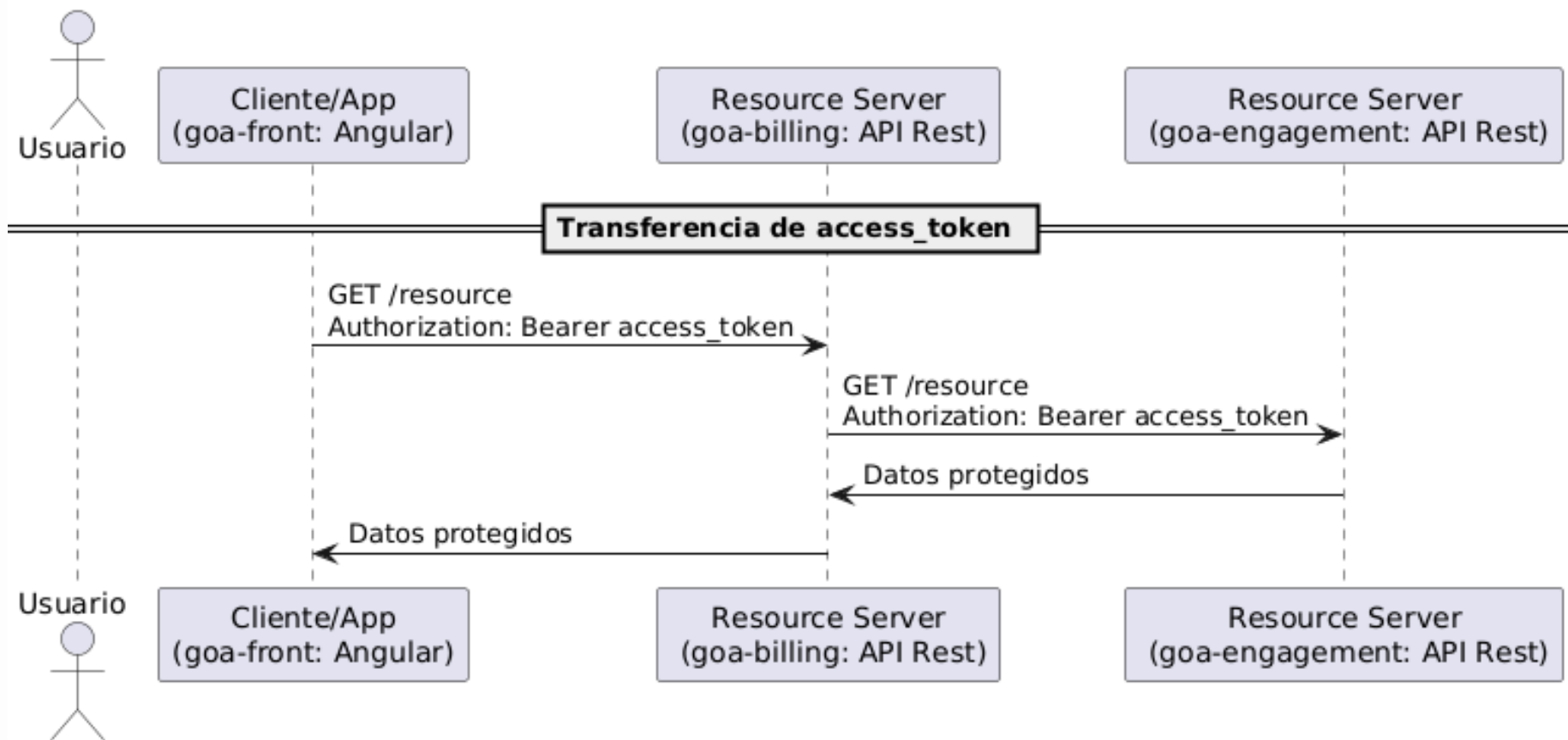
RegisteredClient spaClient =

```
RegisteredClient.withId(UUID.randomUUID().toString())
    .clientId((this.oAuth2Properties.getSpaClientId()))
    .clientAuthenticationMethod(ClientAuthenticationMethod.NONE)
    .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
    .authorizationGrantType(AuthorizationGrantType.REFRESH_TOKEN)
    .redirectUri(this.oAuth2Properties.getSpaLoginRedirectUri())
    .scopes(scopes -> scopes.addAll(Scope.allValues()))
    .tokenSettings(tokenSettings)
    .clientSettings(ClientSettings.builder()
        .requireAuthorizationConsent(false)
        .requireProofKey(true)
        .build())
    .postLogoutRedirectUri(this.oAuth2Properties.getSpaLogoutRedirectUri())
    .build();
```

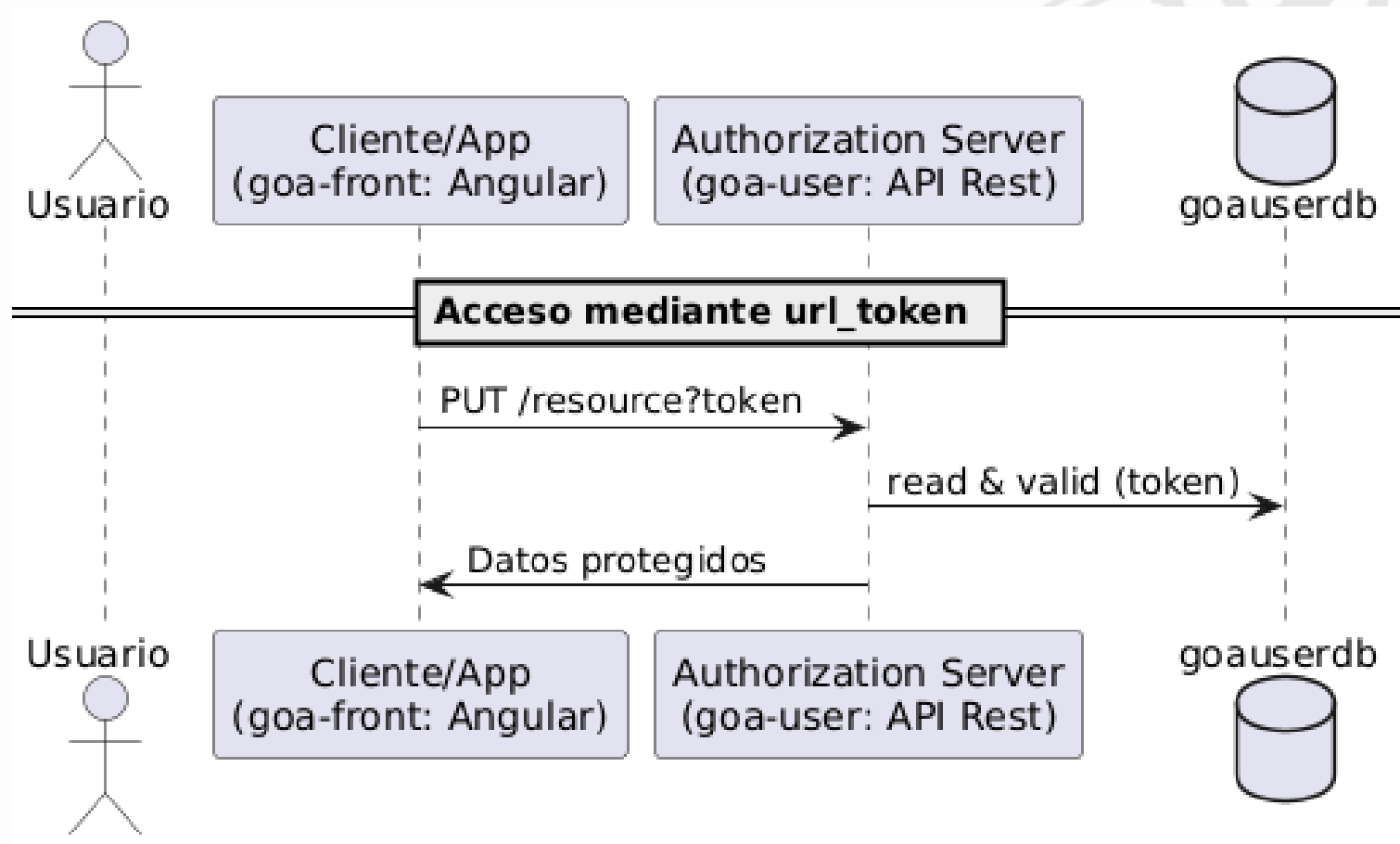
Se usa PKCE

Se añade al *access_token*, un *claim: roles*, que dependerá de la cuenta de usuario para obtener el rol asociado al mismo.

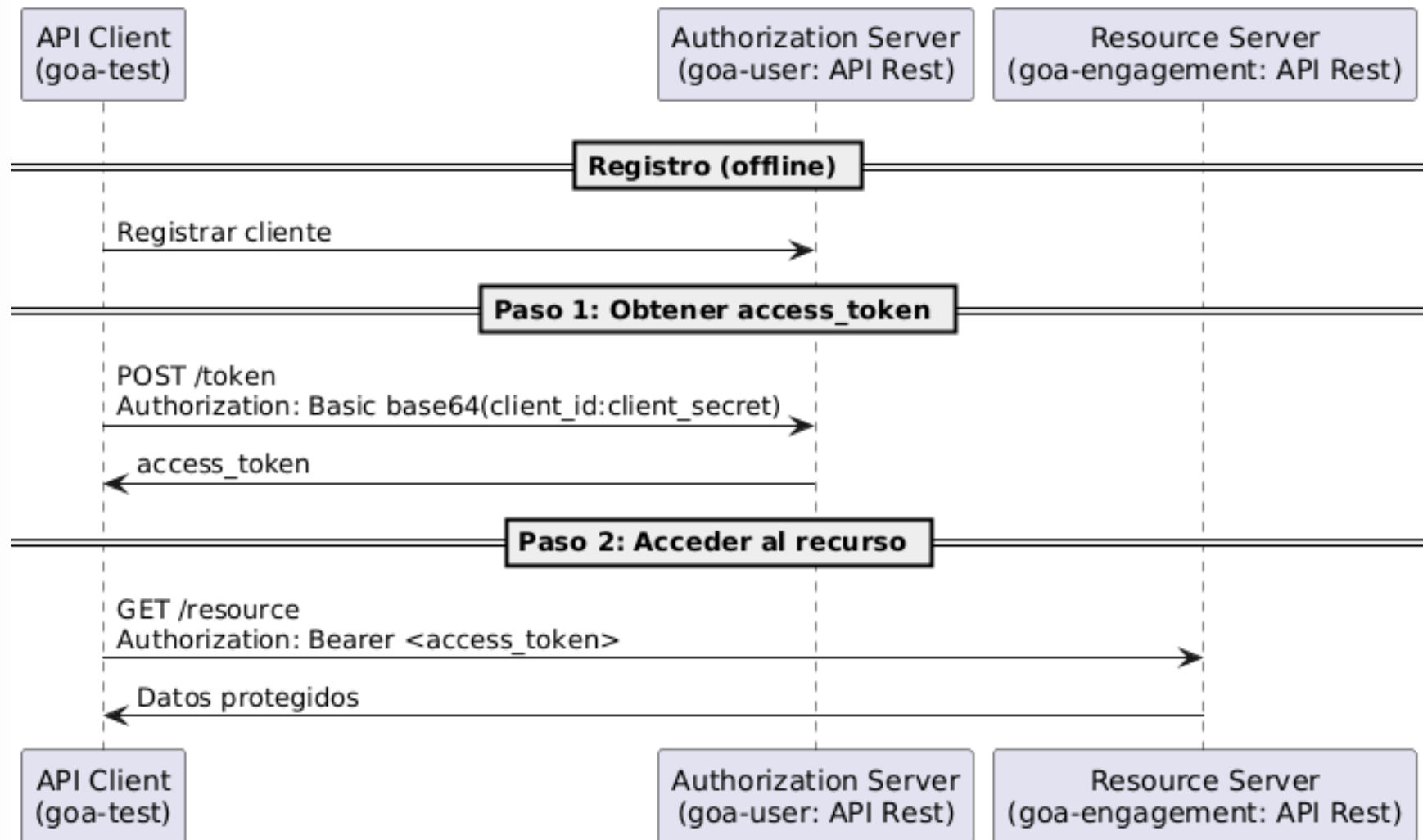
Spring OAuth2. Flujos: API a API



Spring OAuth2. Flujos: Sin Flujo



Spring OAuth2. Flujos: API a API



Spring OAuth2. Flujos: API a API

RegisteredClient apiClient =

```
RegisteredClient.withId(UUID.randomUUID().toString())
    .clientId(this.oAuth2Properties.getApiClientId())
    .clientSecret(passwordEncoder.encode(this.oAuth2Properties.getApiClientSecret()))
    .clientAuthenticationMethod(ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
    .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
    .scope(Scope.PROFILE.value())
    .tokenSettings(tokenSettings)
    .build();
```

Spring

OAuth2. @PreAuthorize("isAnonymous()")

isAuthenticated()

isAnonymous()

hasRole('customer')

hasAnyRole('admin','manager','operator')

hasRole('customer') and #id == authentication.name

hasAuthority('SCOPE_USER')

@PostAuthorize("returnObject.owner == authentication.name")



<https://github.com/miw-upm/betca-spring>

- **betca-rest**

Rest en práctica

- *SecurityResource, CustomReactiveUserDetailsService*
- Crear el end-point DELETE */basic-auth/{ID}*, sólo podrá borrar el usuario “u1” con el rol de “PLAYER”, se configura con anotaciones.

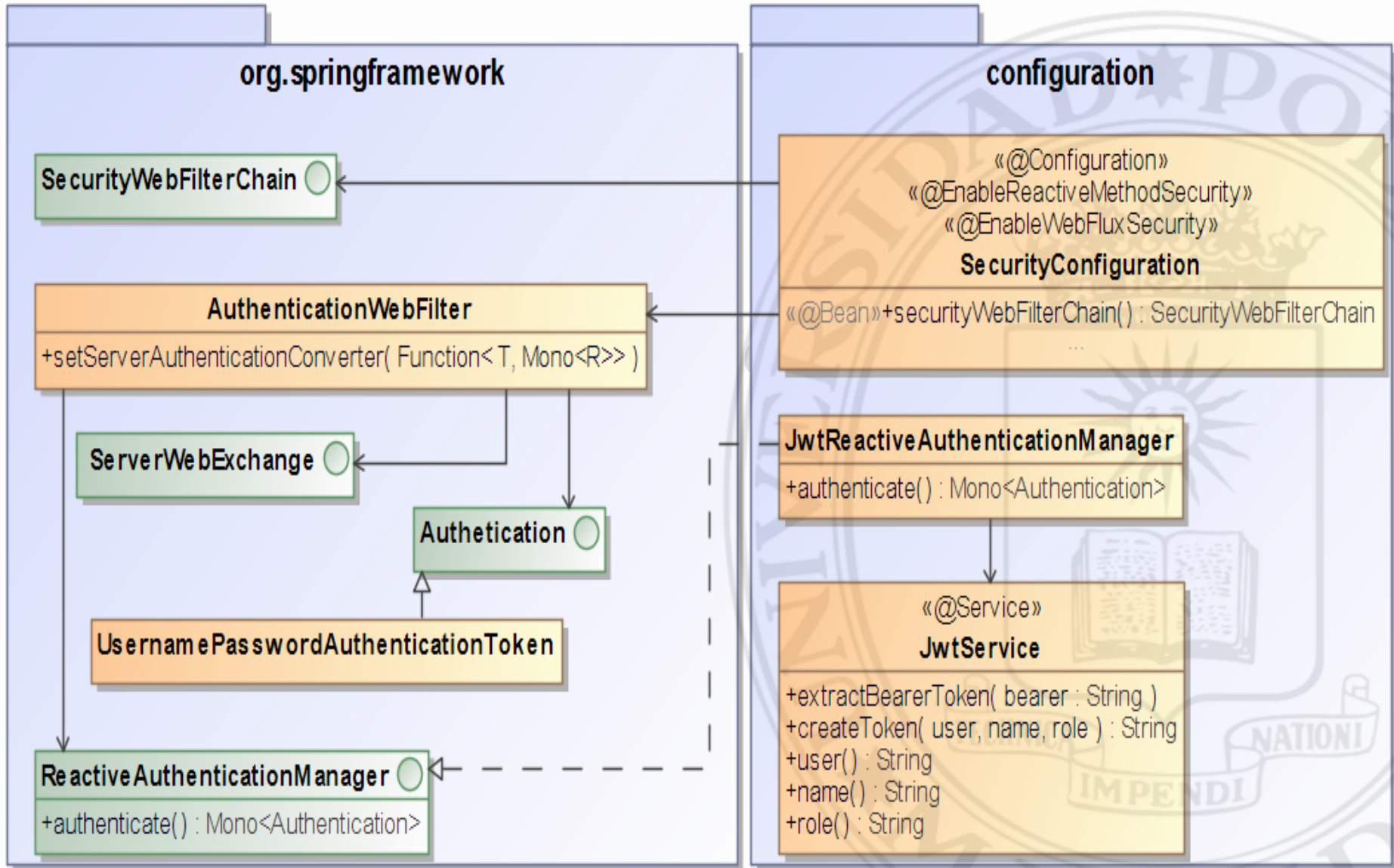


<https://github.com/miw-upm/betca-spring>

- **betca-rest**

 Rest en práctica

- *JwtReactiveAuthenticationManager, JwtService*
- Añadir...



Servicio email

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-mail</artifactId>  
</dependency>
```

pom.xml

application.properties

```
# Email #####  
# Se debe configurar la cuenta de gmail para soportar la conexion de la aplicacion  
# Login to Gmail  
# Access the URL as https://www.google.com/settings/security/lesssecureapps  
# Select "Turn on"  
spring.mail.defaultEncoding=UTF-8  
spring.mail.host=smtp.gmail.com  
spring.mail.port=587  
spring.mail.username=${MAIL_USER}  
spring.mail.password=${MAIL_PASS}  
spring.mail.properties.mail.smtp.auth=true  
spring.mail.properties.mail.smtp.starttls.enable=true  
spring.mail.test-connection=false
```

```
# Mail  
spring.mail.username=notUser  
spring.mail.password=notPass
```

test.properties

Servicio email

```
@TestConfig
public class MailIT {

    @MockBean
    private MailSender mailSender;

    @Autowired
    private MailService mailService;

    @Test
    public void testSendMessage() {
        final ArgumentCaptor<SimpleMailMessage> captor =
            ArgumentCaptor.forClass(SimpleMailMessage.class);
        this.mailService.send("test@gmail.com", "test message");
        verify(mailSender).send(captor.capture());
        assertEquals(mailService.getFrom(), captor.getValue().getFrom());
        assertEquals("test@gmail.com", captor.getValue().getTo()[0]);
        assertEquals(mailService.getSubject(), captor.getValue().getSubject());
        assertEquals("test message", captor.getValue().getText());
    }
}
```

¿Qué es?

- La arquitectura de **microservicios** es un enfoque para desarrollar aplicaciones distribuidas donde *componente* (microservicio) se ejecuta como un proceso independiente.

Desarrollo modular

- Componentes pequeños con equipos multidisciplinares. Filosofías de producto.

Descentralización de BD

- Cada componente gestiona su BD.

Despliegues independientes

- Evoluciones rápidas y de menor riesgo. Mejor efectividad en el balanceo de carga. Mayor facilidad de reusabilidad.

Diversidad Tecnológica

- Cada componente puede ser de diferente tecnología.

Sistema distribuido

- Mas complicado y complejo del despliegue y la consistencia del sistema completo.

Complejidad operacional

- Se necesitan de varios servicios para realizar una operación.

Patrones de diseño

- El punto clave está en la comunicación, la coordinación y el seguimiento.

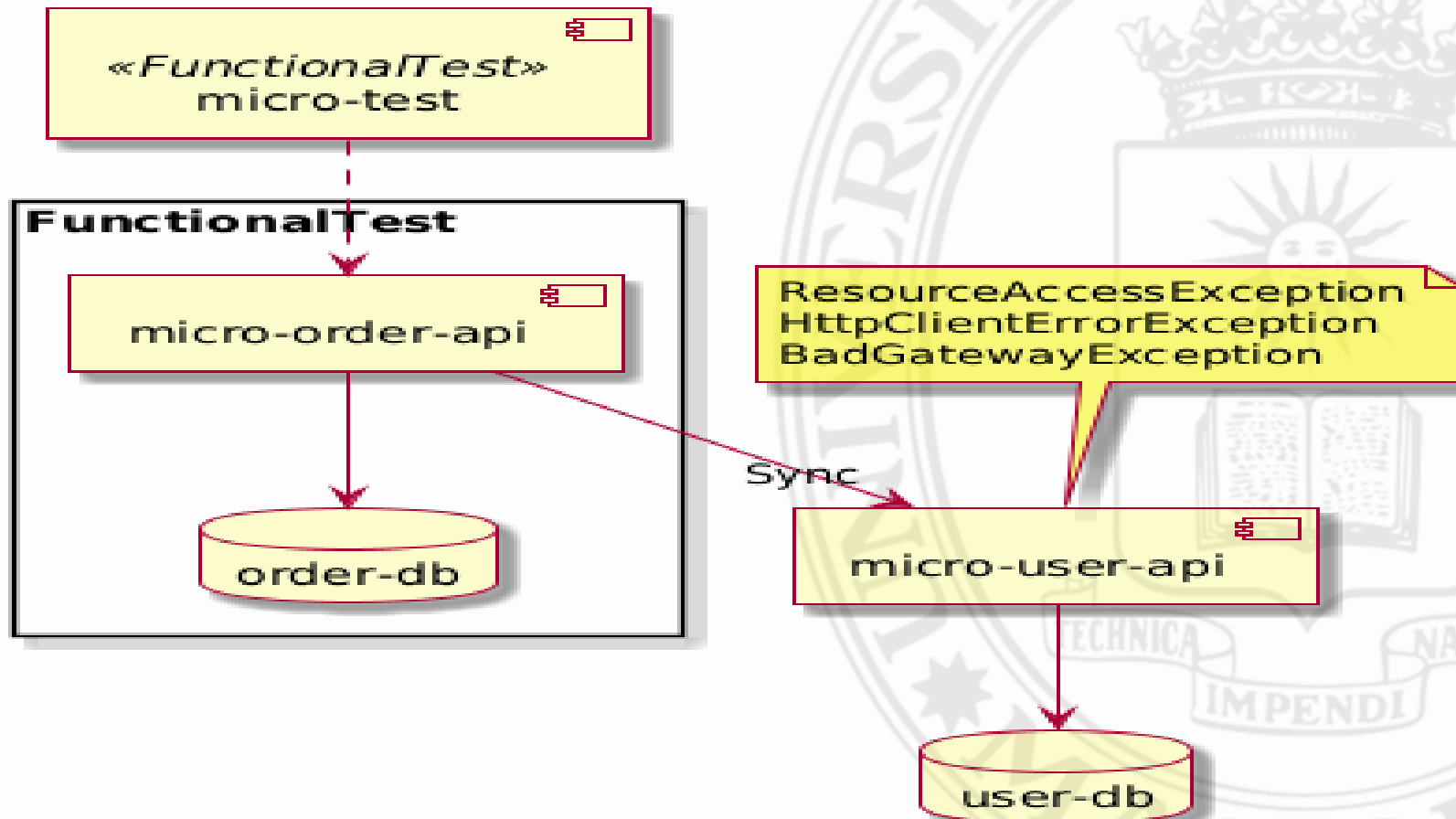
Comunicación síncrona: REST

- Normalmente con HTTP y servicios REST
- El servicio cliente llama al proveedor y espera respuesta.
- Fácil de implementar y facilidad conceptual.
- **Inconvenientes:**
 - Acoplamiento temporal, si un servicio va lento, por cascada, afecta al resto.
 - Escalabilidad limitada, por el consumo excesivo de los *threads*.



Microservicio. Release-1.0

- micro-user-api: release-1.0.0, micro-order-api:release-1.0.0*



Comunicación Asíncrona - Mensajería

- Se realiza a través de mensajería.
- No bloqueantes, se envía en mensaje y se continua sin esperar a su procesamiento.
- Tolerancia a fallos, los mensajes se acumulan hasta poder ser procesados.
- Soporta procesamiento masivo en tiempo real.
- Motores: *Kafka*, *RabbitMQ*, *AWS SQS*...
- **Inconvenientes:**
 - Se necesita un sistema de mensajería. Un componente con mantenimiento.
 - Complejidad en la implementación y seguimiento.

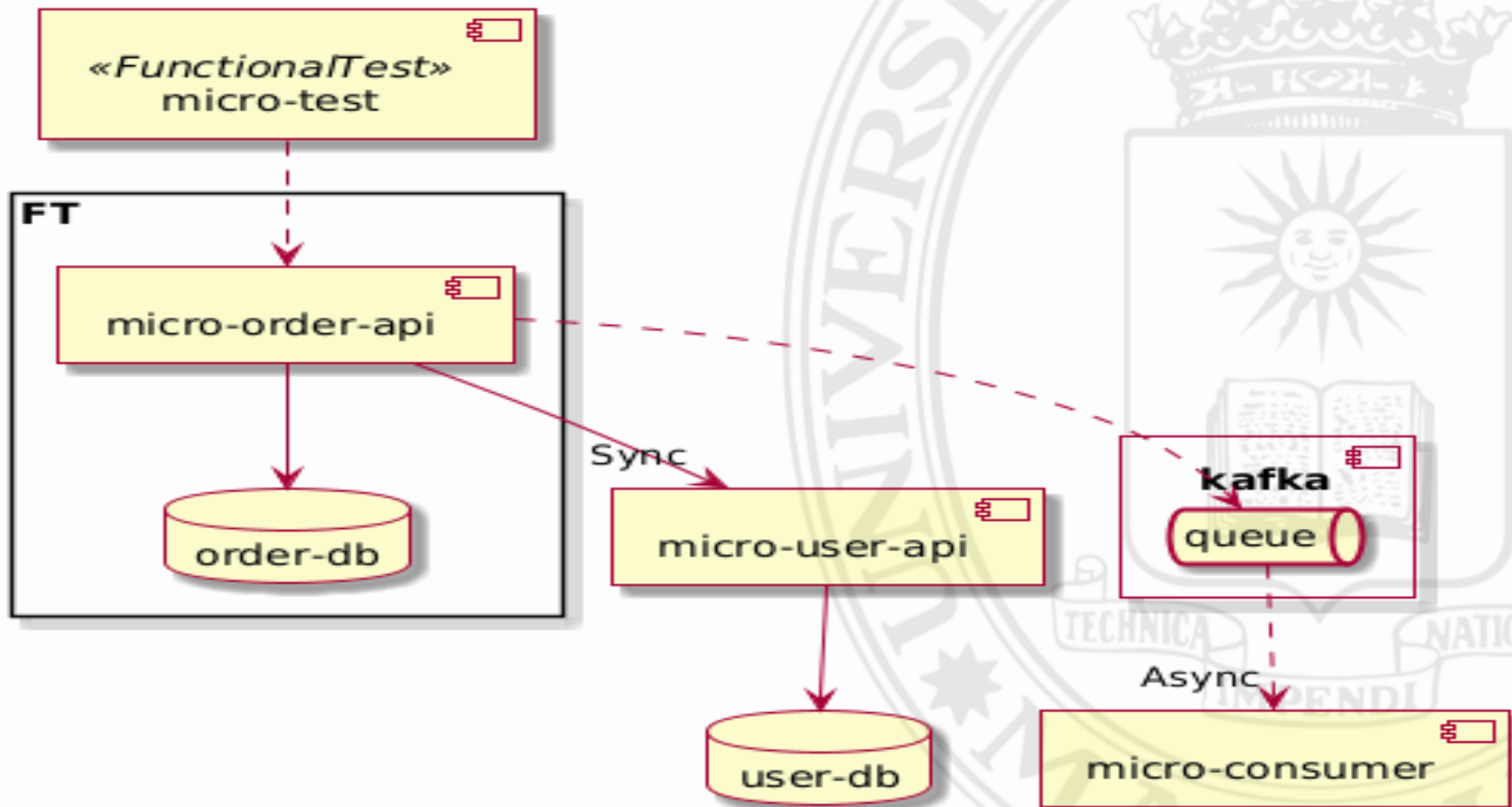
Conceptos

- **Topic.** Es una categoría o canal lógico al que se publican los mensajes. Los productores publican mensajes en un tópico y los consumidores se suscriben a él para recibir esos mensajes.
- **Partition.** Cada *topic* se divide en una o más particiones, que son segmentos de logs ordenados. Cada partición se puede procesar de forma independiente. El orden se garantiza dentro de cada partición no globalmente.
- **Offset.** Es el índice o posición de un mensaje dentro de una partición.
- **Consumer Group.** Conjunto de consumidores. *Kafka* asigna cada partición a un único consumidor.
- **Replication.** Cada partición puede replicarse en varios *brokers* para garantizar la tolerancia a fallos. Un *broker* es un servidor



Microservicio. Comunicación en práctica

- *micro-user-api: release-1.1.0, micro-order-api: release-1.1.0, micro-consumer: release-1.1.0, kafka*



Spring

Microservicios. *Service Discovery*

Service Discovery

- El patrón **Service Discovery** se usa para manejar la **dinámica de direcciones de red**. En un entorno con múltiples microservicios, estos pueden escalar horizontalmente, moverse entre instancias o cambiar de IP.
- En lugar de que cada microservicio tenga que conocer manualmente la dirección IP y el puerto de otros servicios, se usa un **Service Registry**.
- **Spring Cloud** con *Eureka* y *Feign*, nos aporta el patrón y la implementación automática de *WebClient*.

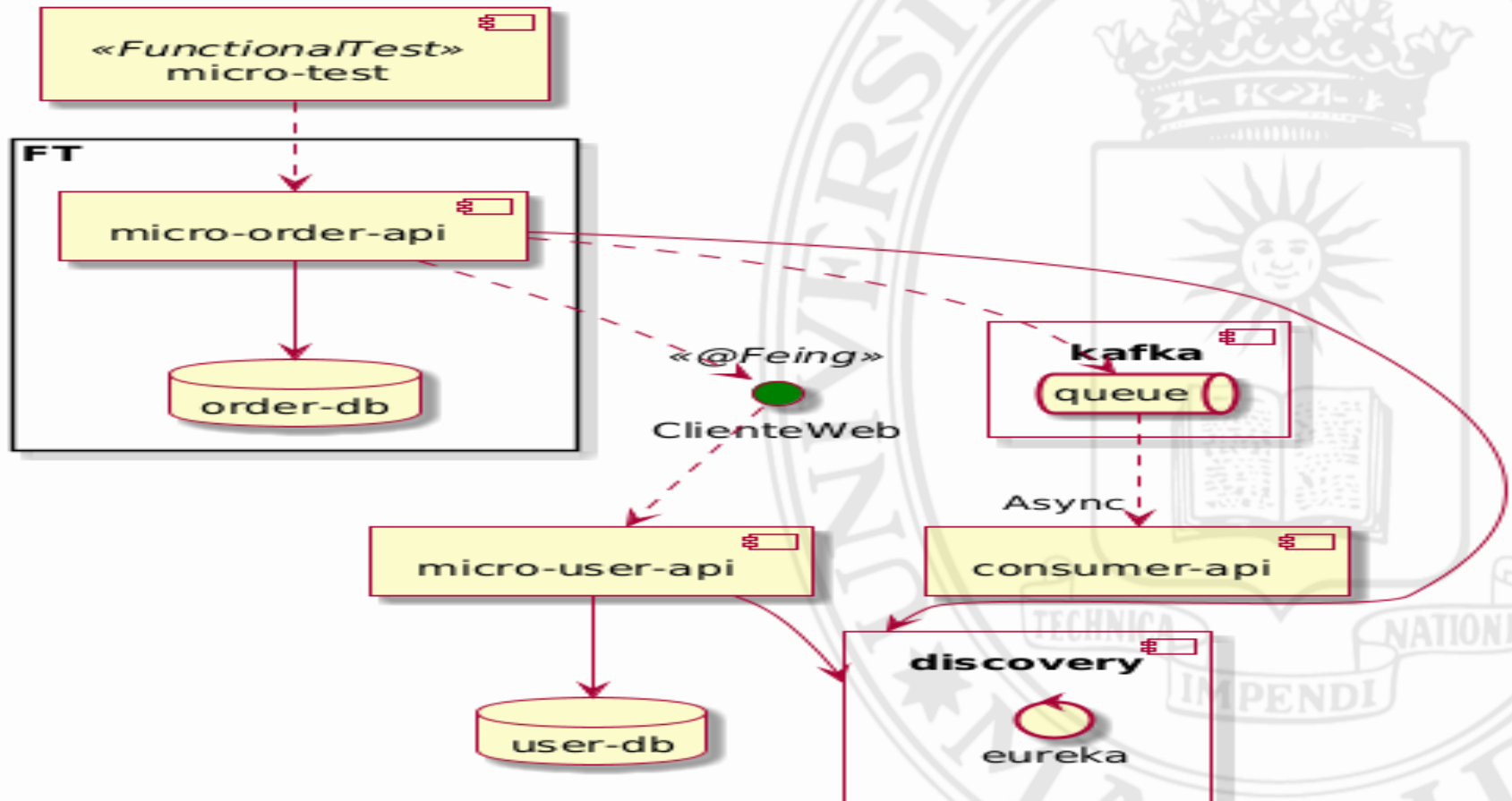
Fabricantes

- **Eureka**. Es de *Netflix OSS* y diseñado para entornos de microservicios *Spring Cloud*. Licencia *Apache 2.0*. Optimizado para AWS y Kubernetes
- **Zookeeper**. Es de *Apache Software Foundation*. Licencia *Apache 2.0*.
- **Consul**. Es de *HashiCorp*. Licencia: *MPL 2.0*.



Microservicio. Service Discovery

- *micro-user-api: release-1.2.0, micro-order-api: release-1.2.0, micro-eureka: release-1.2.0*



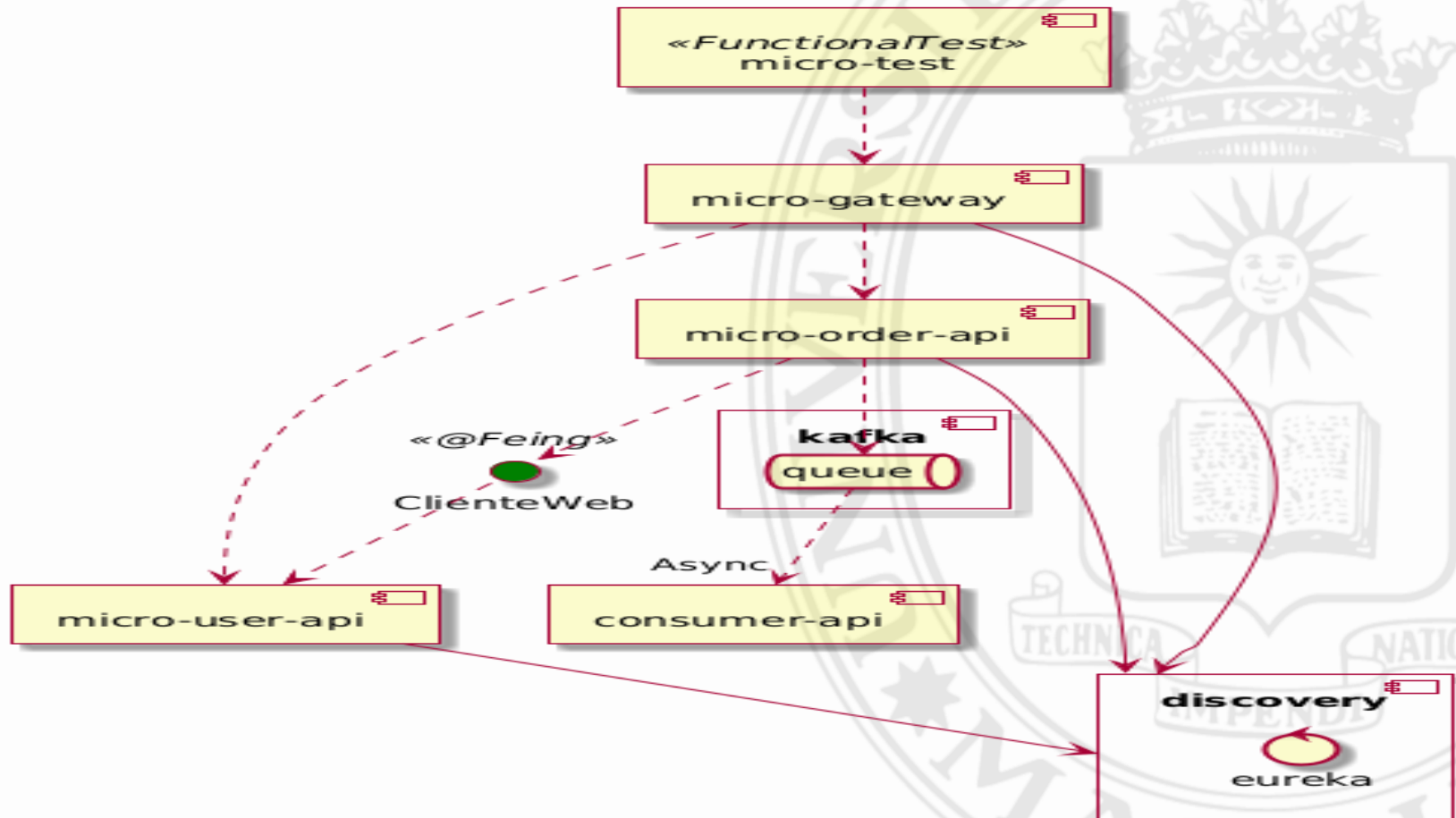
Gateway

- **Enrutamiento Inteligente.** Redirige peticiones a los microservicios correctos (por URL, headers, etc.).
- **Balanceo de Carga.** Distribuye tráfico entre varias instancias de un microservicio.
- **Seguridad y Autenticación.** Implementa autenticación con OAuth2, JWT, API Keys, etc.
- **Control de Versiones.** Permite manejar diferentes versiones de API en paralelo (/v1/users, /v2/users).
- **Monitorización y Logs.** Registra todas las peticiones y respuestas. Existen herramientas externas: **Prometheus, Zipkin y Grafana**
- **Caché.** Reduce la carga en microservicios almacenando respuestas temporales.
- **Transformación de Peticiones/Respuestas.** Puede modificar headers, cambiar formatos (ej. XML a JSON), agregar metadatos. Permite **adaptar APIs sin modificar los microservicios**.
- **Gestión de Errores y Tolerancia a Fallos.** Implementa *Circuit Breakers* y respuestas predeterminadas en caso de error. **Evita caídas masivas** si un microservicio falla.



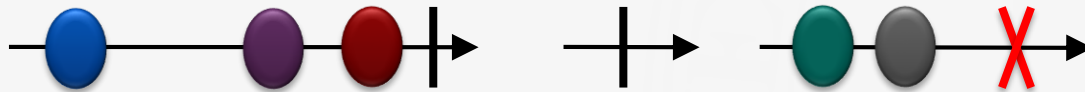
Microservicio. Service Discovery

- *micro-user-api: release-1.2.0, micro-order-api: release-1.2.0, micro-eureka: release-1.2.0*



Reactive Programming

- Flujo de datos asíncronos



- Programación Funcional
- Programación Asíncrona

RxJS



RxJS - Javascript

- Angular
- Clase: *Observable*
- <https://rxjs.dev/guide/overview>

Reactive

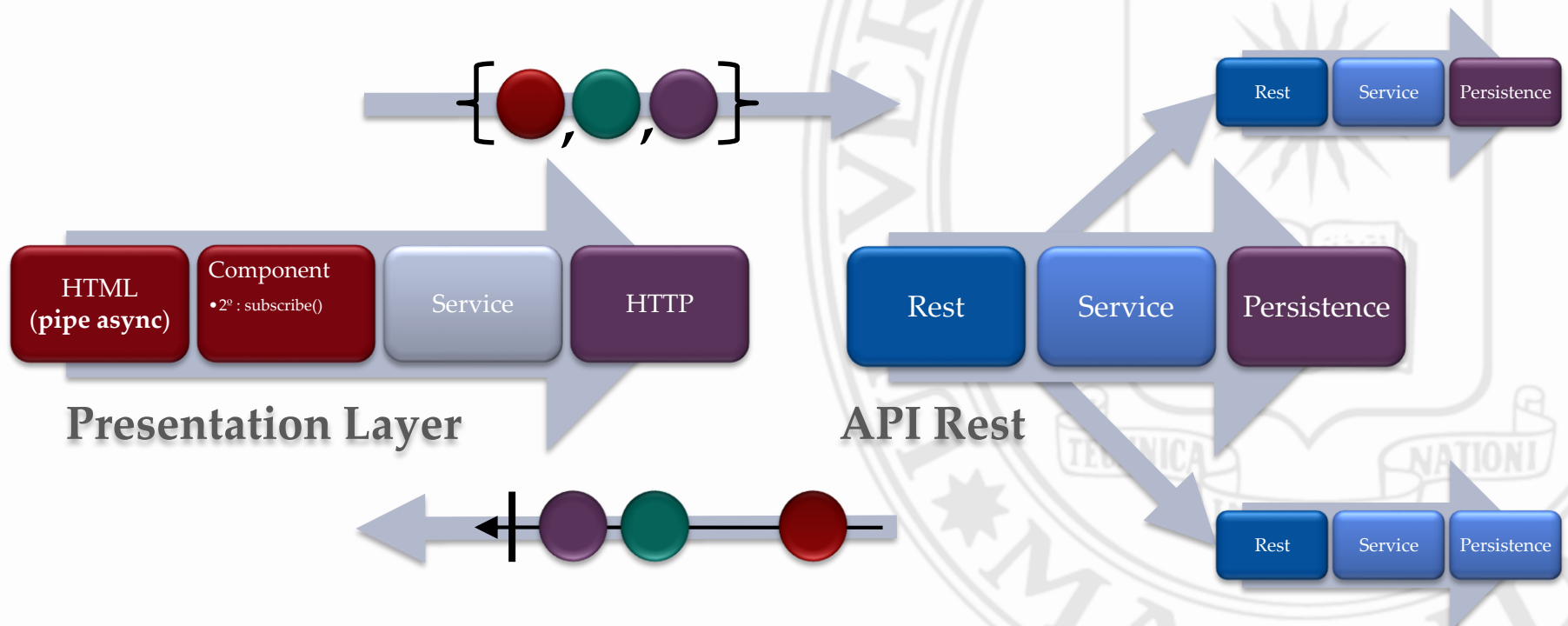


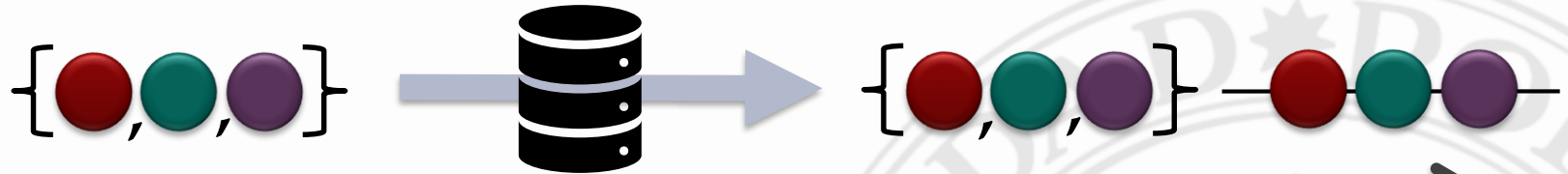
Project
Reactor

Project-reactor - Java

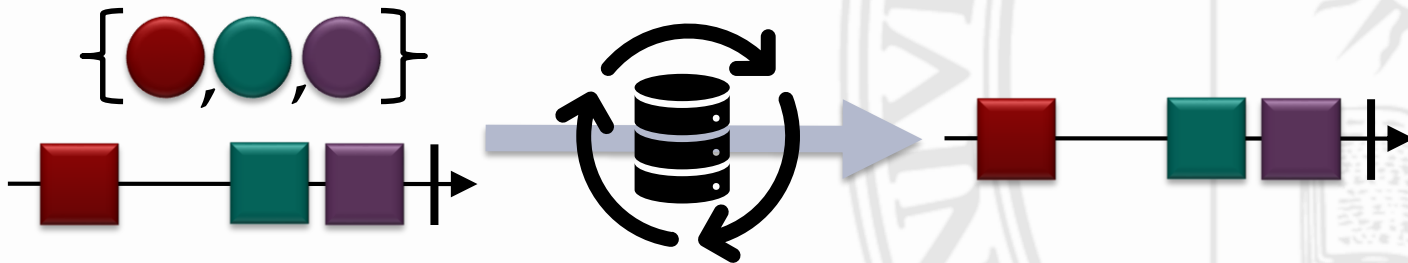
- Java - Spring
- Clases: *Mono & Flux*
- <https://projectreactor.io/docs/core/release/reference>
- <https://projectreactor.io/docs/core/release/api>

Métodos que devuelven un tipo *Publisher* no deberían subscribirse porque ello *podría romper la cadena del publicador*





```
public ArticleDto readArticleSynchronous(String code) {  
    return new ArticleDto(this.articleRepository.findById(code)  
        .orElseThrow(() -> new NotFoundException("Article code (" + code + ")"))  
    );  
}
```



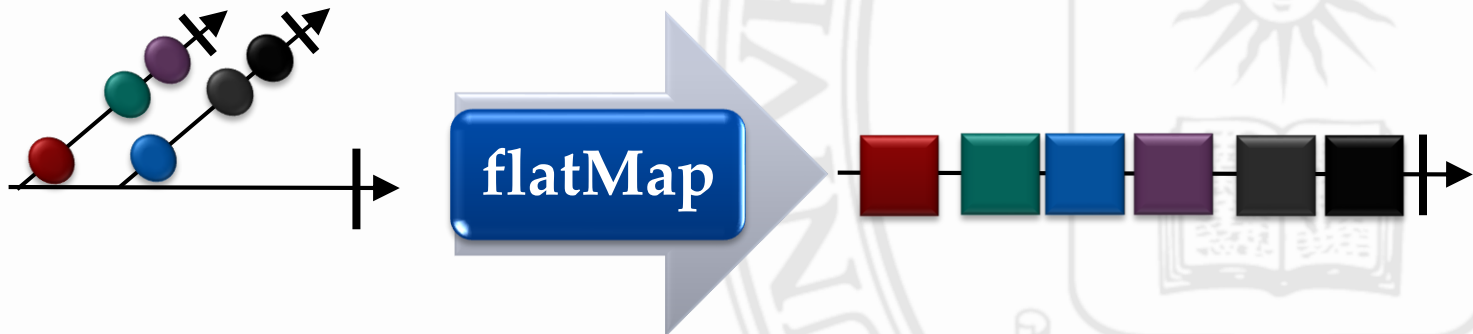
```
public Mono<ArticleDto> readArticle(String code) {  
    return this.articleReactRepository.findById(code)  
        .switchIfEmpty(Mono.error(new NotFoundException("Article code (" + code + ")")))  
        .map(ArticleDto::new);  
}
```

Programación Reactiva

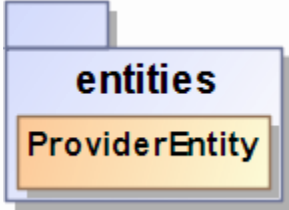
Funciones



```
public Flux<Integer> convertToInteger(Flux<String> flux) {  
    return flux.map(Integer::valueOf);  
}
```



```
public Flux<Integer> convertToIntegerFlatMap(Flux<String> flux) {  
    return flux.flatMap(string -> Mono.just(Integer.valueOf(string)));  
}
```



entities

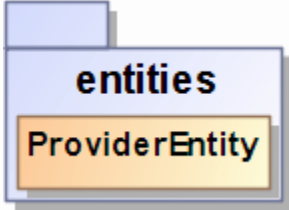
ProviderEntity

Reactive

- *map*: transforma un objeto en otro
- *flatMap*: aplana un flujo en objetos
- *switchIfEmpty*: cambia un flujo vacío por otro flujo

```
@Override
public Mono< Provider > readByCompany(String company) {
    return this.providerReactive.findByCompany(company)
        .switchIfEmpty(Mono.error(new NotFoundException("Non existent company: " + company)))
        .map(ProviderEntity::toProvider);
}
```

```
@Override
public Flux< Provider > findByCompanyAndPhoneAndNoteNullSafe(String company, String phone, String note) {
    return this.providerReactive.findByCompanyAndPhoneAndNoteNullSafe(company, phone, note)
        .map(ProviderEntity::toProvider);
}
```



entities

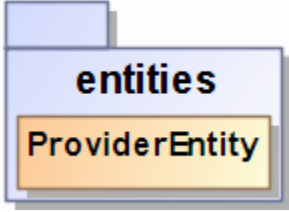
ProviderEntity

Reactive

- *map*: transforma un objeto en otro
- *flatMap*: aplanar un flujo en objetos
- *switchIfEmpty*: cambia un flujo vacío por otro flujo
- *mergeWith*: fusiona un flujo con otro
- *then()*: retorna un *Mono<Void>* cuando el flujo se completa
- *then(Mono<V>)*: retorna el *Mono* cuando el flujo se completa

```
private Mono< Void > assertCompanyAndNifNotExist(String company, String nif) {  
    return this.providerReactive.findByCompany(company) // Mono<ProviderCompany>  
        .mergeWith(this.providerReactive.findByNif(nif)) // Flux<ProviderCompany>  
        .flatMap(provider -> Mono.error(new ConflictException  
            ("Existing already company or nif : " + company + "," + nif)  
        )) // Flux<Object>  
        .then(); // Mono<Void>  
}
```

```
@Override  
public Mono< Provider > create(Provider provider) {  
    return this.assertCompanyAndNifNotExist(provider.getCompany(), provider.getNif()) // Mono<Void>  
        .then(this.providerReactive.save(new ProviderEntity(provider))) // Mono<ProviderEntity>  
        .map(ProviderEntity::toProvider); // Mono<Provider>  
}
```



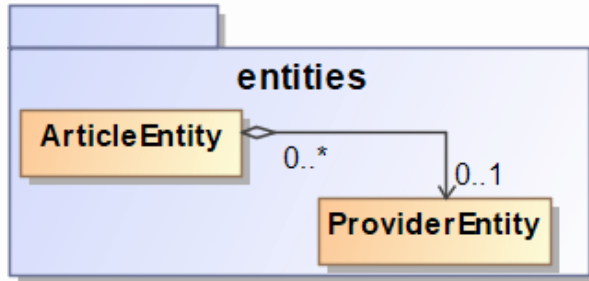
entities

ProviderEntity

Reactive

- *map*: transforma un objeto en otro
- *flatMap*: aplanar un flujo en objetos
- *switchIfEmpty*: cambia un flujo vacío por otro flujo
- *mergeWith*: fusiona un flujo con otro
- *then()*: retorna un *Mono<Void>* cuando el flujo se completa
- *then(Mono<V>)*: retorna el *Mono* cuando el flujo se completa

```
@Override
public Mono< Provider > update(String company, Provider provider) {
    return this.providerReactive.findByCompany(company)
        .switchIfEmpty(Mono.error(new NotFoundException("Non existent company: " + company)))
        .flatMap(providerEntity -> {
            BeanUtils.copyProperties(provider, providerEntity);
            return this.providerReactive.save(providerEntity);
        })
        .map(ProviderEntity::toProvider);
}
```

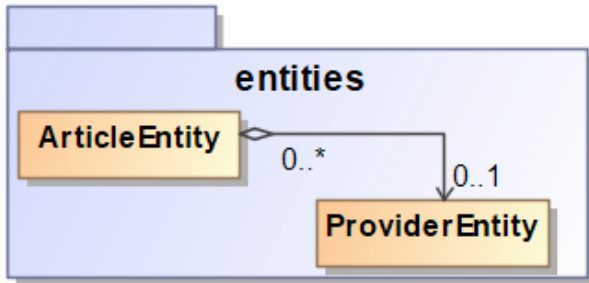


Reactive

- *map*: transforma un objeto en otro
- *flatMap*: aplanar un flujo en objetos
- *switchIfEmpty*: cambia un flujo vacío por otro flujo
- *mergeWith*: fusiona un flujo con otro
- *then()*: retorna un *Mono<Void>* cuando el flujo se completa
- *then(Mono<V>)*: retorna el *Mono* cuando el flujo se completa

```
@Override
public Mono< Article > readByBarcode(String barcode) {
    return this.articleReactive.findByBarcode(barcode)
        .switchIfEmpty(Mono.error(new NotFoundException("Non existent article barcode: " + barcode)))
        .map(ArticleEntity::toArticle);
}
```

```
@Override
public Flux< String > findBarcodeByBarcodeLikeAndNotDiscontinuedNullField(String barcode) {
    return this.articleReactive.findByBarcodeLikeAndNotDiscontinuedNullSafe(barcode)
        .map(ArticleEntity::getBarcode);
}
```

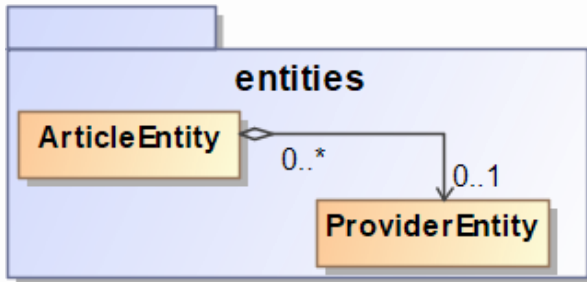


Create Article

- *Barcode* debe ser único
- *Provider* puede ser null, pero si no es null, debe existir
- *justOrEmpty*: crea un *Mono<V>*, si es null, crea un *Mono<Void>*

```
private Mono< Void > assertBarcodeNotExist(String barcode) {
    return this.articleReactive.findByBarcode(barcode)
        .flatMap(articleEntity -> Mono.error(
            new ConflictException("Article Barcode already exists : " + barcode)
        ));
}
```

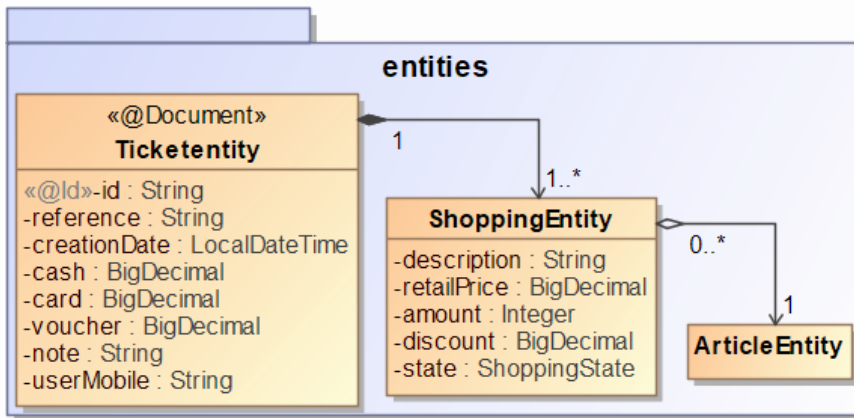
```
@Override
public Mono< Article > create(Article article) {
    return this.assertBarcodeNotExist(article.getBarcode())
        .then(Mono.justOrEmpty(article.getProviderCompany()))
        .flatMap(providerCompany -> this.providerReactive.findByCompany(article.getProviderCompany())
            .switchIfEmpty(Mono.error(
                new NotFoundException("Non existent company: " + article.getProviderCompany())
            ))
        )
        .map(providerEntity -> new ArticleEntity(article, providerEntity))
        .switchIfEmpty(Mono.just(new ArticleEntity(article, null)))
        .flatMap(this.articleReactive::save)
        .map(ArticleEntity::toArticle);
}
```



Reactive

- *map*: transforma un objeto en otro
- *flatMap*: aplana un flujo en objetos
- *switchIfEmpty*: cambia un flujo vacío por otro flujo
- *mergeWith*: fusiona un flujo con otro
- *then()*: retorna un *Mono<Void>* cuando el flujo se completa
- *then(Mono<V>)*: retorna el *Mono* cuando el flujo se completa
- *justOrEmpty*: crea un *Mono<V>*, si es null, crea un *Mono<Void>*

```
@Override
public Mono< Article > update(String barcode, Article article) {
    Mono< ArticleEntity > articleEntityMono;
    if (!barcode.equals(article.getBarcode())) {
        articleEntityMono = this.assertBarcodeNotExist(article.getBarcode())
            .then(this.articleReactive.findByBarcode(barcode));
    } else {
        articleEntityMono = this.articleReactive.findByBarcode(barcode);
    }
    return articleEntityMono
        .switchIfEmpty(Mono.error(new NotFoundException("Non existent article barcode: " + barcode)))
        .flatMap(articleEntity -> {
            BeanUtils.copyProperties(article, articleEntity);
            return this.providerReactive.findByCompany(article.getProviderCompany())
                .switchIfEmpty(Mono.error(
                    new NotFoundException("Non existent company: " + article.getProviderCompany())
                ))
                .map(providerEntity -> {
                    articleEntity.setProviderEntity(providerEntity);
                    return articleEntity;
                });
        })
        .flatMap(this.articleReactive::save)
        .map(ArticleEntity::toArticle);
}
```

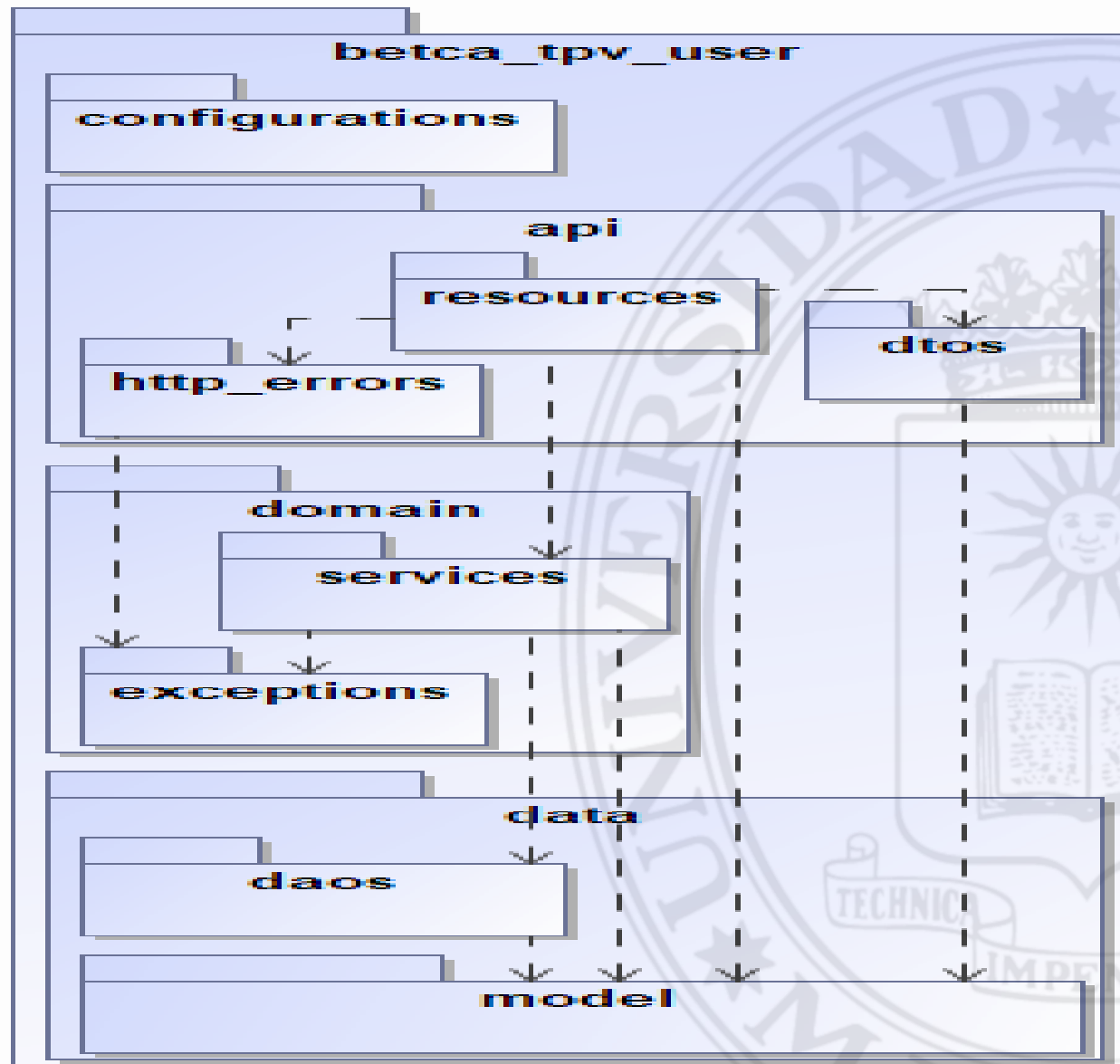



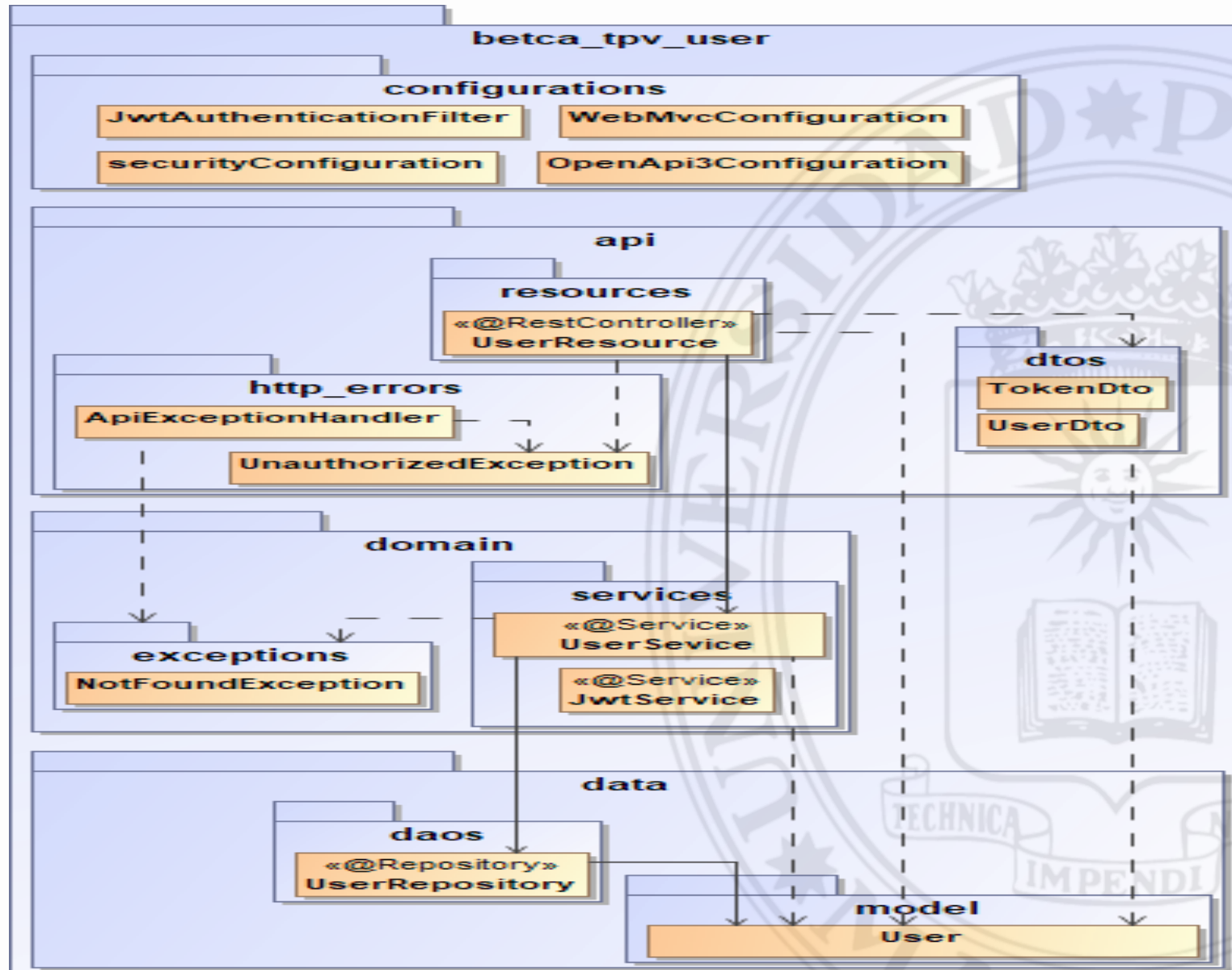
Create Ticket

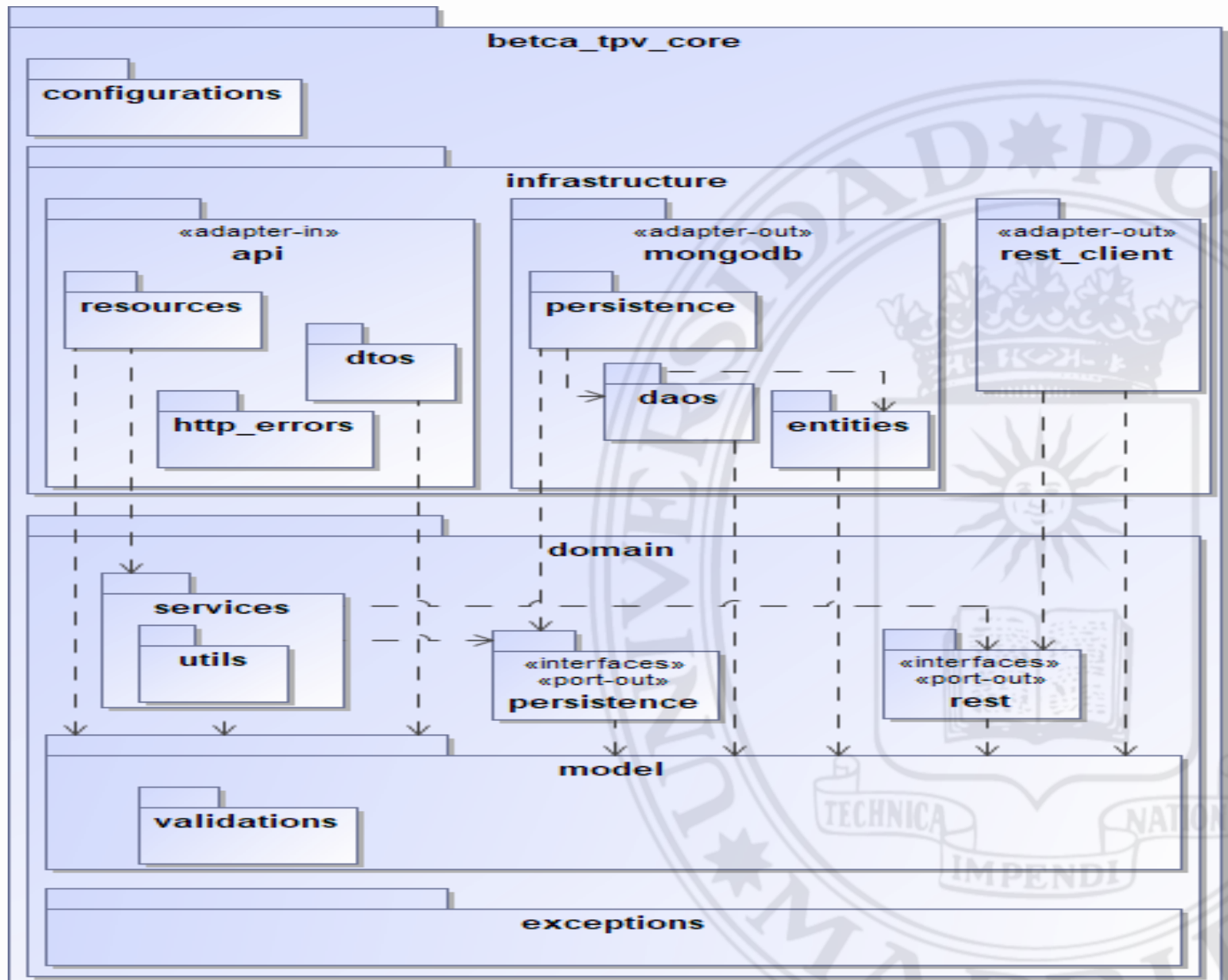
- Por cada *shopping* se debe mirar que exista el artículo, y obtener el *precio* y *descripción* del artículo
- *doOnNext*: añade comportamiento por cada item

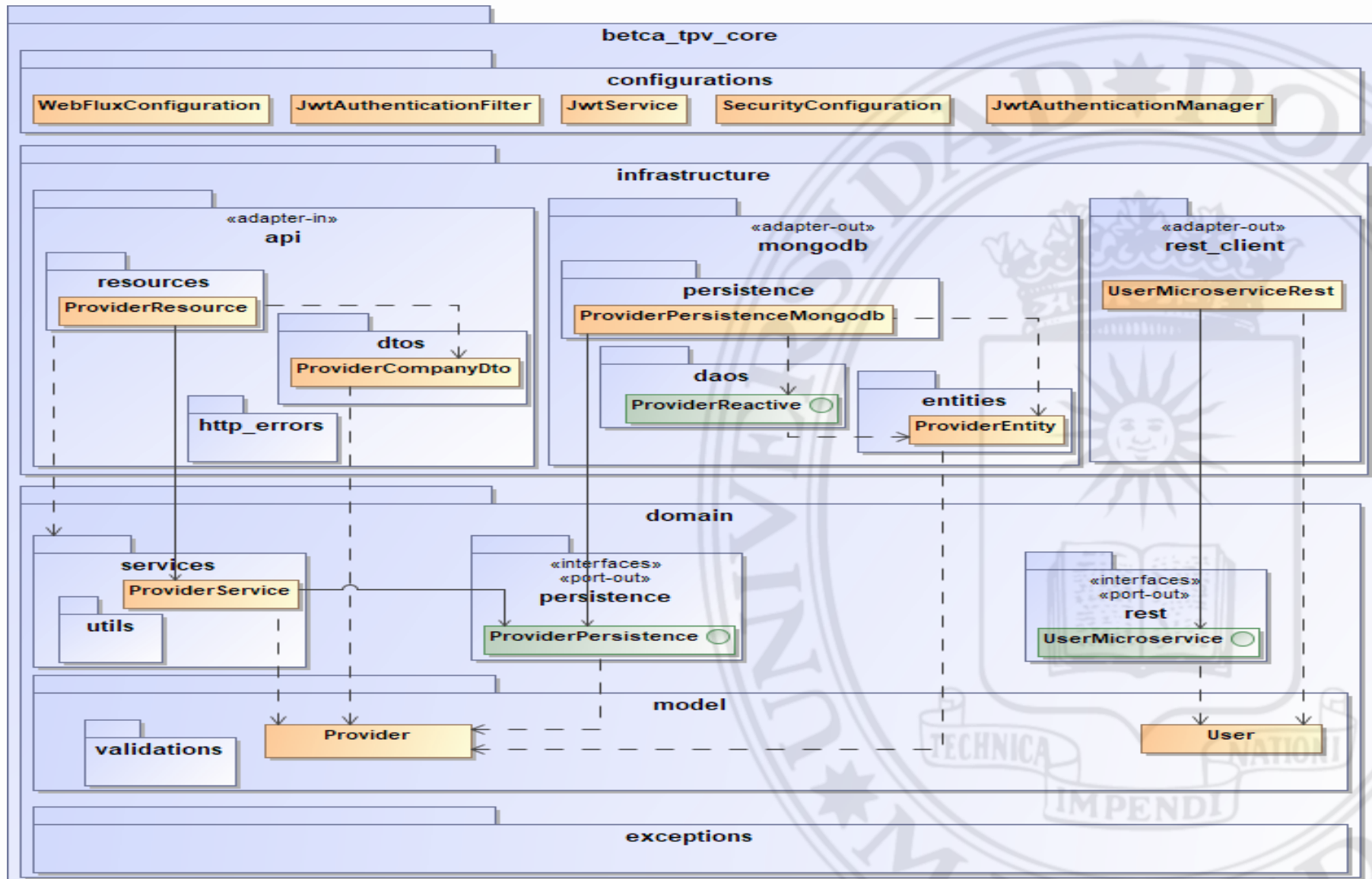
```
@Override
public Mono< Ticket > readById(String id) {
    return this.ticketReactive.findById(id)
        .map(TicketEntity::toTicket);
}
```

```
@Override
public Mono< Ticket > create(Ticket ticket) {
    TicketEntity ticketEntity = new TicketEntity(ticket);
    return Flux.fromStream(ticket.getShoppingList().stream())
        .flatMap(shopping -> {
            ShoppingEntity shoppingEntity = new ShoppingEntity(shopping);
            return this.articleReactive.findByBarcode(shopping.getBarcode())
                .switchIfEmpty(Mono.error(new NotFoundException("Article: " + shopping.getBarcode())))
                .map(articleEntity -> {
                    shoppingEntity.setArticleEntity(articleEntity);
                    shoppingEntity.setDescription(articleEntity.getDescription());
                    return shoppingEntity;
                });
        })
        .doOnNext(ticketEntity::add)
        .then(this.ticketReactive.save(ticketEntity))
        .map(TicketEntity::toTicket);
}
```









Python

Python es un lenguaje de programación de propósito general.

Esta desarrollado bajo *open-source*.

Fue creado por *Guido van Rossum* a final de los 80

Fuente

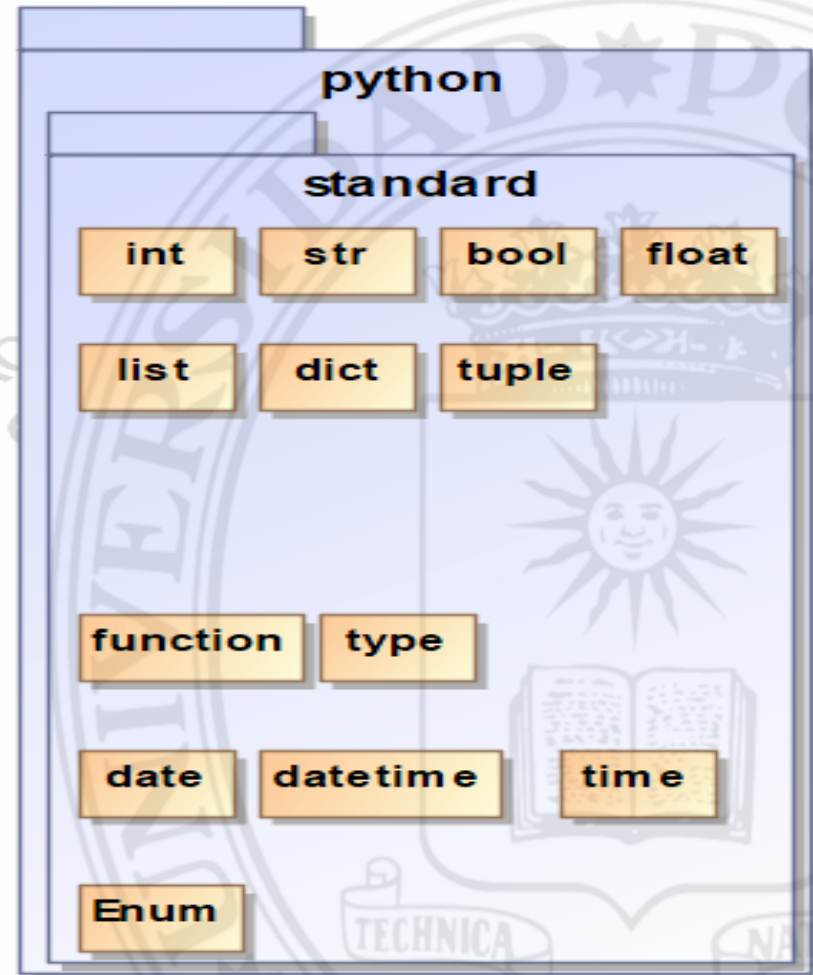
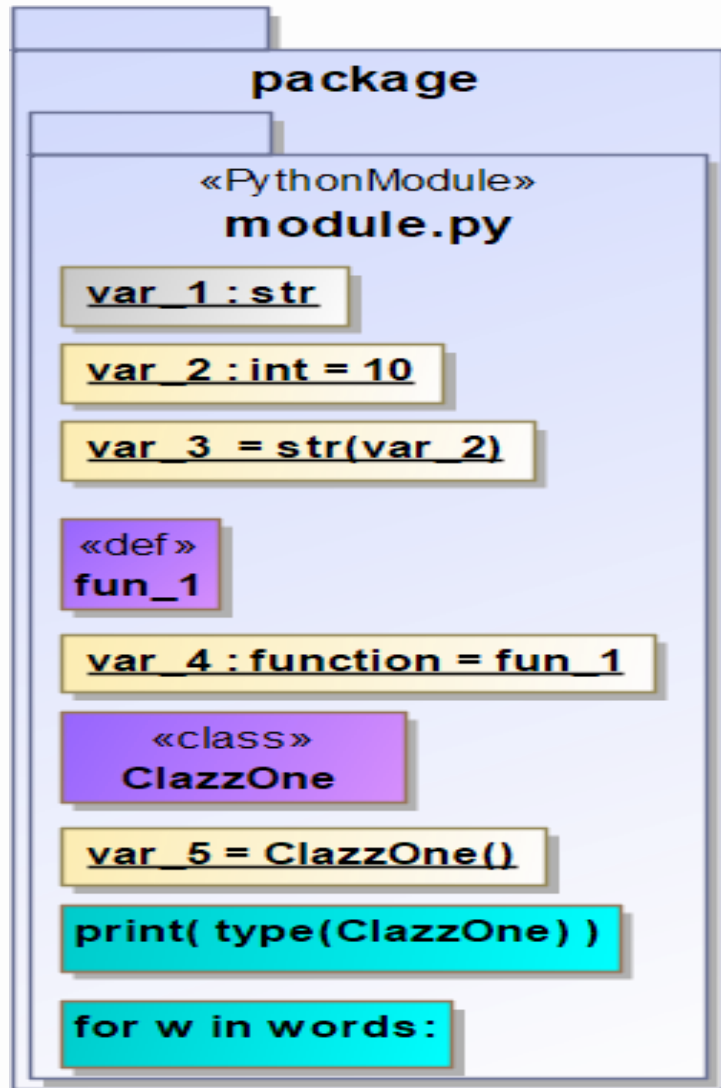
- Alto nivel: muy legible
- Propósito general
- Interpretado
- Multiplataforma
- Tipado
 - Estático
 - Dinámico

Multiparadigma

- Programación Funcional
- Programación Imperativa
- Programación Orientada a Objetos
- Programación Reflexiva
- Programación Asíncrona

Dependencias

- PIP: gestor de paquetes de Python
- Fichero *requirements.txt*: fichero con las dependencias
- PyPI
 - <https://pypi.org>
 - Repositorio de código



Instalar

- Python:
<https://www.python.org/downloads/>
- PyCharm:
<https://www.jetbrains.com/pycharm/>
- Crear un proyecto con entorno virtual:
 - `py -m venv <project-name\venv>`
 - `venv\Scripts\activate.bat`
 - `venv\Scripts\deactivate.bat`
- Abrir con PyCharm
 - Crear fichero: `hello_world.py`
 - Añadir: `print('Hello World')`
 - Ejecutar

Consola

- `Python --version`
- `py -V # py==python`
- `py -h # Help`
- `py -0 # available pythons`
- `pip list`
- `pip --version`
- `py -m pip --version # run pip library module as a script`
- `pip show pip # show pip version`
- `pip install -U <module> # update`
- `pip install -U pip # -U == --upgrade`
- `py -m pip uninstall <module>`
- `pip install -r requirements.txt`



<https://github.com/miw-upm/betca-Python>

- 1_hello_world.py
- 2_variables.py
- 3_operators.py
- 4_flows.py
- 5_dicts.py
- 6_functions.py
- 7_clases.py

 Clonar el proyecto

- *Ejecutar, modificar, aprender...*

unittest

- *Python standar library*
- Heredar de *TestCase*
- Similar a *JUnit*
- Consola: `>py -m unittest discover tests`

Ciclo de vida

- *@classmethod def setUpClass: antes de todos los tests*
- *def setUpself: antes de cada test*
- **def test_1: un test**
- *def tearDownself: después de cada test*
- *@classmethod def tearDownClass: después de todos los tests*

Asserts

- `self.assertEqual`, `self.assertNotEqual`, `self.assertTrue`, `self.assertFalse`, `self.assertIs` (same object), `self.assertIsNot`, `self.assertIsNone`, `self.assertIsNotNone`, `self.assertIn`, `self.assertNotIn`
- `with self.assertRaises(ZeroDivisionError):`
- `self.assertAlmostEqual`, `self.assertNotAlmostEqual`
- `self.assertGreater`, `self.assertGreaterEqual`, `self.assertLess`, `self.assertLessEqual`
- `self.assertRegex`, `self.assertNotRegex`

Otros ...

- `pytest` (★7k): <https://docs.pytest.org>

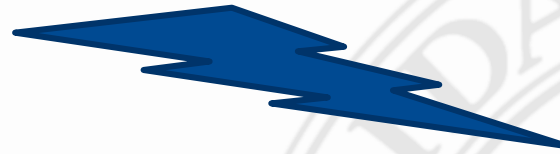
```
def helper_method():  
    print('--- helper:', method(666))
```

```
def method(arg):  
    print('>>> method... return:', arg)  
    return arg
```

```
class TestDemo(TestCase):  
    @mock.patch('tests.helper.method', return_value=0)  
    def test_two_a(self, mocked_method):  
        helper_method()  
        mocked_method.assert_called
```

```
class TestDemo(TestCase):  
    @mock.patch('tests.helper.method', side_effect=fake_method)  
    def test_two_b(self, mocked_method):  
        helper_method()  
        mocked_method.assert_called_with(666)
```

```
def fake_method(arg):  
    print('*** mock... arg:', arg)  
    return -111
```



```
class HelperClass:
    def helper(self):
        print('helper:',Clazz(111).class_method(666))
```

```
class Clazz:
    attribute: int
    def __init__(self, attribute):
        self.attribute = attribute
    def class_method(self, arg: int):
        print('>>> Clazz::method... arg:', arg)
        return self.attribute + arg
```

```
class TestDemo(TestCase):
    @mock.patch('tests.actual.Clazz.class_method', return_value=0)
    def test_five_a(self, mocked_method):
        HelperClass().helper()
        mocked_method.assert_called_once()
```

```
class TestDemo(TestCase):
    @mock.patch('tests.actual.Clazz.class_method', side_effect=fake_method)
    def test_five_b(self, method):
        HelperClass().helper()
        method.assert_called_with(666)
```

```
def fake_method(arg):
    print('*** mock... arg:', arg)
    return -111
```

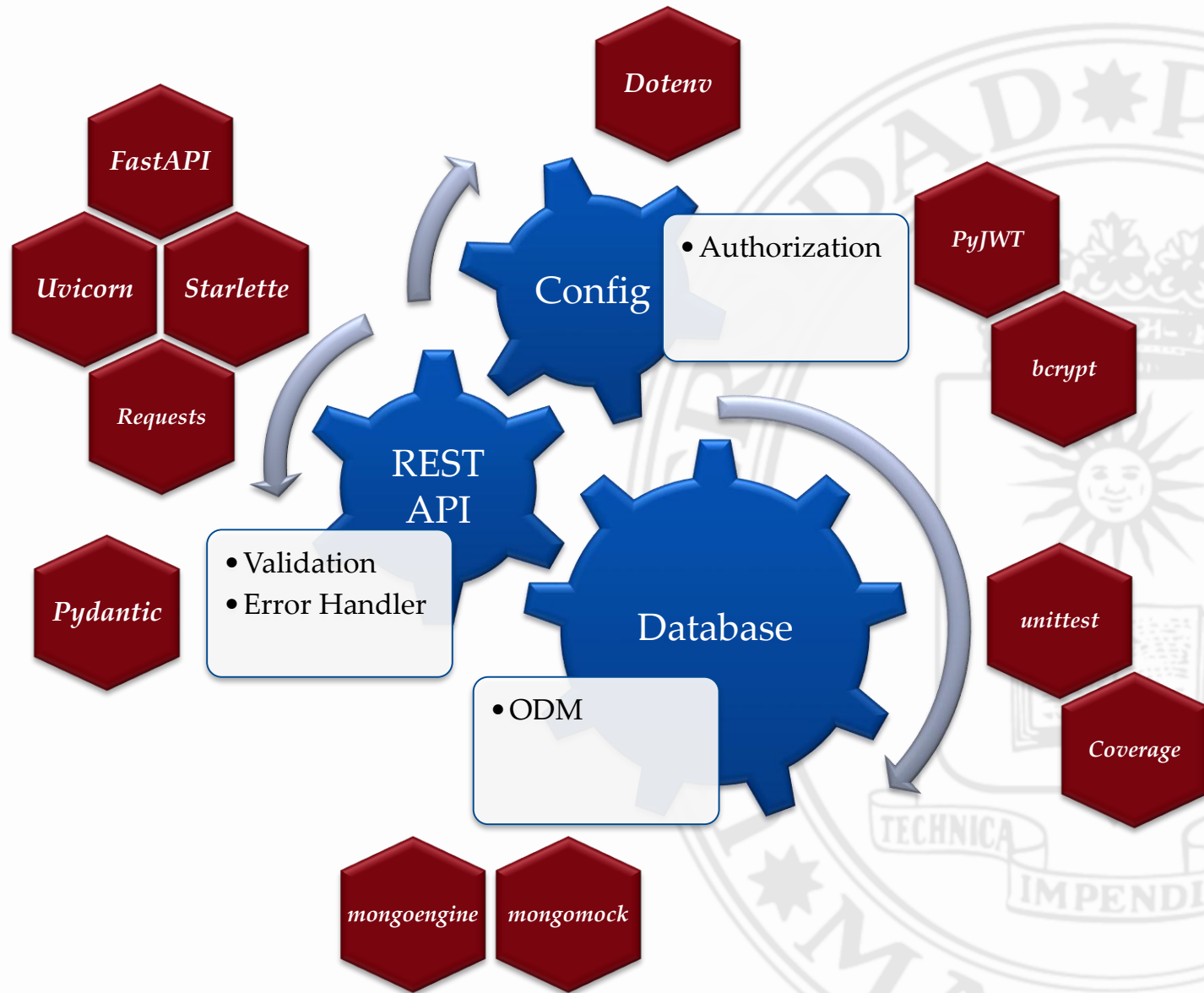


<https://github.com/miw-upm/betca-Python>

- `test_unittest.py`
- `actual.py`
- `helper.py`
- `test_mock.py`

 tests

- *Ejecutar, modificar, aprender...*





Django

- <https://www.djangoproject.com/>
- GitHub: ★55k
- Framework Web creado en 2003 y basado en MVC. Soporta ASGI (Asynchronous Server Gateway Interface)



Flask

- <https://flask.palletsprojects.com>
- GitHub: ★54k
- Microframework basado en WSGI (Web Server Gateway Interface) y Jinja



FastAPI

- <https://fastapi.tiangolo.com/>
- GitHub: ★26k
- Microframework, moderno y asíncrono.
- Basado en Starlette (uvicorn: ASGI) & Pydantic



Django REST Framework

- <https://www.django-rest-framework.org/>
- GitHub: ★20k



Flask-RESTful

- <https://flask-restful.readthedocs.io>
- GitHub: ★6k



Otros

- Tornado: ★20k
- Bottle: ★7k
- Pyramid: ★4k
- ...

FastAPI. <https://fastapi.tiangolo.com>

- Instalación:
 - `pip install fastapi`
 - `pip install uvicorn # ASGI server`
- Ejecución:
 - `uvicorn rest.main:app --reload --port 8085`
 - <http://localhost:8000/docs> (openAPI)
- Dependencias:
 - **Starlette** (Web). Es un framework/librería muy ligera ideal para construir servicios asíncronos: ASGI (Asynchronous Server Gateway Interface). <https://www.starlette.io/>
 - **Pydantic** (data). Validación de datos y gestion de configuración. <https://pydantic-docs.helpmanual.io/>
 - **Uvicorn** – Servidor Web ligero y rápido que implementa ASGI. <https://www.uvicorn.org/>

Python

REST. FastAPI

```
app = FastAPI(title='BETCA-Python')
app.include_router(basic)
```

```
from pydantic import BaseModel

class Dto(BaseModel):
    id: str
    name: str
    description: str
    value: Optional[int]
```

```
> uvicorn rest.main:app --reload --port 8085
```

```
basic = APIRouter(
    prefix="/basic",
    tags=["basic"], # OpenAPI
)

@basic.get("/search")
def find(name: str) -> [Dto]:
    return [Dto(id='1', name=name, description='desc'),
            Dto(id='2', name=name, description='desc2', value=2)]

@basic.post("")
def create(dto: Dto) -> Dto:
    return dto

@basic.get("/{ide}")
def read(ide: str) -> Dto:
    return Dto(id=ide, name="dto", description='desc')

@basic.put("/{ide}")
def update(ide: str, dto: Dto) -> Dto:
    dto.id = ide
    return dto

@basic.delete("/{ide}")
def delete(ide: str):
    print('Dto(', ide, ') deleted')
```

PUT /basic/666 {dto}

GET /basic/search?name=Miw

```
client = TestClient(app)

class TestService(TestCase):
    def test_read_basic(self):
        response = client.get("/basic/666")
        self.assertEqual(HTTPStatus.OK, response.status_code)
        self.assertEqual('666', response.json()['id'])
        self.assertEqual({'id': '666', 'name': 'dto', 'description': 'desc', 'value': None}, response.json())
        print('response: ', response.json())

    def test_create(self):
        response = client.post("/basic", json=Dto(id='1', name="dto", description='desc').dict())
        self.assertEqual(HTTPStatus.OK, response.status_code)
        print(response.json())

    def test_update(self):
        response = client.put("/basic/666", json={'id': '666', 'name': 'dto', 'description': 'desc'})
        self.assertEqual(HTTPStatus.OK, response.status_code)
        print(response.json())

    def test_delete(self):
        response = client.delete("/basic/666")
        self.assertEqual(HTTPStatus.OK, response.status_code)
```

```
class FruitEnum(str, Enum):
    pear = 'pear'
    banana = 'banana'

class ValidationDto(BaseModel):
    id: constr(min_length=1, max_length=10)
    name: constr(min_length=4, strip_whitespace=True)
    description: constr(regex=r'^miw-(betca|spring|python)$')
    value: Optional[conint(multiple_of=5)]
    adult: StrictBool
    color: Color
    fruit: FruitEnum
```

<https://pydantic-docs.helpmanual.io>

Constrained Types:

- NegativeFloat, NegativeInt, PositiveFloat, PositiveInt, conbytes, condecimal, confloat, conint, conlist, conset, constr

Strict Types

- StrictStr, StrictInt, StrictFloat, and StrictBool

Color Type, Datetime Types...

- 'black', '7fffd4', (255, 255, 255, 0.5), 'rgb(255, 255, 255)', 'hsl(270, 60%, 70%)'



<https://github.com/miw-upm/betca-python>

- **betca-rest**

Rest en práctica

- *basic_resource.py*
- Añadir end-point POST /basic/calc, recibe en el cuerpo *{name,value}* y hace un eco
- Añadir end-point PUT /basic/calc, recibe en el cuerpo *{id,name,value}* y hace un eco
- Añadir end-point PATCH /basic, recibe en el cuerpo *[{id,name}]* para actualizar
- *validation_resource.py*

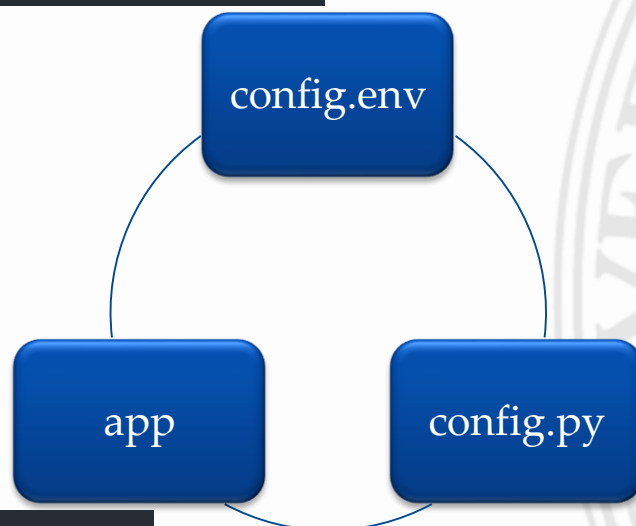
Python

Pydantic: Configuración

```
ENVIRONMENT=prod
```

```
DEV_VAR=value-on-dev  
DEV_JWT_SECRET=secret-to-test  
DEV_*
```

```
PROD_VAR=value-on-production  
PROD_JWT_SECRET=secret-${USERNAME}  
DEV_*
```



```
print(config.ENVIRONMENT)  
print(config.VAR)  
print(config.JWT_SECRET)
```

```
class Settings(BaseSettings):  
    ENVIRONMENT: str = Field(None, env="ENVIRONMENT")  
    VAR: Optional[str]  
    JWT_SECRET: Optional[str]
```

```
class Config:  
    env_file: str = "../config.env"
```

```
class DevSettings(Settings):  
    class Config:  
        env_prefix: str = "DEV_"
```

```
class ProdSettings(Settings):  
    class Config:  
        env_prefix: str = "PROD_"
```

```
def get_settings():  
    if "prod" == Settings().ENVIRONMENT:  
        return ProdSettings()  
    else:  
        return DevSettings()
```

```
config = get_settings()
```

```
def test_read_auth_ok(self):
    response = client.get("/auth/basic", auth=('jes', 'pass'))
    self.assertEqual(HTTPStatus.OK, response.status_code)
```

pyjwt: ★4k

bcrypt: ★1k

cryptography: ★4k

pycrypto: ★2k

```
def basic_authentication(credentials: HTTPBasicCredentials = Depends(HTTPBasic())) -> str:
    correct_username = secrets.compare_digest("jes", credentials.username) # avoid timing attacks
    bd_hashed_password = bcrypt.hashpw(b"pass", bcrypt.gensalt())
    correct_password = bcrypt.checkpw(credentials.password.encode('utf-8'), bd_hashed_password)
    if not (correct_username and correct_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect user or password",
        )
    return credentials.username

@auth.get("/basic")
def read_with_basic(username: str = Depends(basic_authentication)):
    return {"username": username}
```

Python

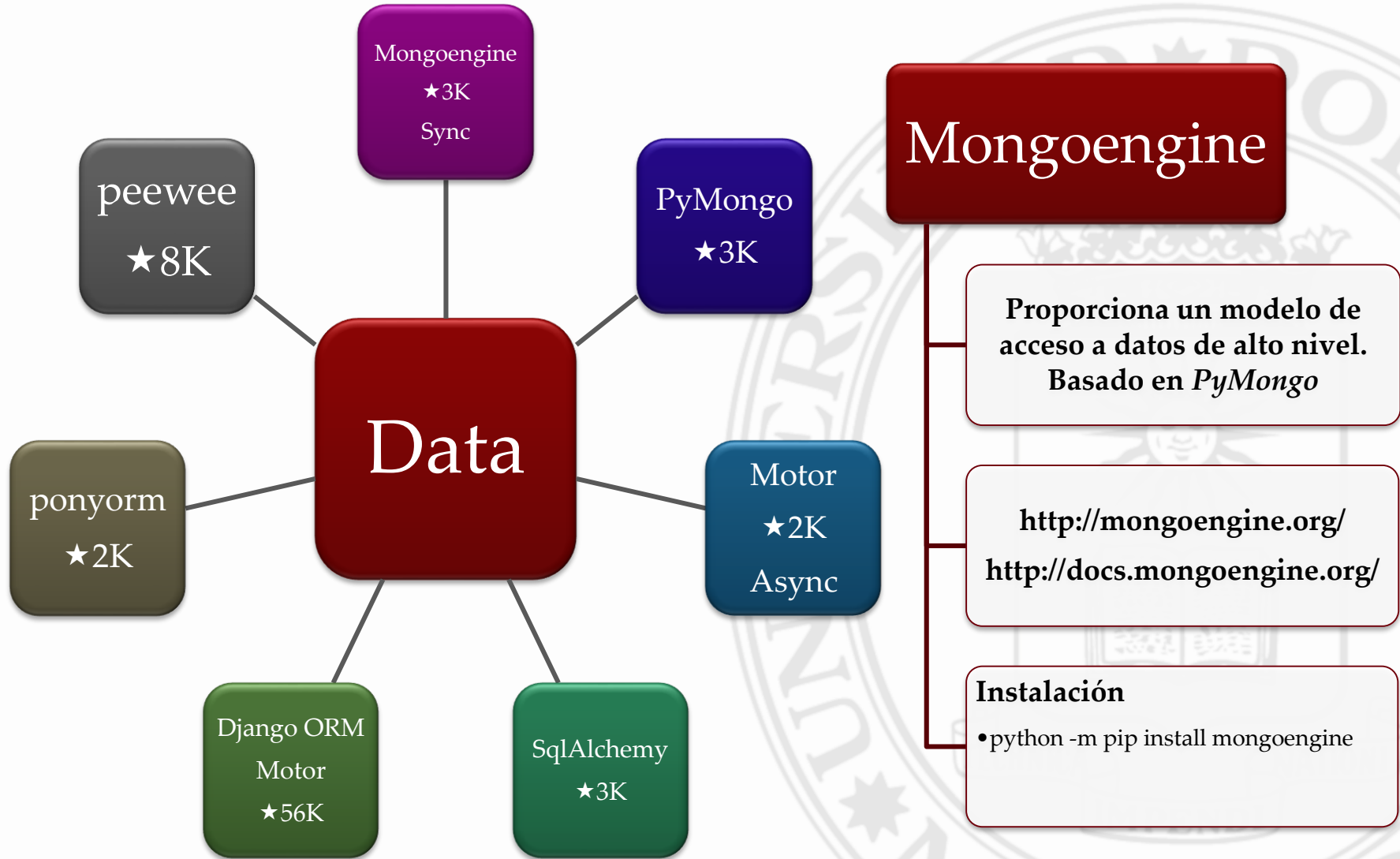
Seguridad. JWT

```
def test_create_token_and_get(self):
    response = client.post("/auth/bearer", auth=('jes', 'pass'))
    self.assertEqual(HTTPStatus.OK, response.status_code)
    token = response.json()['token']
    response = client.get("/auth/bearer", headers={"Authorization": "Bearer " + token})
    self.assertEqual(HTTPStatus.OK, response.status_code)
```

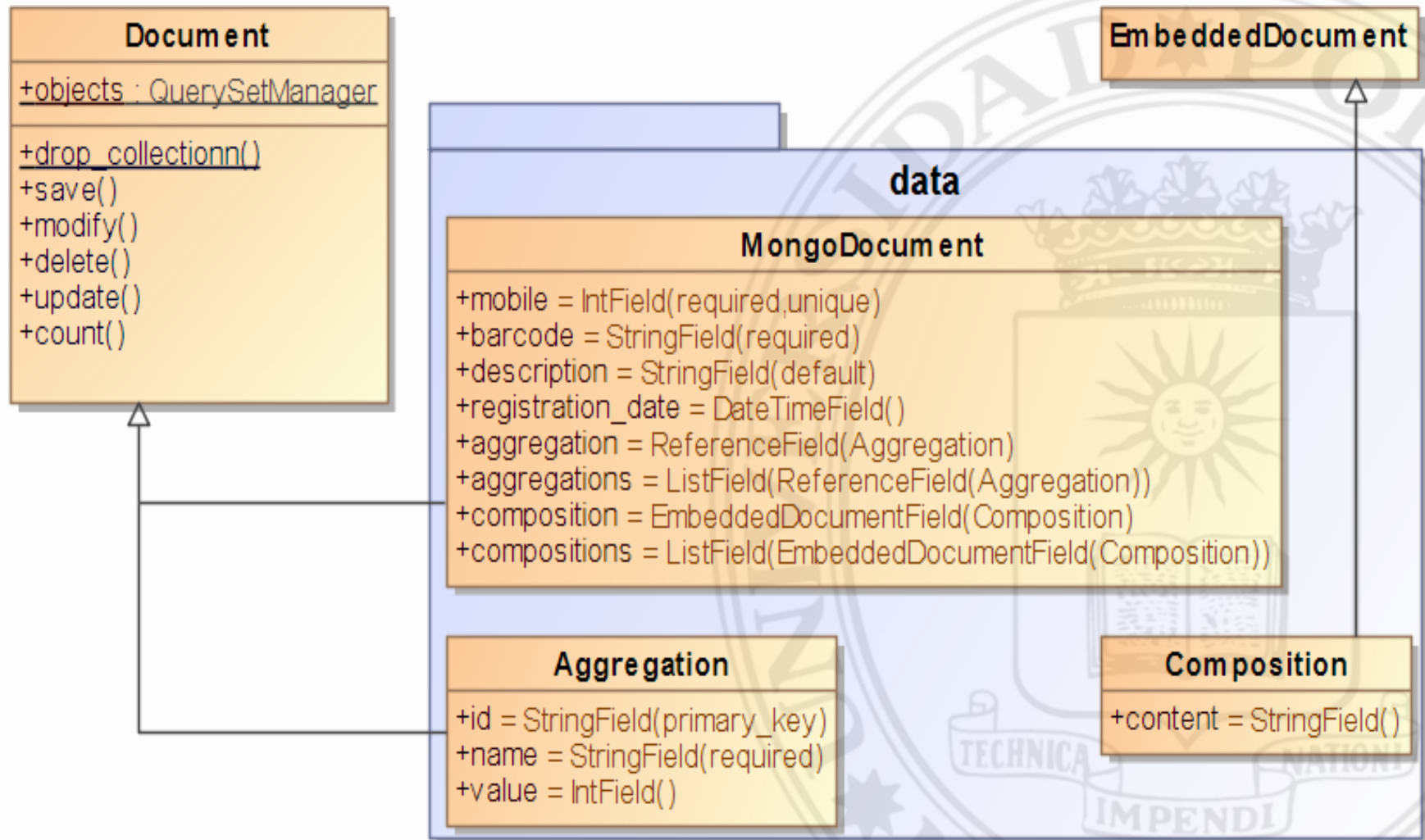
```
@auth.post("/bearer")
def create_jwt_token(username: str = Depends(basic_authentication)):
    expire = datetime.utcnow() + timedelta(minutes=60)
    encoded_jwt = jwt.encode({'sub': username, 'exp': expire, 'secret_key-Y01oEAL3iz', algorithm='HS256'})
    return {"token": encoded_jwt}

def bearer_authentication(credentials: HTTPAuthorizationCredentials = Depends(HTTPBearer())) -> str:
    try:
        payload = jwt.decode(credentials.credentials, 'secret_key-Y01oEAL3iz', algorithms=["HS512", "HS256"])
        username: str = payload.get("sub")
        if username is None:
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Non username")
        if username not in ["jes"]:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN, detail="Not enough permissions")
        return username
    except jwt.DecodeError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid token")
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="expired token")

@auth.get("/bearer")
def read_with_jwt_token(username: str = Depends(bearer_authentication)):
    return {"username": username}
```

Persistencia. Monoengine





<https://github.com/miw-upm/betca-python>

- **betca-rest**

 Rest en práctica

- *documents.py*

